

操作系统重点总结

目录

操作系统重点总结	1
进程与线程	3
进程	3
线程	11
处理机调度	15
调度	15
进程调度	18
调度算法	19
进程同步、进程互斥	25
同步、互斥问题	25
进程互斥的软件实现算法	26
进程互斥的硬件实现方法	29
信号量	30
信号量机制	30
信号量机制实现进程互斥	32
信号量机制实现进程同步	33
信号量模型	34
管程（封装思想）	40
死锁	41

死锁的处理策略	43
内存	46
内存的基础知识	46
内存保护	48
内存空间的扩充	48
内存空间的分配与回收	50
两级页表	57
基本分段存储管理	59
段页式管理方式	61
虚拟内存	62
页面置换算法	66
页面分配策略	69
文件	71
文件管理	71
文件的逻辑结构	72
文件目录	73
文件的物理结构	74
文件的基本操作	79
文件共享	82
文件保护	82
文件层次结构	83
磁盘	83

磁盘的结构.....	83
磁盘调度算法（减少寻道时间）.....	84
减少磁盘延迟时间.....	86
磁盘管理.....	87
IO.....	88
IO 控制器.....	88
IO 控制方式.....	90
IO 软件的层次.....	95
IO 核心子系统.....	96
假脱机技术 SPOOLING.....	96
设备的分配与回收.....	97
缓冲区管理.....	100

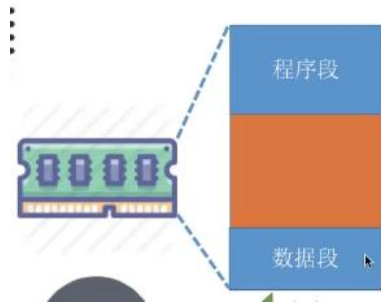
进程与线程

进程

程序：

就是一个指令序列 早期的计算机只支持单道程序

程序的代码放在程序段中，程序运行过程处理的数据放在数据段内（如变量）



程序段存放在内存的低地址位置，数据段存放在内存的高地址位置

CPU 执行指令时 到内存的低地址取得指令，按照指令执行，当寻找变量时，到内存的高地址取得数据

引入多道程序技术之后

为了方便操作系统管理，完成各种程序并发执行，引入了进程、进程实体的概念

系统为每个运行的程序配置一个数据结构（结构体），称为进程控制块 PCB。用来描述进程的各种信息（程序代码存放位置等）

PCB、程序段、数据段三部分组成了进程实体 → 简称**进程** 例如创建进程是创建进程实体中的 PCB

PCB 是进程存在的唯一标志

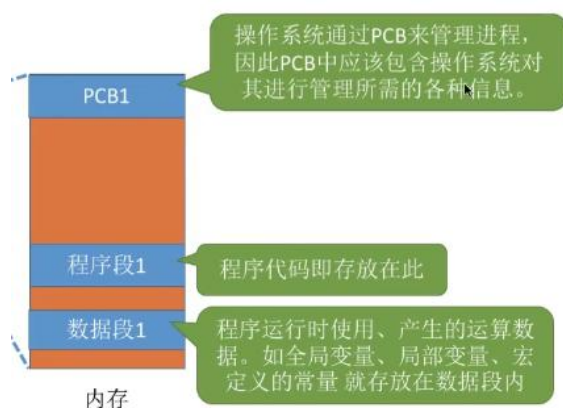
进程的定义

1. 进程是程序的一次**执行过程**
2. 进程是一个程序及其数据在处理机上顺序执行时**发生的活动**
3. 进程是具有独立功能的程序在数据集合上**运行的过程**。他是系统进行资源分配

和调度的一个独立单位

进程是一个动态的过程

进程是进程实体的运行过程 进程实体是静态的，进程则是动态的



PCB :

进程描述信息：进程标识符 PID 当进程被创建时，操作系统会为该进程分配唯一的一个不重复的 ID，用于区分不同的进程

用户标识符 UID

进程控制和管理信息：进程当前状态

进程优先级

资源分配清单：

处理及相关信息：各种寄存器值 当进程切换时需要把运行的情况记录下来保存在 PCB 中，如程序计数器的值表示了当前程序执行到哪一句

进程的组织

进程的组成讨论的是一个进程内部有哪些部分构成，进程的组织讨论的是多个进程之间的组织方式

进程的组织方式：链接方式： 按照进程状态将 PCB 分为多个队列

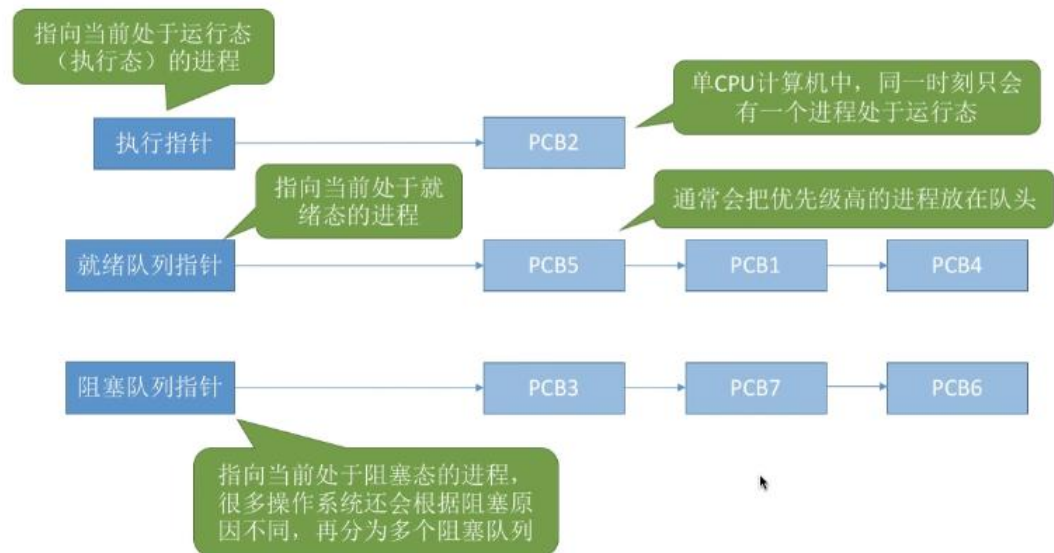
操作系统持有指向各个队列的指针

索引方式 根据进程状态的不同，建立几张索引表

操作系统持有指向各个索引表的指针



进程的组织——链接方式



进程的特征

动态性：进程是程序的一次执行过程，是动态的产生、变化和消亡。

进程的基本特性

并发性：内存中有多个进程实体，各进程可并发执行

独立性：进程是独立运行的，**进程是资源分配接受调度的基本单位**

异步性：个进程按各自独立的不可预知的速度向前推进 → **进程同步机制**

结构性：每个进程都有配置一个 PCB。从结构上看，进程由程序段、数据段、PCB

组成

进程的状态 → 三种基本状态

运行态：占有 CPU，并在 CPU 上运行

单核处理机，每一个时刻最多只有一个进程处于运行态

就绪态：已经具备运行条件，但由于 CPU 没有空闲，而暂时不能运行

进程已经具备了除处理机之外所有的资源

阻塞态：(等待态) 因等待一事件而暂时不能运行

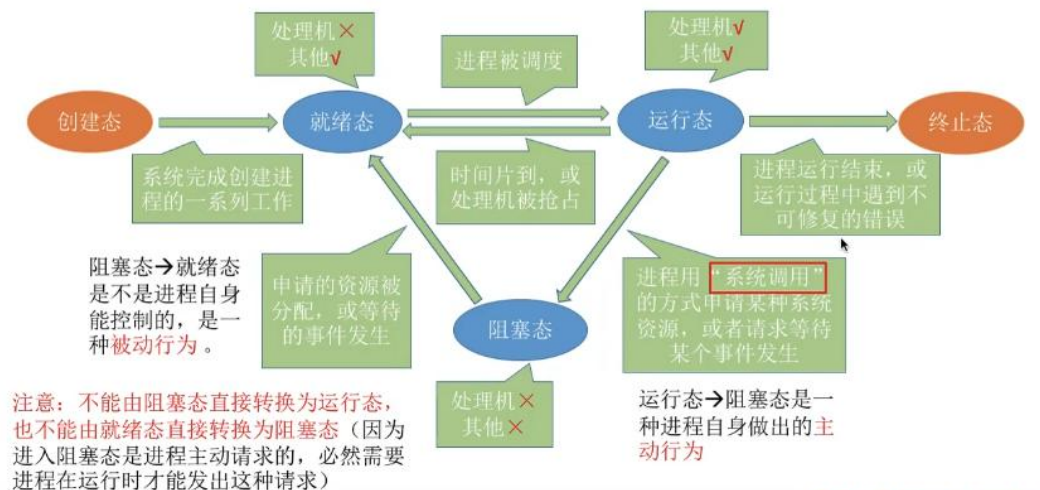
进程的状态 → 另外两种状态

创建态：操作系统需要完成创建进程。操作系统为该进程分配所需的内存空间等系统资源，并为其创建初始化 PCB（如为进程分配 PID）

终止态：进程运行结束或者 bug，需要撤销进程

操作系统需要完成撤销进程相关的工作。完成将分配给进程的资源回收，撤销进程 PCB 等工作

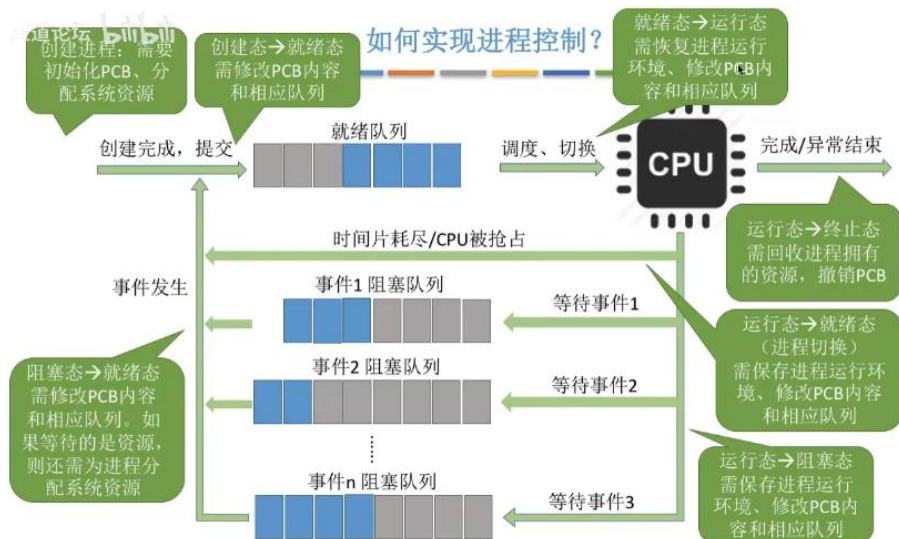
进程状态的转换



进程控制

进程控制的主要功能是对系统中的所有进程实施有效的管理，它具有创建新进程、

撤销已有进程、实现进程状态转换等功能 → 实现进程状态的转换



用原语实现进程控制。原语的特点是执行期间不允许被中断，只能一气呵成

原语采用“关中断指令”和“开中断指令”

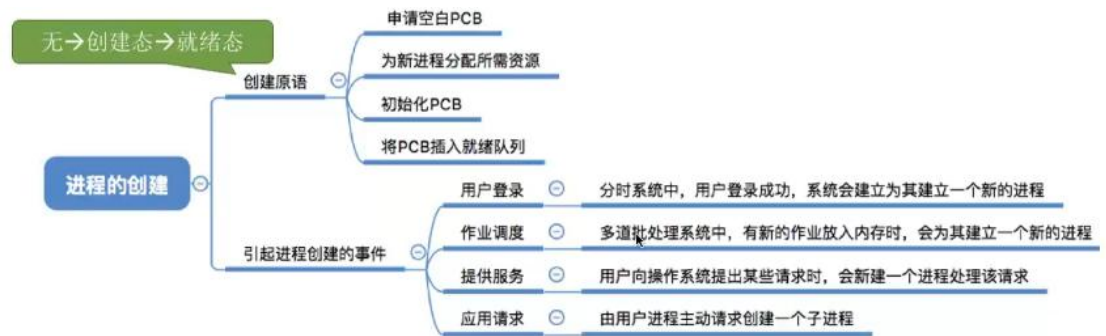


关/开中断的权限非常大，必须只允许在核心态下执行的特权指令

进程控制相关的原语

1. 更新 PCB 中的信息
 - a. 所有的进程控制原语一定都会修改进程状态标志
 - b. 剥夺当前运行进程的 CPU 使用权必然需要保存其运行环境
 - c. 某进程开始执行前必须要恢复其运行环境
2. 将 PCB 插入合适的队列
3. 分配/回收资源

无 → 创建态 → 就绪态



进程的相关原语中的阻塞原语和唤醒原语要成对出现

进程通信

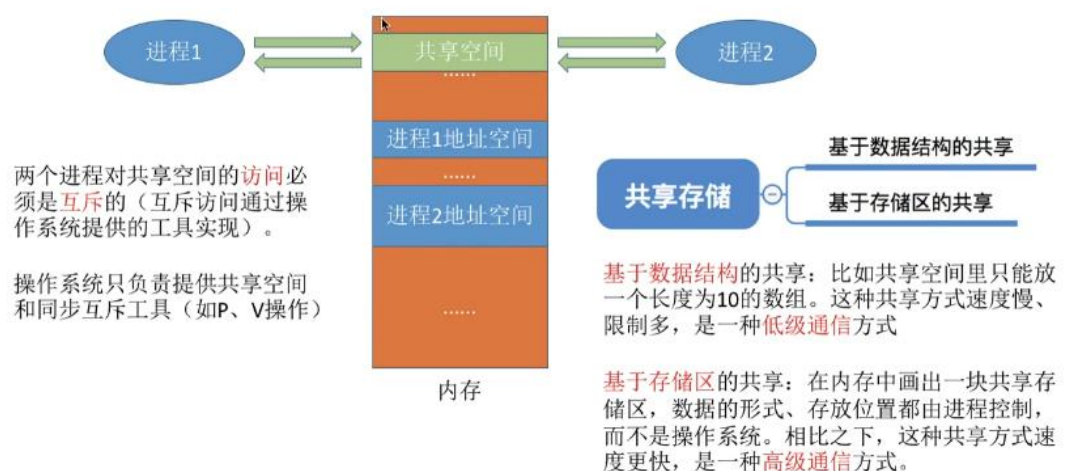
进程之间的信息交换

进程是分配系统资源的单位, 因此各进程拥有的内存地址空间相互独立 一个进程

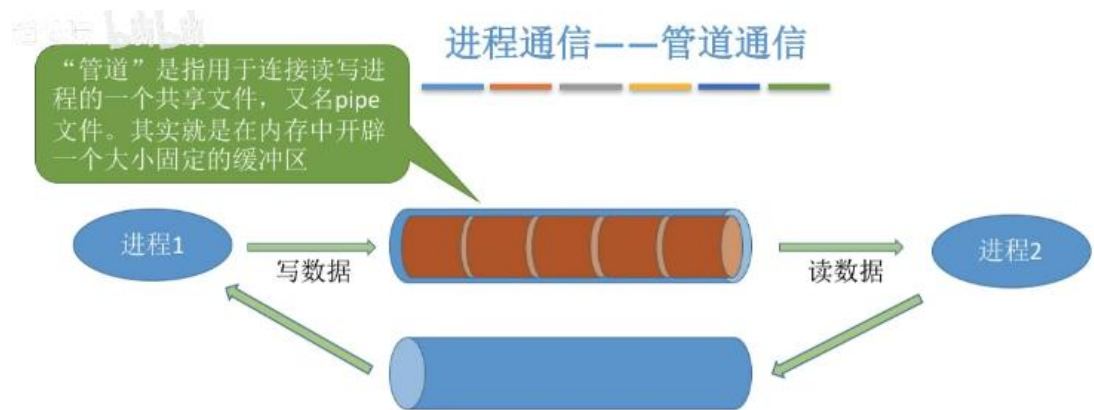
不能直接访问另一个进程的地址空间

共享存储、消息传递、管道通信

共享存储



管道通信

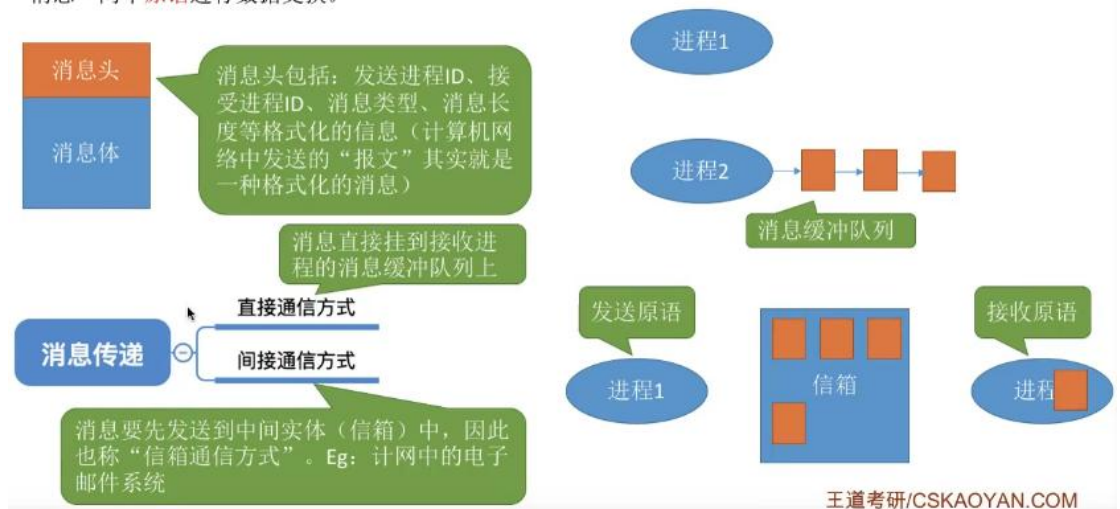


1. 管道只能采用半双工通信，某一时间段内只能实现单向的传输。如果要实现双向同时通信，则需要设置两个管道
2. 各进程要互斥地访问管道
3. 数据以字符流的形式写入管道，当管道写满时，写进程的 `write()` 系统调用将被阻塞，等待读进程将数据取走。当读进程将数据全部取走之后，管道变空，此时读进程的 `read` 系统调用将被阻塞
4. 如果没写满，就不允许读。如果没读空，就不允许写。Linux 中无此规定；
5. 数据一旦被读出，就从管道中被抛弃，这就意味着读进程最多只有一个。否则可能会有读错数据的情况

消息传递

进程间的数据交互以**格式化的消息**为单位。进程通过操作系统提供的“发送消息、接收消息”两个**原语**进行数据交换

进程间的数据交换以**格式化的消息（Message）**为单位。进程通过操作系统提供的“发送消息/接收消息”两个**原语**进行数据交换。

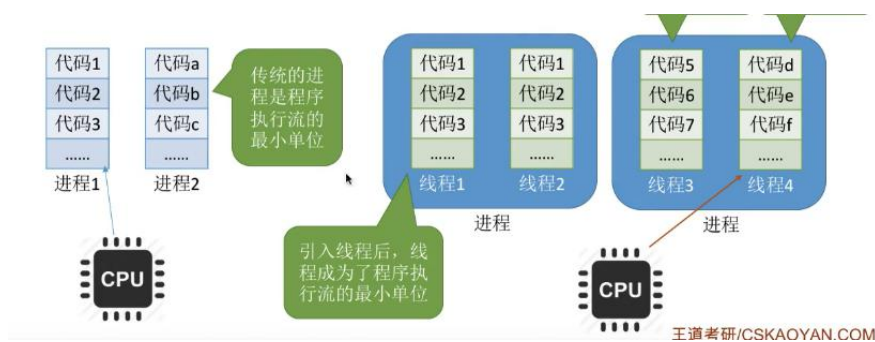


线程

引入线程的目的

有的进程可能需要“同时做很多事，而传统的进程只能串行地执行一系列程序，为此引入了“线程”地概念，来增加并发度

传统的进程是执行流的最小单位



引入线程后成为了程序执行流的最小单位

线程是一个最基本的 CPU 执行单位，也是程序执行流的最小单位

引入线程之后，不仅是**进程**之间可以并发，进程内的**各线程之间**也可以并发，从而进一步提高了**系统的并发度**。

引入线程后，**进程**只作为除 CPU 之外的**系统资源的分配单元**

进程是资源分配的基本单位，线程是调度的基本单位

线程的属性

线程是处理机调度的基本单位

多 CPU 计算机中，各个线程可以占用不同的 CPU

每个线程都有一个线程 ID、线程控制块（TCB）

线程也有就绪态、运行态、阻塞态三种基本单位

线程几乎不用有系统资源

同一进程的不同线程间共享进程的资源

由于共享内存地址空间，同一进程中的线程间通信甚至无需系统干扰

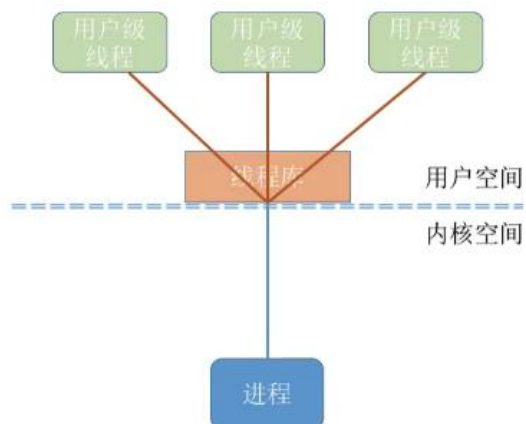
同一进程中的线程切换，不会引起进程切换

不同进程中的线程切换，会引起进程切换

切换同进程内的线程，系统开销很小；切换进程系统开销大

线程的实现方式

用户级线程



用户级线程由应用程序通过线程库实现。所有的线程管理工作都由应用程序负责
(包括线程切换)

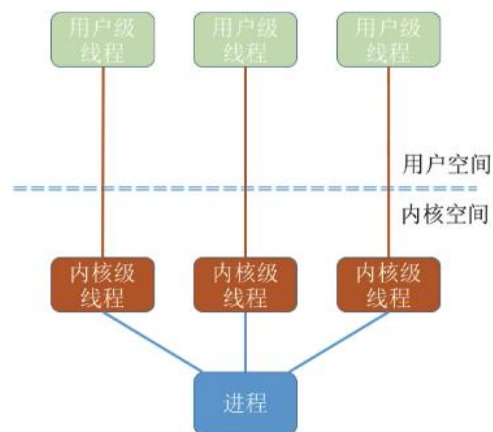
用户级线程中，线程切换可以在用户态下即可完成，无需操作系统干预。

在用户看来，是有多个线程。但是在操作系统内核看来，并意识不到线程的存在。

(用户级线程对用户不透明，对操作系统透明)

可以这样理解，“用户级线程”就是“从用户视角看能看到的线程”

内核级线程



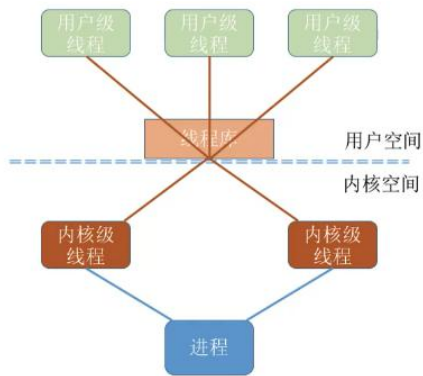
内核级线程的管理工作由操作系统内核完成。

线程调度、切换等工作都由内核负责，因此内核级线程的切换必然需要在核心态下才能完成。

可以这样理解，“内核级线程”就是“从操作系统内核视角看能看到的线程”

内核级线程才是处理机分配的单位

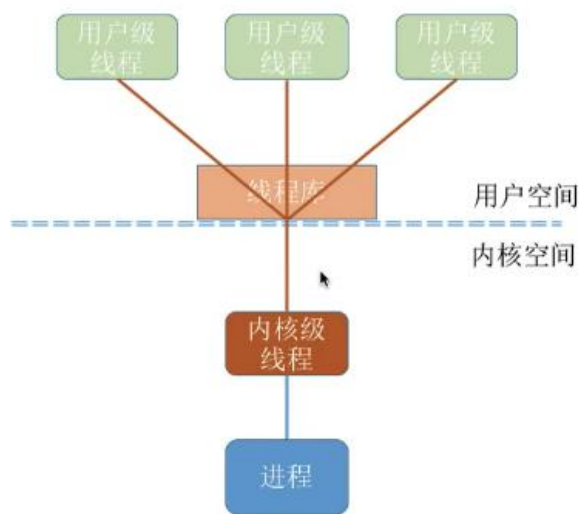
多线程模型



重点重点重点：
操作系统只“看得见”内核级线程，因此只有**内核级线程**才是处理机分配的单位。

例如：左边这个模型中，该进程由两个内核级线程，三个用户级线程，在用户看来，这个进程中有三个线程。但即使该进程在一个4核处理机的计算机上运行，也最多只能被分配到两个核，最多只能有两个用户线程并行执行。

多对一模型：

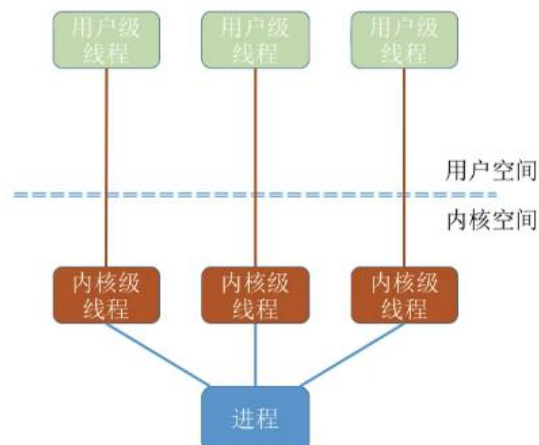


优点：用户级线程的切换在用户空间即可完成，不需要切换到核心态。线程管

理的系统开销小效率高

缺点：当一个用户级线程被阻塞后，整个进程都会被堵塞。并发度不高

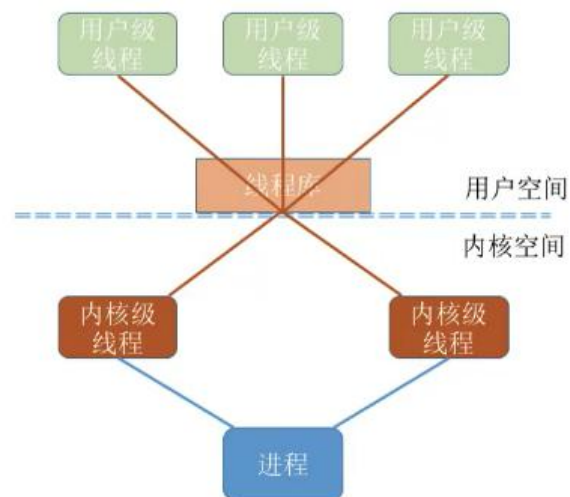
一对一模型：



优点：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强

缺点：一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，需要切换到核心态。成本高开销大

多对多模型：



及二者所长

处理机调度

调度

调度的基本概念

当有一堆任务要处理，但是由于资源有限，这些事情没法同时处理。这就需要确定某种规则来决定处理这些任务的顺序，这就是“调度”研究的问题

在多道程序系统中，进程的数量往往是多于处理机的个数，这样不可能同时并行的处理各个进程。

处理机调度 就是从就绪队列中按照一定的算法选择一个进程并将处理机分配给它运行，以实现进程的并发执行

调度的三个层次

由于内存空间有限,有时无法将用户提交的作业全部放入内存,因此就需要确定某种规则来决定将作业调入内存的顺序

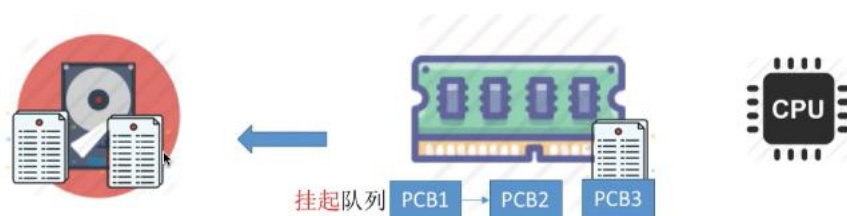
高级调度 (作业调度)。按照一定的原则从外存上处于后备队列的作业中挑选一个 (多个) 作业给他们分配内存等必要资源,并**建立相依的进程 (建立 PCB)**,以使他们**获得竞争处理及的权利**。高级调度主要是指调入的问题,因为只有调入的时机需要操作系统来确定,但调出的时机必然是作业运行结束才调出。

引入了虚拟存储技术之后,可将暂时不能运行的进程调至外存等待,等它重新具备了运行条件且内存又稍有空闲时,再**重新调入内存**

这么做的目的是为了提高内存利用率和系统吞吐量

暂时调到外存等待的进程状态称为**挂起状态**。**PCB** 不会一起调到外存,而是**会常驻内存**。PCB 中会记录进程数据在外存中的存放位置,进程状态等信息,操作系统通过内存中的 PCB 来保持对各个进程的监控、管理。被挂起的进程 PCB 会被放到**挂起队列**中

中级调度 (内存调度),就是要决定那个处于挂起状态的进程重新调入内存

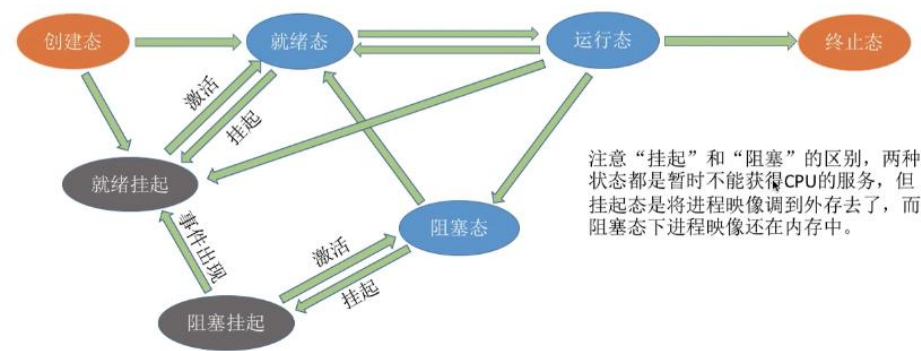


一个进程可能会被多次**调出、调入内存**,因此**中级调度**发生的频率要比高级调度**更**

高

七状态模型

暂时调到外存等待的进程状态为挂起状态（挂起态，suspend）
挂起态又可以进一步细分为就绪挂起、阻塞挂起两种状态
五状态模型 → 七状态模型



低级调度（进程调度），其主要任务是按照某种方法和策略从就绪队列中选取一个进程，将处理机分配给它。

进程调度使操作系统中最基本的一种调度，在一般的操作系统中都必须配置进程调度，进程调度的频率很高。

	要做什么	调度发生在..	发生频率	对进程状态的影响
高级调度（作业调度）	按照某种规则，从后备队列中选择合适的作业将其调入内存，并为其创建进程	外存→内存（面向作业）	最低	无→创建态→就绪态
中级调度（内存调度）	按照某种规则，从挂起队列中选择合适的进程将其数据调回内存	外存→内存（面向进程）	中等	挂起态→就绪态（阻塞挂起→阻塞态）
低级调度（进程调度）	按照某种规则，从就绪队列中选择一个进程为其分配处理机	内存→CPU	最高	就绪态→运行态

进程调度

进程调度的时机

进程调度（低级调度），按照一定的算法从就绪队列中选择一个进程为其分配处理

机



临界资源：一个时间段内只允许一个进程使用的资源，各进程需要互斥的访问临界资源

临界区：访问临界资源的那段代码

内核程序临界区：一般是用来访问某种内核数据结构的，比如进程的就绪队列



进程调度的方式

非剥夺调度方式（非抢占方式）：只允许进程主动放弃处理机。在运行过程中即便有紧

迫的任务到达，当前进程依然会继续使用处理机，直到该进程终止或主动要求进入阻塞态

剥夺调度方式 (抢占方式) : 当一个进程正在处理机上执行时, 如果有一个更重要的进程需要处理机, 立即暂停正在执行的进程, 将处理及分配给更重要紧迫的哪个进程

进程的切换与过程

狭义的进程调度指的是从**就绪队列**中选中一个要运行的过程

进程切换是指一个进程让出处理机, 由另一个进程占用处理机的过程

进程切换的过程主要完成了:

1. 对原来运行进程各种数据的保存
2. 对新的进程各种数据的恢复

进程切换是有代价的, 切换过于频繁的话, 必然会使整个系统的效率降低

调度算法

CPU 利用率: 指 CPU “忙碌” 的时间占总时间的比例

系统吞吐量: 单位时间内完成作业的数量

周转时间: 是指从作业被提交给系统, 到作业完成为止的时间间隔

等待时间: 作业处于等待处理机状态时间之和, 等待时间越长, 用户满意度低

响应时间: 从用户提交请求到首次产生相应所用的时间

先来先服务 (FCFS) **短作业优先 (SJF)** **高响应比优先 (HRRN)** **时间片轮转 (RR)**

优先级调度、多级反馈队列调度

先来先服务 (FCFS)

按照作业\进程到达的先后顺序进行服务

用于作业调度时，考虑的是那个作业先到达后备队列；用于进程调度时，考虑的是哪个进程先到达就绪队列，非抢占式算法

优点：公平、算法实现简单；缺点：对长作业有利，对短作业不利；

不会导致饥饿

短作业优先 (SJF)

最短的作业/进程优先得到服务（所谓“最短”，是指要求服务时间最短）

即可用于作业调度，也可用于进程调度；用于进程调度时称为“短进程优先算法 (SPF)”

SPF 与 SJF 是非抢占式的算法。但是也有抢占式的版本 → 最短剩余时间优先算法

短作业/进程优先调度算法：每次调度时选择当前已到达且运行时间最短的作业/进程

优点：“最短的”平均等待时间、平均周转时间；缺点：对短作业有利，对长作业不利

可能会产生饥饿现象。如果源源不断地有短作业/进程到来，可能使长作业/进程长时间得不到服务，产生“饥饿现象”。如果一直得不到服务，则称为“饿死”

高响应比优先 (HRRN)

要综合考虑作业/进程的等待时间和要求服务时间

在每次调度时，先计算各个作业/进程的响应比，选择响应比最高的作业/进程为其服务

既可以用于作业调度，也可以作用于进程调度

非抢占式的调度算法，只有当前运行的进程主动放弃 CPU 时，才需要进行调度

响应比 = (等待时间 + 要求服务时间) / 要求服务时间

优点：克服短作业优先缺点，响应比 $R = 1 + W/T$ ，其中 W 为等待时间， T 为执行时间，也就是短作业优先中，优先级随着等待时间而增加。

等待时间相同时，要求服务时间短的优先（SJF 的优点）

要求服务时间相同时，等待时间长的优先（FCFS 的优点）

对于长作业来说，随着等待时间越来越久，其响应比也会越来越大，从而

避免了长作业饥饿的问题

缺点：在同一时间段内处理的作业数少于短作业优先，因此吞吐量将小于短作业优先；由于每次调度前都要计算响应比，系统开销也要增加。

算法	思想&规则	可抢占?	优点	缺点	考虑到等待时间&运行时间?	会导致饥饿?
FCFS	自己回忆	非抢占式	公平；实现简单	对短作业不利	等待时间 $\sqrt{}$ 运行时间 $\times$	不会
SJF/S PF	自己回忆	默认为非抢占式，也有SJF的抢占式版本最短剩余时间优先算法（SRTN）	“最短的”平均等待/周转时间；	对长作业不利，可能导致饥饿；难以做到真正的短作业优先	等待时间 $\times$ 运行时间 $\sqrt{}$	会
HRRN	自己回忆	非抢占式	上述两种算法的权衡折中，综合考虑的等待时间和运行时间		等待时间 $\sqrt{}$ 运行时间 $\sqrt{}$	不会

时间片轮转调度算法（RR）

公平地、轮流地为各个进程服务，让每个进程在一定时间间隔内都可以得到响应

按照个进程到达就绪队列的顺序，轮流让各个进程执行一个时间片（100ms）若进程未在一个时间片内执行完，则剥夺处理机，将进程重新放到就绪队列队尾重新排队

用于进程调度（只有作业放入了内存建立了 PCB 后，才能被分配处理机时间片）

算法属于抢占式的算法。由时钟装置发出时钟中断来通知 CPU 时间片已到

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用时间片轮转调度算法，分析时间片大小分别是2、5时的进程运行情况。

进程	到达时间	运行时间
P1	0	5
P2	2	4
P3	4	1
P4	5	6

时间片轮转调度算法：轮流让就绪队列中的进程依次执行一个时间片（每次选择的都是排在就绪队列队头的进程）



0时刻（P1(5)）：0时刻只有P1到达就绪队列，让P1上处理机运行一个时间片
 2时刻（P2(4) → P1(3)）：2时刻P2到达就绪队列，P1运行完一个时间片，被剥夺处理机，重新放到队尾。此时P2排在队头，因此让P2上处理机。（注意：2时刻，P1下处理机，同一时刻新进程P2到达，如果在题目中遇到这种情况，默认新到达的进程先进入就绪队列）
 4时刻（P1(3) → P3(1) → P2(2)）：4时刻，P3到达，先插到就绪队尾，紧接着，P2下处理机也插到队尾
 5时刻（P3(1) → P2(2) → P4(6)）：5时刻，P4到达插到就绪队尾（注意：由于P1的时间片还没用完，因此暂时不调度。另外，此时P1处于运行态，并不在就绪队列中）

王道考研/CSKAOYAN.COM

时间片为 5 时：

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用时间片轮转调度算法，分析时间片大小分别是2、5时的进程运行情况。

进程	到达时间	运行时间
P1	0	5
P2	2	4
P3	4	1
P4	5	6

时间片轮转调度算法：轮流让就绪队列中的进程依次执行一个时间片（每次选择的都是排在就绪队列队头的进程）



时间片大小为 5

0时刻（P1(5)）：只有P1到达，P1上处理机。
 2时刻（P2(4)）：P2到达，但P1时间片尚未结束，因此暂不调度
 4时刻（P2(4) → P3(1)）：P3到达，但P1时间片尚未结束，因此暂不调度
 5时刻（P2(4) → P3(1) → P4(6)）：P4到达，同时，P1运行结束。发生调度，P2上处理机。
 9时刻（P3(1) → P4(6)）：P2运行结束，虽然时间片没用完，但是会主动放弃处理机。发生调度。
 10时刻（P4(6)）：P3运行结束，虽然时间片没用完，但是会主动放弃处理机。发生调度。
 15时刻（）：P4时间片用完，但就绪队列为空，因此会让P4继续执行一个时间片。
 16时刻（）：P4运行完，主动放弃处理机。所有进程运行完。

如果时间片太大，使得每个进程都可以在一个时间片内就完成，则时间片轮转调度算法

退化为先来先服务调度算法，并且会增大进程响应时间，因此时间片不能太大

如果时间片太小，会导致进程切换过于频繁，系统会花大量的时间来处理进程切换，从

而导致实际用于进程执行的时间比例减小

优点：公平、响应快、适用于分时操作系统

缺点：由于高频率的进程切换，因此有一定开销；不区分人物的紧急程度

不会导致饥饿

优先级调度算法

每个作业/进程有各自的优先级，调度时选择优先级最高的作业/进程

既可以用于作业调度，也可以用于进程调度

抢占式、非抢占式都有

非抢占式例题

例题：各进程到达就绪队列的时间、需要的运行时间、进程优先数如下表所示。使用非抢占式的优先级调度算法，分析进程运行情况。（注：优先数越大，优先级越高）



注：以下括号内表示当前处于就绪队列的进程

0时刻（P1）：只有P1到达，P1上处理机。

7时刻（P2、P3、P4）：P1运行完成主动放弃处理机，其余进程都已到达，P3优先级最高，P3上处理机。

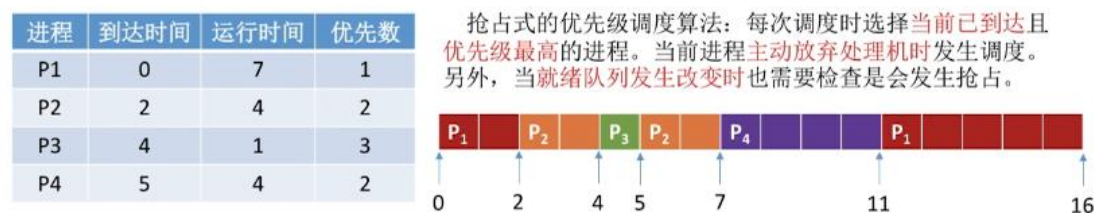
8时刻（P2、P4）：P3完成，P2、P4优先级相同，由于P2先到达，因此P2优先上处理机

12时刻（P4）：P2完成，就绪队列只剩P4，P4上处理机。

16时刻（）：P4完成，所有进程都结束

抢占式例题

例题：各进程到达就绪队列的时间、需要的运行时间、进程优先数如下表所示。使用抢占式的优先级调度算法，分析进程运行情况。（注：优先数越大，优先级越高）



注：以下括号内表示当前处于就绪队列的进程

0时刻（P1）：只有P1到达，P1上处理机。

2时刻（P2）：P2到达就绪队列，优先级比P1更高，发生抢占。P1回到就绪队列，P2上处理机。

4时刻（P1、P3）：P3到达，优先级比P2更高，P2回到就绪队列，P3抢占处理机。

5时刻（P1、P2、P4）：P3完成，主动释放处理机，同时，P4也到达，由于P2比P4更先进入就绪队列，因此选择P2上处理机

7时刻（P1、P4）：P2完成，就绪队列只剩P1、P4，P4上处理机。

11时刻（P1）：P4完成，P1上处理机

通常：系统进程优先级 高于 用户进程

前台进程优先级 高于 后台进程

操作系统更偏好 I/O 型进程

当采用动态优先级时，可以从追求公平、提高资源利用率调整优先级

当进程在就绪队列中等待了很长时间，则可以适当提高其优先级；反之

优点：用优先级区分紧急程度，适用于实时操作系统。可灵活的调整对各种作业的偏好程度

缺点：若源源不断有高优先级进程到来，可能会导致饥饿

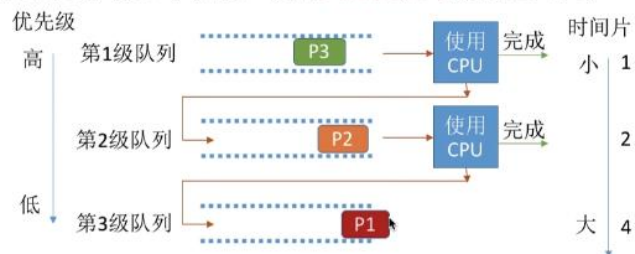
多及反馈队列调度算法

设置多级就绪队列，各级队列优先级从高到低，时间片从小到大

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

P1(1) → P2(1) → P1(2)
→ P2(1)



设置多级就绪队列，各级队列优先级从高到低，时间片从小到大

新进程到达时先进入第1级队列，按FCFS原则排队等待被分配时间片。若用完时间片进程还未结束，则进程进入下一级队列队尾。如果此时已经在最下级的队列，则重新放回最下级队列队尾

只有第k级队列为空时，才会为k+1级队头的进程分配时间片
被抢占处理机的进程重新放回原队列队尾

优点：对各类型进程相对公平 (FCFS 优点)；每个新到达的进程都可以很快就得到响应

(RR 的优点)；段进程只用较少的时间就可以完成 (SPF 优点)；不必实现估计进程的运行

时间；可灵活的调整对各类进程的偏好程度。

可能会导致饥饿；有源源不断的短进程到来时

算法	思想&规则	可抢占？	优点	缺点	会导致饥饿？	补充
时间片轮转		抢占式	公平，适用于分时系统	频繁切换有开销，不区分优先级	不会	时间片太大或太小有何影响？
优先级调度		有抢占式的，也有非抢占式的。注意做题时的区别	区分优先级，适用于实时系统	可能导致饥饿	会	动态/静态优先级。各类型进程如何设置优先级？如何调整优先级？
多级反馈队列	较复杂，注意理解	抢占式	平衡优秀 666	一般不说它有缺点，不过可能导致饥饿	会	

进程同步、进程互斥

同步、互斥问题

进程同步

进程具有异步性的特征，各并发执行的进程以各自独立的、不可预知的速度向前推进

进程通信 → 管道通信：读进程和写进程并发的执行，由于并发必然导致异步性，因此“写数据”和“读数据”两个操作执行的先后顺序是不确定的。而实际中 **写数据 → 读数据** 的顺序来执行。如何**解决这种异步问题**，这就是**进程同步**的内容

同步也称直接制约关系，它是指为完成某种任务而建立的两个或多个进程，这些进程因为需要在某些位置上**协调**他们的**工作次序**而产生的制约关系。

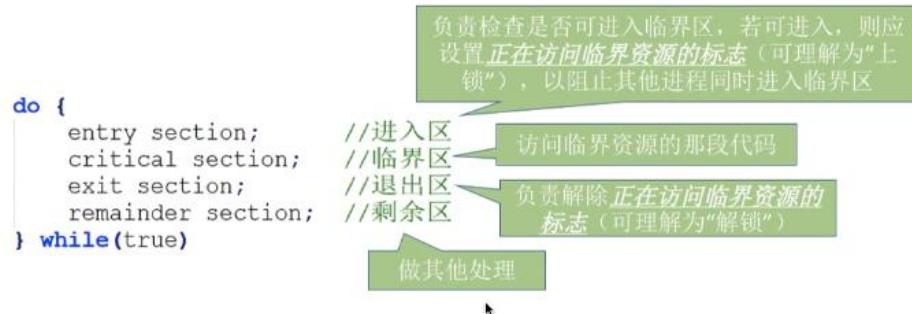
进程互斥

互斥共享方式：系统中的某些资源虽然可以提供给多个进程使用，但**一个时间段内只允许一个进程访问该资源**

我们把**一个时间段内只允许一个进程使用的资源称为临界资源**。

对临界资源的访问，必须**互斥**地进行。互斥，也称**间接制约关系**。进程互斥指当一个进程访问临界资源时，另一个想去访问临界资源的进程必须等待

对临界资源的互斥访问，可以在逻辑上分为如下四个部分：



临界区是进程中**访问临界资源**的代码段

进入区和**退出区**是**负责实现互斥**的代码段

临界区也可称问临界段

空闲让进：临界区空闲时，可以允许一个请求进入临界区的进程立即进入临界区

忙则等待：当已有进程进入临界区时，其他试图进入临界区的进程必须等待

有限等待：对请求访问的进程，应保证能在有限时间内进入临界区

让权等待：当进程不能进入临界区时，应当立即释放处理机，防止进程忙等待

进程互斥的软件实现算法

单标志法

两个进程在访问完临界区后会把使用临界区的权限转交给另一个进程。也就是说每个进程进入临界区的权限只能被另一个进程赋予

```
int turn = 0; //turn 表示当前允许进入临界区的进程号
```

P0 进程:

```
while (turn != 0); ①  
critical section; ②  
turn = 1; ③  
remainder section; ④
```

P1进程:

```
while (turn != 1); ⑤ //进入区  
critical section; ⑥ //临界区  
turn = 0; ⑦ //退出区  
remainder section; ⑧ //剩余区
```

turn 的初值为 0，即刚开始只允许 0 号进程进入临界区。

若 P1 先上处理机运行，则会一直卡在 ⑤。直到 P1 的时间片用完，发生调度，切换 P0 上处理机运行。代码 ① 不会卡住 P0，P0 可以正常访问临界区，在 P0 访问临界区期间即时切换回 P1，P1 依然会卡在 ⑤。只有 P0 在退出区将 turn 改为 1 后，P1 才能进入临界区。

因此，该算法可以实现“同一时刻最多只允许一个进程访问临界区”

对于临界区的访问，一定是按 P0→P1→P1.....

如果 P0 不进入临界区此时临界区空闲，但是并不允许 P1 访问

违背了空闲让进的原则

双标志先检查法

```
bool flag[2]; //表示进入临界区意愿的数组  
flag[0] = false;  
flag[1] = false; //刚开始设置为两个进程都不想进入临界区  
  
P0 进程: P1 进程:  
while (flag[1]); ① while (flag[0]); ⑤ //如果此时 P0 想进入临界区，P1 就一直循环等待  
flag[0] = true; ② flag[1] = true; ⑥ //标记为 P1 进程想要进入临界区  
critical section; ③ critical section; ⑦ //访问临界区  
flag[0] = false; ④ flag[1] = false; ⑧ //访问完临界区，修改标记为 P1 不想使用临界区  
remainder section; remainder section;
```

若按照 ①⑤②⑥③⑦....的顺序执行，P0 和 P1 将会同时访问临界区。

双标志先检查法的主要问题是：违反“忙则等待”原则

原因在于，进入区的“检查”和“上锁”两个处理不是一气呵成的。“检查”后，

“上锁”前可能发生进程切换

双标志后检查法

```
bool flag[2];           //表示进入临界区意愿的数组
flag[0] = false;
flag[1] = false;       //刚开始设置为两个进程都不想进入临界区

P0 进程:                P1 进程:
flag[0] = true; ①      flag[1] = true; ⑤ //标记为 P1 进程想要进入临界区
while (flag[1]); ②    while (flag[0]); ⑥ //如果 P0 也想进入临界区, 则 P1 循环等待
critical section; ③    critical section; ⑦ //访问临界区
flag[0] = false; ④    flag[1] = false; ⑧ //访问完临界区, 修改标记为 P1 不想使用临界区
remainder section;    remainder section;

若按照 ①⑤②⑥....的顺序执行, P0 和 P1 将都无法进入临界区
```

双标志后检查法虽然解决了“忙则等待”的问题,但是又违背了“空闲让进”和“有限等待”原则,会因各进程都长期无法访问临界资源而产生“饥饿”现象

Peterson 算法

```
bool flag[2];           //表示进入临界区意愿的数组, 初始值都是false
int turn = 0;           //turn 表示优先让哪个进程进入临界区

P0 进程:
flag[0] = true; ①
turn = 1; ②
while (flag[1] && turn==1); ③
critical section; ④
flag[0] = false; ⑤
remainder section;

P1 进程:
flag[1] = true; ⑥ //表示自己进入临界区
turn = 0; ⑦ //可以优先让对方进入临界区
while (flag[0] && turn==0); ⑧ //对方想进, 且最后一次是自己“让梨”, 那自己就循环等待
critical section; ⑨
flag[1] = false; ⑩ //访问完临界区, 表示自己已经不想访问临界区了
remainder section;
```

两种双标志法的问题都是由于进入区的几个操作不能一气呵成导致的。我们可以推理验证在 Peterson 算法中, 两个进程进入区中的各个操作按不同的顺序穿插执行会发生什么情况:

①②③⑥⑦⑧...
①⑥②③...

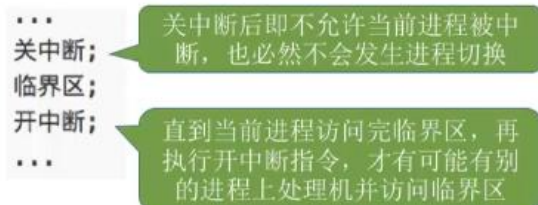
进入区

Peterson 算法用软件方法解决了进程互斥问题, 遵循了空闲让进、忙则等待、有限等待三个原则, 但是依然未遵循让权等待的原则

进程互斥的硬件实现方法

中断屏蔽方法

利用 “开/关中断指令” 实现（与原语的实现思想相同）



优点：简单、高效

缺点：不适用于多处理机；只适用于操作系统内核进程，不适用于用户进程

TestAndSet （TS 指令/TSL 指令）

TSL 指令使用硬件实现的，执行的过程不允许被中断，只能一气呵成

C 语言描述的逻辑

```
// 布尔型共享变量 lock 表示当前临界区是否被加锁  
// true 表示已加锁, false 表示未加锁  
bool TestAndSet (bool *lock){  
    bool old;  
    old = *lock; // old用来存放lock 原来的值  
    *lock = true; // 无论之前是否已加锁, 都将lock设为true  
    return old; // 返回lock原来的值  
}  
  
// 以下是使用 TSL 指令实现互斥的算法逻辑  
while (TestAndSet (&lock)); // "上锁" 并 "检查"  
临界区代码段...  
lock = false; // "解锁"  
剩余区代码段...
```

相比于软件实现的方法，TSL 指令把 “上锁” 和 “检查” 操作用硬件的方式变成了

一气呵成的原子操作

优点：实现简单，适用于多处理机环境；无需像软件实现方式那样严格检查是否有逻辑

漏洞

缺点：不满足 “让权等待” 原则，暂时无法进入临界区的进程会占用 CPU 循环执行

TSL 指令从而导致忙等

SWAP 指令

SWAP 指令也是使用硬件实现的，执行的过程不允许被中断，只能一气呵成

C 语言描述的逻辑

有的地方也叫 Exchange 指令，或简称 XCHG 指令。

Swap 指令是用硬件实现的，执行的过程不允许被中断，只能一气呵成。以下是用C语言描述的逻辑

```
//Swap 指令的作用是交换两个变量的值
Swap (bool *a, bool *b) {
    bool temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

```
//以下是用 Swap 指令实现互斥的算法逻辑
//lock 表示当前临界区是否被加锁
bool old = true;
while (old == true)
    Swap (&lock, &old);
临界区代码段...
lock = false;
剩余区代码段...
```

逻辑上来看 Swap 和 TSL 并无太大区别，都是先记录下此时临界区是否已经被上锁（记录在 old 变量上），再将上锁标记 lock 设置为 true，最后检查 old，如果 old 为 false 则说明之前没有别的进程对临界区上锁，则可跳出循环，进入临界区。

优点：实现简单，无需像软件实现方法那样严格检查是否会有逻辑漏洞；适用于多处理机环境

缺点：不满足“让权等待”原则，暂时无法进入临界区的进程会占用CPU并循环执行TSL指令，从而导致“忙等”。

信号量

信号量机制

用户进程可以通过使用操作系统提供的一对原语来对信号量进行操作，从而方便的实现了进程互斥、进程同步。

信号量其实就是一个变量（可以是一个整数、也可以是更复杂的记录型变量），可以用一个信号量来表示系统中某种资源的数量

一对原语：wait (S)、signal (S) 常简称为 P、V 操作 常用 P(S)、V(S)

整型信号量

用一个整数型的变量作为信号量，用来表示系统中某种资源的数量

与普通整数变量的区别：对信号量的操作只有三种：初始化、P 操作、V 操作

```
int S = 1; // 初始化整型信号量s, 表示当前系统中可用的打印机资源数

void wait (int S) { //wait 原语, 相当于“进入区”
    while (S <= 0); //如果资源数不够, 就一直循环等待
    S=S-1;          //如果资源数够, 则占用一个资源
}

void signal (int S) { //signal 原语, 相当于“退出区”
    S=S+1;           //使用完资源后, 在退出区释放资源
}
```

“检查”和“上锁”一气呵成, 避免了并发、异步导致的问题

存在的问题: 不满足“让权等待”原则, 会发生“忙等”

进程P0:

```
...
wait(S);          //进入区, 申请资源
使用打印机资源... //临界区, 访问资源
signal(S);         //退出区, 释放资源
...
```

进程P1:

```
...
wait(S);
使用打印机资源...
signal(S);
...
```

进程Pn:

```
...
wait(S);
使用打印机资源...
signal(S);
...
```

记录型信号量

整型信号量的缺陷是存在“忙等”问题, 因此人们又提出了“记录型信号量”, 即

用记录型数据结构表示的信号量

```
/*记录型信号量的定义*/
typedef struct {
    int value;          // 剩余资源数
    struct process *L; // 等待队列
} semaphore;
```

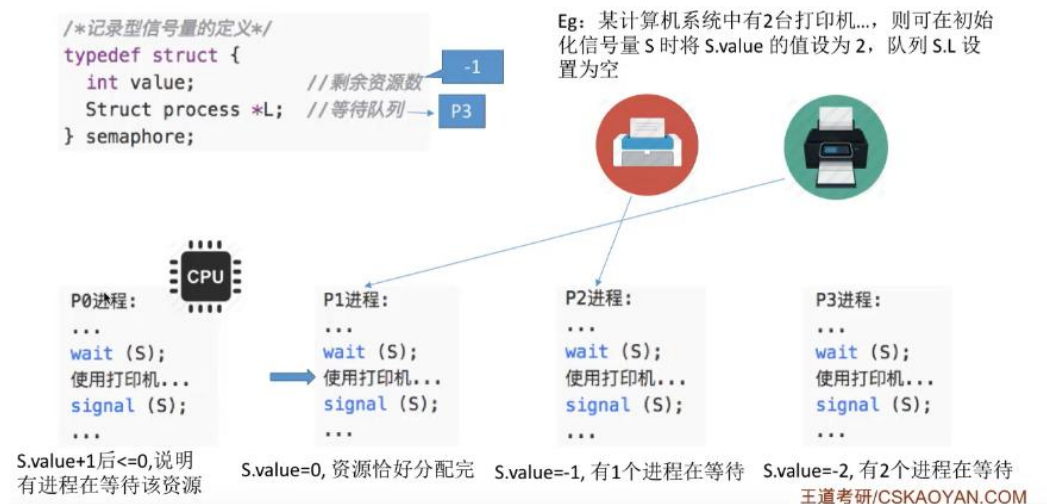
```
/*某进程需要使用资源时, 通过 wait 原语申请*/
void wait (semaphore S) {
    S.value--;
    if (S.value < 0) {
        block (S.L);
    }
}
```

如果剩余资源数不够, 使用block原语使进程从运行态进入阻塞态, 并把挂到信号量S的等待队列(即阻塞队列)中

```
/*进程使用完资源后, 通过 signal 原语释放*/
void signal (semaphore S) {
    S.value++;
    if (S.value <= 0) {
        wakeup(S.L);
    }
}
```

释放资源后, 若还有别的进程在等待这种资源, 则使用wakeup原语唤醒等待队列中的一个进程, 该进程从阻塞态变为就绪态

e.g.



wait 和 signal 可以用于系统资源的申请和释放

S.value 的初值表示某种资源的数目

对信号量 S 的一次 P 操作意味着进程请求一个单位的该类资源, 因此 S.value 数目 -1。

当 S.value < 0 时 表示该资源已经分配完毕, 因此进程应调用 block 原语进行自我阻塞(从运行态 → 阻塞态), 主动放弃处理机, 并进入等待队列, 因此 该机制遵循了 “让权等待”

原则, 不会出现 “忙等” 现象

对信号量 S 的一次 V 操作意味着进程释放一个单位的该类资源, 因此 S.value 数目 +1。当 S.value ≤ 0 时 表示仍有进程在等待资源, 因此进程应调用 wakeup 原语进行自唤醒(从阻塞态 → 就绪态)。

信号量机制实现进程互斥

1. 分析并发进程的关键活动, 划定临界区
2. 设置互斥信号量 mutex, 初值为 1
3. 在临界区之前执行 P

4. 在临界区之后执行 V

```
/*信号量机制实现互斥*/
semaphore mutex=1; //初始化信号量

P1(){
    ...
    P(mutex);    //使用临界资源前需要加锁
    临界区代码段...
    V(mutex);    //使用临界资源后需要解锁
    ...
}

P2(){
    ...
    P(mutex);
    临界区代码段...
    V(mutex);
    ...
}
```

对不同的临界资源需要设置不同的互斥信号量

P、V 操作必须成对出现

信号量机制实现进程同步

进程同步：要让各并发进程按要求有序的推进

1. 必须保证 “一前一后” 执行的两个操作

2. 设置同步信号量 S ，初值为 0

3. 在 “前操作” 之后执行 V(S)

4. 在 “后操作” 之前执行 P(S)

```
/*信号量机制实现同步*/
semaphore S=0; //初始化同步信号量，初始值为0
```

```
P1(){
    代码1;
    代码2;
    V(S);
    代码3;
}
```

```
P2(){
    P(S);
    代码4;
    代码5;
    代码6;
}
```

若先执行到 V(S) 操作，则 S++ 后 S=1。之后当执行到 P(S) 操作时，由于 S=1，表示有可用资源，会执行 S--，S 的值变回 0，P2 进程不会执行 block 原语，而是继续往下执行代码4。

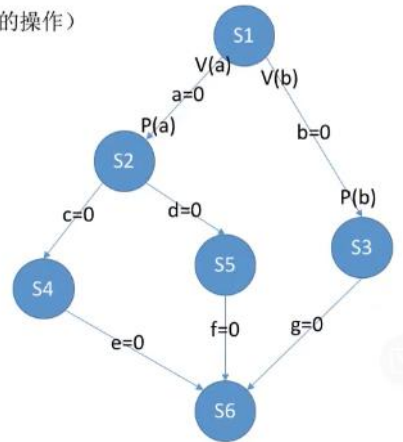
若先执行到 P(S) 操作，由于 S=0，S-- 后 S=-1，表示此时没有可用资源，因此P操作中会执行 block 原语，主动请求阻塞。之后当执行完代码2，继而执行 V(S) 操作，S++，使 S 变回 0，由于此时有进程在该信号量对应的阻塞队列中，因此会在 V 操作中执行 wakeup 原语，唤醒 P2 进程。这样 P2 就可以继续执行 代码4 了

e.g.

其实每一对前驱关系都是一个进程同步问题（需要保证一前一后的操作）
因此，

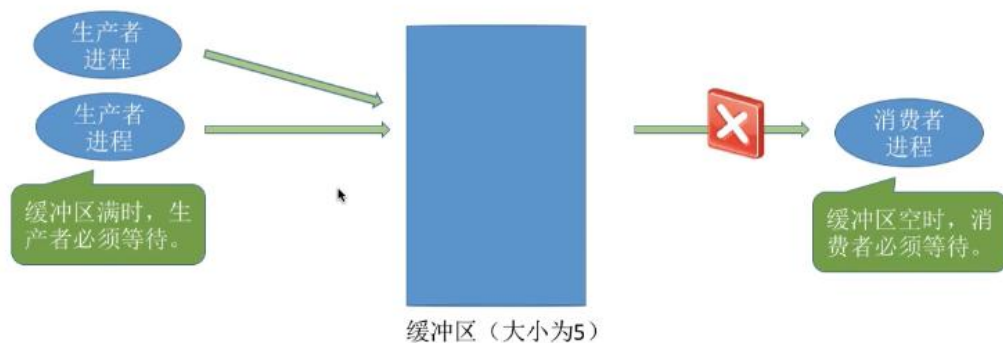
1. 要为每一对前驱关系各设置一个同步变量
2. 在“前操作”之后对相应的同步变量执行 V 操作
3. 在“后操作”之前对相应的同步变量执行 P 操作

```
P1() {  
    ...  
    S1;  
    V(a);  
    V(b);  
    ...  
}  
  
P2() {  
    ...  
    P(a);  
    S2;  
    V(c);  
    V(d);  
    ...  
}  
  
P3() {  
    ...  
    P(b);  
    S3;  
    V(g);  
    ...  
}  
  
P4() {  
    ...  
    P(c);  
    S4;  
    V(e);  
    ...  
}  
  
P5() {  
    ...  
    P(d);  
    S5;  
    V(f);  
    ...  
}  
  
P6() {  
    ...  
    P(e);  
    P(f);  
    P(g);  
    S6;  
    ...  
}
```



信号量模型

生产者→消费者问题



系统中有一组生产者进程和一组消费者进程，生产者进程每次生产一个产品放入缓冲区，

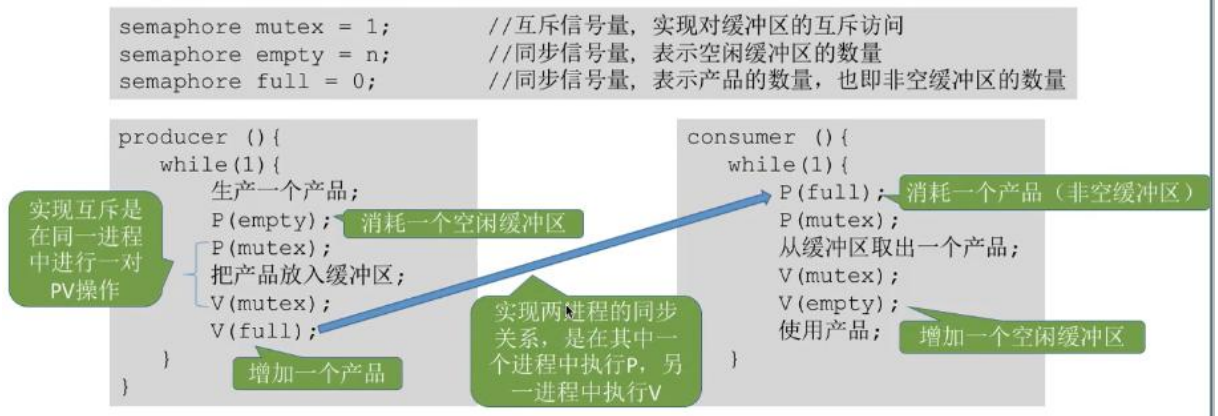
消费者进程每次从缓冲区中取出一个产品并使用

生产者、消费者共享一个初始为空、大小为 n 的缓冲区

只有缓冲区没满时，生产者才能把产品放入缓冲区，否则必须等待 → 进程同步

只有缓冲区不空时，消费者才能把产品取出缓冲区，否则必须等待 → 进程同步

缓冲区是临界资源，个进程必须互斥地访问 → 进程互斥

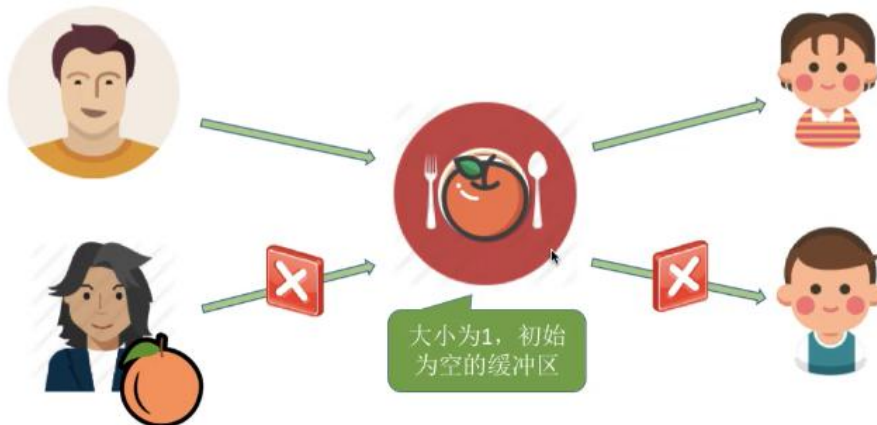


相邻 P、V 操作不能颠倒。双方都等待被对方唤醒，出现死锁现象

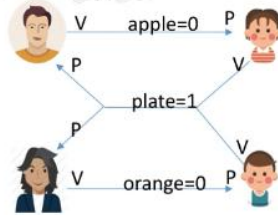
因此，实现互斥地 P 操作一定要在实现同步的 P 操作之后

多生产者→多消费者问题

桌子上有一只盘子，每次只能向其中放入一个水果。爸爸专向盘子中放苹果，妈妈专向盘子中放橘子，儿子专等着吃盘子中的橘子，女儿专等着吃盘子中的苹果。只有盘子空时，爸爸或妈妈才可向盘子中放一个水果。仅当盘子中有自己需要的水果时，儿子或女儿可以从盘子中取出水果。用PV操作实现上述过程。



如何实现



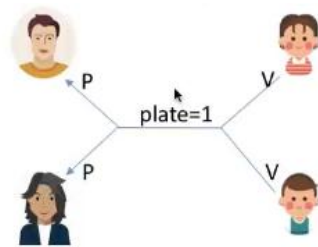
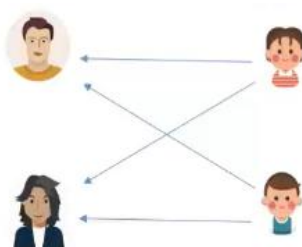
```
semaphore mutex = 1; //实现互斥访问盘子（缓冲区）
semaphore apple = 0; //盘子中有几个苹果
semaphore orange = 0; //盘子中有几个橘子
semaphore plate = 1; //盘子中还可以放多少个水果
```

<pre>dad () { while(1) { 准备一个苹果; P(plate); P(mutex); 把苹果放入盘子; V(mutex); V(apple); } }</pre>	<pre>mom () { while(1) { 准备一个橘子; P(plate); P(mutex); 把橘子放入盘子; V(mutex); V(orange); } }</pre>	<pre>daughter () { while(1) { P(apple); P(mutex); 从盘中取出苹果; V(mutex); V(plate); 吃掉苹果; } }</pre>	<pre>son () { while(1) { P(orange); P(mutex); 从盘中取出橘子; V(mutex); V(plate); 吃掉橘子; } }</pre>
---	--	--	--

当 plate 初始值（缓冲区）= 1 时，可以不用设置互斥访问缓冲区信号量

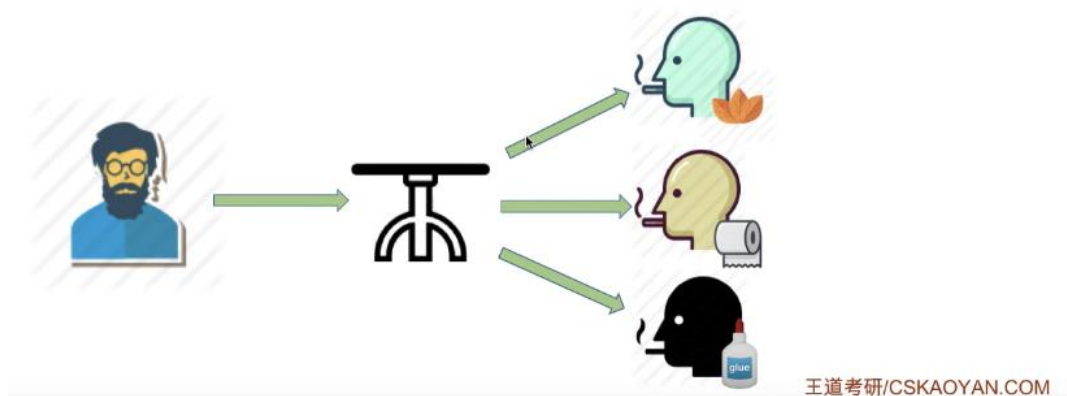
正确的分析方法应从：“事件”的角度老考虑，将上述四对“进程行为的前后关系”抽象为一对“事件的前后关系”

盘子变空事件 → 放入水果事件 “盘子变空事件”既可以由儿子引发，也可以由女儿引发，“放水果事件”既可能是父亲执行，也可能是母亲执行，这样就可以用一个同步信号量解决了



吸烟者问题

假设一个系统有三个吸烟者进程和一个供应者进程。每个吸烟者不停地卷烟并抽掉它，但是要卷起并抽掉一支烟，吸烟者需要三种材料：烟草、纸和胶水。三个吸烟者中，第一个拥有烟草、第二个拥有纸、第三个拥有胶水。供应者进程无限地提供三种材料，供应者每次将两种材料放桌子上，拥有剩下那种材料的吸烟者卷一根烟并抽掉它，并给供应者进程一个信号告诉完成了，供应者就会放另外两种材料再桌上，这个过程一直重复（让三个吸烟者轮流地抽烟）

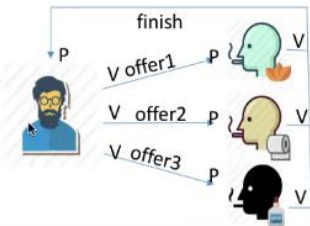


王道考研/CSKAOYAN.COM



是否需要设置一个专门的互斥信号量？

如何实现



```
provider () {  
    while(1) {  
        if(i==0) {  
            将组合一放桌上;  
            V(offer1);  
        } else if(i==1) {  
            将组合二放桌上;  
            V(offer2);  
        } else if(i==2) {  
            将组合三放桌上;  
            V(offer3);  
        }  
        i = (i+1)%3;  
        P(finish);  
    }  
}
```

```
semaphore offer1 = 0; //桌上组合一的数量  
semaphore offer2 = 0; //桌上组合二的数量  
semaphore offer3 = 0; //桌上组合三的数量  
semaphore finish = 0; //抽烟是否完成  
int i = 0; //用于实现“三个吸烟者轮流抽烟”
```

```
smoker1 () {  
    while(1) {  
        P(offer1);  
        从桌上拿走组合一; 卷烟; 抽掉;  
        V(finish);  
    }  
}
```

```
smoker2 () {  
    while(1) {  
        P(offer2);  
        从桌上拿走组合二; 卷烟; 抽掉;  
        V(finish);  
    }  
}
```

```
smoker3 () {  
    while(1) {  
        P(offer3);  
        从桌上拿走组合三; 卷烟; 抽掉;  
        V(finish);  
    }  
}
```

读者写者模型

有读者和写者两组并发进程，共享一个文件，当两个或两个以上的读进程同时访问共享数据时不会产生副作用，但若某个写进程和其他进程（读进程或写进程）同时访问共享数据时则可能导致数据不一致的错误。因此要求：①允许多个读者可以同时~~对文件执行读操作~~；②只允许一个写者往文件中写信息；③任一写者在完成写操作之前不允许其他读者或写者工作；④写者执行写操作前，应让已有的读者和写者全部退出。

1. 关系分析。找出题目中描述的各个进程，分析它们之间的同步、互斥关系。
2. 整理思路。根据各进程的操作流程确定P、V操作的大致顺序
3. 设置信号量。设置需要的信号量，并根据题目条件确定信号量初值。（互斥信号量初值一般为1，同步信号量的初始值要看对应资源的初始值是多少）

两类进程：写进程、读进程

互斥关系：写进程—写进程、写进程—读进程。读进程与读进程不存在互斥问题。

写者进程和任何进程都互斥，设置一个互斥信号量 **rw**，在写者访问共享文件前后分别执行P、V操作。

读者进程和写者进程也要互斥，因此读者访问共享文件前后也要对 **rw** 执行P、V操作。

如果所有读者进程在访问共享文件之前都执行 **P(rw)** 操作，那么会导致各个读进程之间也无法同时访问文件。**Key: 读者写者问题的核心思想——怎么处理该问题呢？**

P(rw) 和 **V(rw)** 其实就是对共享文件的“加锁”和“解锁”。既然各个读进程需要同时访问，而读进程与写进程又必须互斥访问，那么我们可以让第一个访问文件的读进程“加锁”，让最后一个访问完文件的读进程“解锁”。可以设置一个整数变量 **count** 来记录当前有几个读进程在访问文件。

```
semaphore rw=1;      //用于实现对文件的互斥访问
int count = 0;        //记录当前有几个读进程在访问文件
semaphore mutex = 1;  //用于保证对count变量的互斥访问
semaphore w = 1;      //用于实现“写优先”
```

分析以下并发执行 **P(w)** 的情况：

读者1→读者2
写者1→写者2
写者1→读者1
读者1→写者1→读者2
写者1→读者1→写者2

结论：在这种算法中，连续进入的多个读者可以同时读文件；写者和其他进程不能同时访问文件；写者不会饥饿，但也并不是真正的“写优先”，而是相对公平的先来先服务原则。

```
writer () {
    while(1) {
        P(w);
        P(rw);
        写文件...
        V(rw);
        V(w);
    }
}
```

```
reader () {
    while(1) {
        P(w);
        P(mutex);
        if(count==0)
            P(rw);
        count++;
        V(mutex);
        V(w);
        读文件...
        P(mutex);
        count--;
        if(count==0)
            V(rw);
        V(mutex);
    }
}
```


哲学家进餐问题

一张圆桌上坐着5名哲学家，每两个哲学家之间的桌上摆一根筷子，桌子的中间是一碗米饭。哲学家们倾注毕生的精力用于思考和进餐，哲学家在思考时，并不影响他人。只有当哲学家饥饿时，才试图拿起左、右两根筷子（一根一根地拿起）。如果筷子已在他人手上，则需等待。饥饿的哲学家只有同时拿起两根筷子才可以开始进餐，当进餐完毕后，放下筷子继续思考。

思考并尝试用代码实现这两种方案...

如何防止死锁的发生？

①可以对哲学家进程施加一些限制条件，比如最多允许四个哲学家同时进餐。这样可以保证至少有一个哲学家是可以拿到左右两只筷子的

②要求奇数号哲学家先拿左边的筷子，然后再拿右边的筷子，而偶数号哲学家刚好相反。用这种方法可以保证如果相邻的两个奇偶号哲学家都想吃饭，那么只会有其中一个可以拿起第一只筷子，另一个会直接阻塞。这就避免了占有一支后再等待另一只的情况。

王道考研/CSKAOYAN.COM

王道考研 CSKAOYAN

如何实现

因此这种方法并不能保证只有两边的筷子都可用时，才允许哲学家拿起筷子。

更准确的说法应该是：各哲学家拿筷子这件事必须互斥的执行。这就保证了即使一个哲学家在拿筷子拿到一半时被阻塞，也不会有别的哲学家会继续尝试拿筷子。这样的话，当前正在吃饭的哲学家放下筷子后，被阻塞的哲学家就可以获得等待的筷子了。



```
semaphore chopstick[5]={1,1,1,1,1};
semaphore mutex = 1; //互斥地取筷子
Pi () { //i号哲学家的进程
    while(1){
        P(mutex);
        P(chopstick[i]); //拿左
        P(chopstick[(i+1)%5]); //拿右
        V(mutex);
        吃饭...
        V(chopstick[i]); //放左
        V(chopstick[(i+1)%5]); //放右
        思考...
    }
}
```

王道考研/CSKAOYAN.COM

哲学家进餐的问题关键在于解决进程死锁

这些进程之间只存在互斥关系，每个进程都需要同时持有两个临界资源，因此就有思索

的隐患

管程（封装思想）

管程的定义及基本特征

信号量机制存在的问题：编写程序困难、易出错

管程：高级同步机制，用来实现进程之间同步、互斥

管程的组成部分：

1. 局部于管程的共享数据结构（如生产者消费者问题的缓冲区）说明；
2. 对该数据结构进行操作的一组过程（函数）；
3. 对局部于管程的共享数据设置初始值的语句
4. 管程有一个名字

管程的基本特征：

1. 局部于管程的数据只能被局部于管程的过程所访问
2. 一个进程只能通过调用管程内的过程才能进入管程访问共享数据
3. 每次仅允许一个进程在管程内执行某个内部过程（管程中有很多入口，但是每次只能开放其中一个入口，并且只能让一个进程或者线程进入，这种互斥特性由编译器负责实现的，程序员不用担心）

```

monitor ProducerConsumer
condition full, empty; //条件变量用来实现同步（排队）
int count=0; //缓冲区中的产品数
void insert (Item item) { //把产品item放入缓冲区
    if (count == N)
        wait (full);
    count++;
    insert_item (item);
    if (count == 1)
        signal(empty);
}
Item remove () { //从缓冲区中取出一个产品
    if (count == 0)
        wait (empty);
    count--;
    if (count == N-1)
        signal(full);
    return remove_item();
}
}
end monitor;

```

由编译器负责实现
各进程互斥地进入
管程中的过程

```

//生产者进程
producer(){
    while(1){
        item = 生产一个产品;
        ProdecerConsumer.insert (item);
    }
}

//消费者进程
consumer(){
    while(1){
        item = ProdecerConsumer.remove ();
        消费产品item;
    }
}

```

每次仅允许一个进程在管程内执行某个内部过程。
例1: 两个生产者进程并发执行，依次调用了
insert 过程...

死锁

死锁的概念

在并发环境下，各进程因为竞争资源而造成的一种 这就是死锁。发生死锁后若无外力干涉，这些进程都将无法向前推进

饥饿：由于长期得不到想要的资源，某进程无法向前推进的现象。比如短进程优先（SPF）算法中，若有源源不断地短进程到来，则长进程将一直得不到处理机，从而发生长进程“饥饿”。

死循环：某进程执行过程中一直跳不出某个循环的现象。有时是因为程序逻辑 bug 导致的，有时是程序员故意设计的

共同点：都是进程无法顺利向前推进的现象

不同点：**死锁**：至少有两个或两个以上的进程同时发生死锁

饥饿：可能只有一个进程发生饥饿

死循环：死循环的进程可以上处理机运行，只不过无法像期待的那样顺利推进。死

锁和饥饿时管理者（操作系统）的问题，死循环是被管理者的问题

死锁产生的必要条件

1. **互斥条件**：只有对必须互斥使用的资源的争抢才会导致死锁

2. **不剥夺条件**：进程所获得的资源在未使用完之前，**不能有其他进程强行夺走**，只能主动释放

3. **请求和保持条件**：进程已经**保持了至少一个资源**，但又提出了新的资源请求，而该资源又被其他进程占有，此时请求进程被阻塞，但又对自己已有的资源**保持**不放

4. **循环等待条件**：**存在一种进程资源的循环等待链**，链中的每一个进程已获得的资源同时被下一个进程所请求

注意：发生死锁时一定有循环等待，但是发生循环等待时未必死锁

什么时候会发生死锁

1. 对系统资源的竞争

2. 进程推进顺序非法

3. 当信号量的使用不当也会造成死锁

死锁的处理策略

1. 预防死锁

2. 避免死锁

3. 死锁的检测和接触

死锁的处理策略

预防死锁

破坏互斥条件：只有对必须互斥使用的资源的争抢才会导致死锁

如果把只能互斥使用的资源改造为允许共享使用，则系统不会进入死锁状态；比如：
SPOOLing 技术。操作系统可以采用 **SPOOLing** 技术把独占设备在逻辑上改造成共享设备
缺点：并不是所有的资源都可以改造成这种共享使用的资源；并且为了系统安全，很多地方必须保护这种互斥性。因此

破坏不剥夺条件：进程所获得的资源在未使用完之前，不能由其他进程强行夺走，只能主动释放

方案一：某个进程请求新的资源得不到满足时，它必须立即释放保持的所有资源，待以后需要时再重新申请（**可能会导致进程饥饿**）

方案二：当某个进程需要的资源被其他进程所占有的时候，可以有操作系统协助，将想要的资源强行剥夺。这种方式一般需要考虑各进程的优先级

缺点：实现起来较复杂

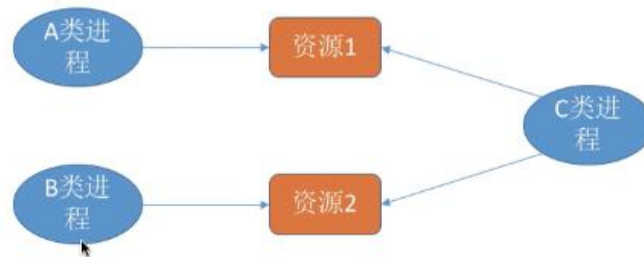
只适用于保存和恢复状态的资源，比如：CPU

反复的申请释放，增加系统开销，降低系统吞吐量

破坏请求和保持条件：进程已经**至少保持了至少一个资源**，但又提出了新的资源**请求**，而该资源由被其他进程占有，此时请求进程被阻塞，但又对自己已有的**资源保持不放**

采用静态分配法 进程在运行前一次申请完它所需要的全部资源 ,在他的资源未满足时 ,不让他投入运行。一旦投入运行后 ,这些资源就一直归他所有 ,该进程就不会再请求别的任何资源

缺点： 资源利用率低 , 可能导致某些进程饥饿



破坏循环等待条件 : 存在一种进程资源的循环等待链 , 链中的每一个进程以获得的资源同时被下一个进程所请求

顺序资源分配法 : 首先给系统中的资源编号 , 规定每个进程必须按编号递增的顺去请求资源 , 同类资源一次申请完 (一个进程只能已占有小编号的资源时 , 才有资格申请更大编号的资源。按此规则 , 已持有大编号资源的进程不可能逆向地回来申请小编号的资源 , 从而就不会产生循环等待的现象)

缺点： 不方便增加新设备

进程实际使用的资源顺序可能和编号递增顺序不一致 , 会导致资源浪费

必须按照规定次序申请资源 , 变成麻烦

避免死锁

安全序列 : 如果系统按照这种序列分配资源 , 则每个进程都能顺利完成 , 只要能找出一个安全序列 , 系统就是安全的 , 安全序列可能有多 ; 如果分配资源之后 , 找不出任何一个

安全序列，系统就进入了不安全状态，这就意味着可能所有进程都无法顺利执行下去

如果系统处于安全状态，就一定不会发生死锁，如果系统进入不安全状态，就可能会发生死锁

因此可以在系统分配之前预先判断这次分配是否会导致系统进入不安全状态，一次

决定是否答应资源分配请求 → 银行家算法



什么是安全序列

你是一位成功的银行家，手里掌握着100个亿的资金...
有三个企业想找你贷款，分别是企业B、企业A、企业T，为描述方便，简称BAT。
B表示：“大哥，我最多会跟你借70亿...”
A表示：“大哥，我最多会跟你借40亿...”
T表示：“大哥，我最多会跟你借50亿...”
然而...江湖中有个不成文的规矩：如果你借给企业的钱总数达不到企业提出的最大要求，那么不管你之前给企业借了多少钱，那些钱都拿不回来了...
刚开始，BAT三个企业分别从你这儿借了20、10、30亿...



	最大需求	已借走	最多还会借
B	70	20	50
A	40	10	30
T	50	30	20

	最大需求	已借走	最多还会借
B	70	20	50
A	40	10+20=30	30-20=10
T	50	30	20

手里还有：40亿

此时... A 还想借 20 亿，你敢借吗？
假如答应了A的请求.....

手里还有：20亿

可以先把20亿全部借给T，等T把钱全部还回来了，手里就会有20+30=50亿，再把这些钱全借给B，B还钱后总共有50+20=70亿，最后再借给A



银行家算法

进程	最大需求	已分配	最多还需要
P0	(7, 5, 3)	(0, 1, 0)	(7, 4, 3)
P2	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)
P4	(4, 3, 3)	(0, 0, 2)	(4, 3, 1)

资源总数 (10, 5, 7)，剩余可用资源 (7, 4, 3)

此时系统是否处于安全状态？

思路：尝试找出一个安全序列... {P1, P3, P0, P2, P4}

依次检查剩余可用资源 (3, 3, 2) 是否能满足各进程的需求

可满足P1需求，将 P1 加入安全序列，并更新剩余可用资源值为 (5, 3, 2)

依次检查剩余可用资源 (5, 3, 2) 是否能满足剩余进程（不包括已加入安全序列的进程）的需求

可满足P3需求，将 P3 加入安全序列，并更新剩余可用资源值为 (7, 4, 3)

依次检查剩余可用资源 (7, 4, 3) 是否能满足剩余进程（不包括已加入安全序列的进程）的需求.....

.....

以此类推，共五次循环检查即可将5个进程都加入安全序列中，最终可得一个安全序列。该算法称为安全性算法。可以很方便地用代码实现以上流程，每一轮检查都从编号较小的进程开始检查。实际做题时可以更快速的得到安全序列。

死锁的检测和解除

如何检测：资源分配图

死锁检测算法

如何解除：资源剥夺法

撤销进程法

进程回退法

内存

内存的基础知识

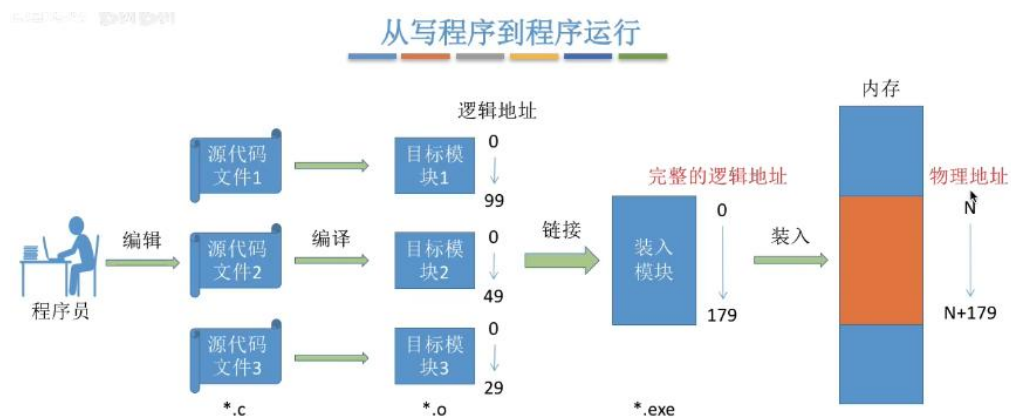
内存（高速存储设备）是用于存放数据的硬件。程序执行前需要先放到内存中才能被

CPU 处理

区分各个程序的数据是放在什么地方：给内存的存储单元编地址

内存地址从 0 开始，每个地址对应一个存储单元

存储单元：“按字节编址”、“按字编址”



编译：由编译程序将用户源代码编译成若干个目标模块（编译就是把高级语言翻译为机器语言）

链接：由链接程序将编译后形成的一组目标模块，以及所需库函数链接在一起，形成一个完整的装入模块

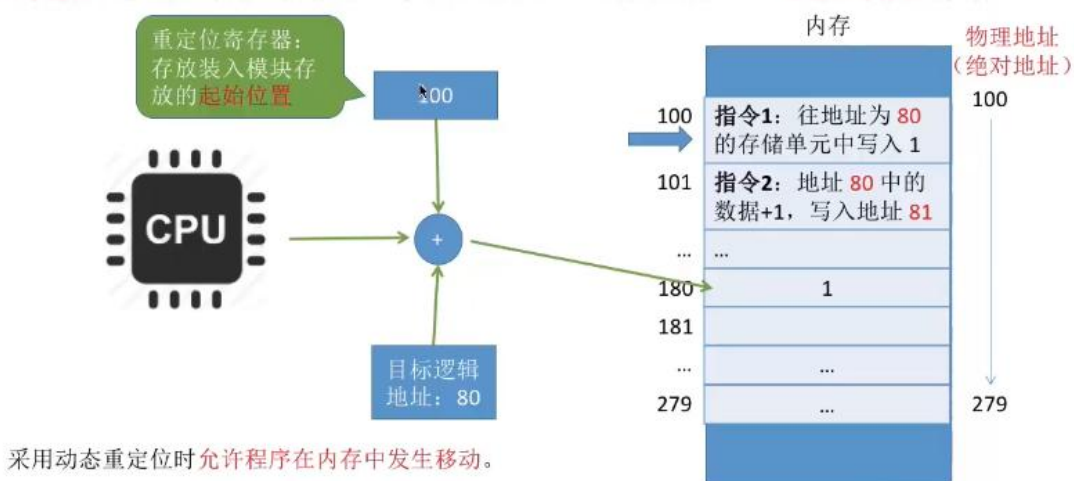
装入模块装入内存（装入 → 物理地址）

装入的三种方式：1. 绝对装入（**编译时产生绝对地址**）：在编译时，如果知道程序将放到内存中的那个位置，编译程序将产生绝对地址的目标代码。装入程序按照装入模块中的地址，将数据和程序装入内存（只适用于单道程序环境）

2. 静态重定位（**装入时将逻辑地址转换为物理地址**）：装入时对地址进行重定位，将逻辑地址变为物理地址（装入的起始地址为 100，则所有有关地址的参数都 +100）特点：作业再装入内存时，**必须分配器要求的全部内存空间**，如果没有足够的内存，就不能装入该作业。作业一旦进入内存后，**在运行期间就不能移动**，也不能申请内存空间

3. 动态重定位（**运行时将逻辑地址转换为物理地址，需要设置重定位寄存器**）：动态运行时装入，并且可将程序分配到不连续的存储区中：在程序运行前只需装入他的部分代码即可投入运行，然后在程序运行期间，根据需要动态申请分配内存：便于程序段的共享，可以向用户提供一个比存储空间大得多的地址空间

动态重定位：又称**动态运行时装入**。编译、链接后的装入模块的地址都是从0开始的。装入程序把装入模块装入内存后，并不会立即把逻辑地址转换为物理地址，而是把地址转换推迟到程序真正要执行时才进行。因此装入内存后所有的地址依然是逻辑地址。这种方式需要一个**重定位寄存器**的支持。



链接的三种方式（链接 → 逻辑地址）

1. 静态链接：在程序运行之前，先将各目标模块及他们所需的库函数连接成一个完整的可执行文件
2. 装入时动态链接：装入时动态链接：将各目标模块装入内存时，边装入边链接的链接方式
3. 运行时动态链接：在程序执行中需要该目标模块时，才能对他进行链接。其优点是便于修改和更新，便于实现对目标模块的共享

内存保护

1. 操作系统负责内存空间的分配与回收
2. 操作系统需要提供某种技术从逻辑上对内存进行扩充
3. 提供地址转换功能，负责程序的 逻辑地址 → 物理地址
4. 内存保护，保证各进程在各自存储空间内运行，互不干扰
 - a. 设置上下限寄存器
 - b. 利用重定位寄存器（基地址寄存器）、界地址寄存器（最大逻辑地址长度）

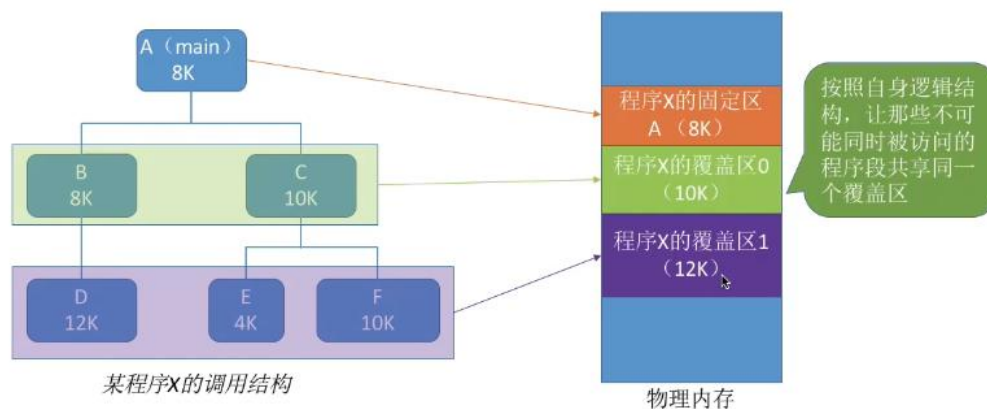
进行判断

内存空间的扩充

覆盖技术： 解决程序大小超过物理内存总和的问题

将程序分为多个段（多个模块）。常用的段常驻内存，不常用的段需要时调入内存。内存中分一个 “固定区” 和若干个 “覆盖区”。需要常驻内存的段放在 “固定区” 中，调

入后就不再调出（除非运行结束）。**不常用的段**放在“覆盖区”，需要用到时调入内存，用不到调出



缺点：必须由程序员声明覆盖结构，对用户不透明，增强了用户变成负担

交换技术：内存空间紧张时，系统将内存中某些进程暂时**换出**外存，把外存中某些已具备运行条件的进程**换入**内存

中级调度（内存调度）：就是要决定将那个处于挂起状态的进程重新调入内存

交换（对换）技术的设计思想：内存空间紧张时，系统将内存中某些进程暂时**换出**外存，把外存中某些已具备运行条件的进程**换入**内存（进程在内存与磁盘间动态调度）



1. 应该在外存（磁盘）的什么位置保存被换出的进程？
2. 什么时候应该交换？
3. 应该换出哪些进程？



磁盘存储空间

1. 具有对换功能的操作系统中，通常把磁盘空间分为**文件区**和**对换区**两部分。**文件区**主要用于存放文件，**主要追求存储空间的利用率**，因此对文件区空间的管理采用**离散分配方式**；**对换区**空间只占磁盘空间的小部分，**被换出的进程数据就存放在对换区**。由于对换的速度直接影响到系统的整体速度，因此对换区空间的管理**主要追求换入换出速度**，因此通常对换区采用**连续分配方式**（学过文件管理章节后即可理解）。总之，**对换区的I/O速度比文件区的更快**。
2. 交换通常在许多进程运行且内存吃紧时进行，而系统负荷降低就暂停。例如：在发现许多进程运行时经常发生缺页，就说明内存紧张，此时可以换出一些进程；如果缺页率明显下降，就可以暂停换出。
3. 可优先换出阻塞进程；可换出优先级低的进程；为了防止优先级低的进程在被调入内存后很快又被换出，有的系统还会考虑进程在内存的驻留时间...
(注意：**PCB**会常驻内存，不会被换出外存)

覆盖与交换的区别：覆盖是在同一个程序或进程中的

交换是在不同进程之间的

内存空间的分配与回收

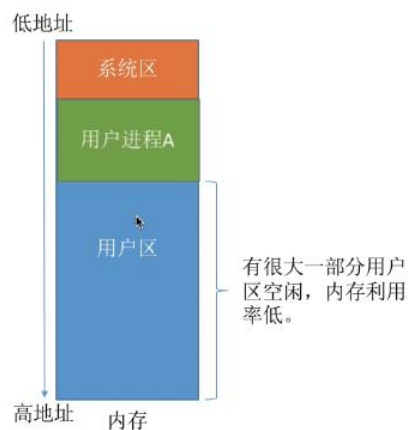
连续分配：指为用户进程分配的必须是一个连续的内存空间

连续分配管理方式

单一连续分配方式：内存被分为**系统区**和**用户区**。系统区通常位于内存的低地址部分，用于存放操作系统相关数据；用户区用于存放用户进程相关数据。内存中**只能有一道用户程序**，用户程序独占整个用户区空间

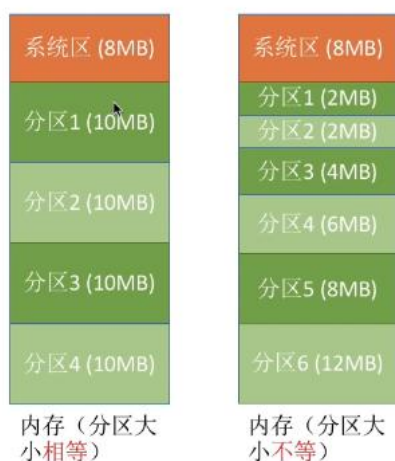
优点：实现简单；**无外部碎片**；可采用覆盖技术扩充内存；不一定需要采取内存保护

缺点：只能用于单用户、单任务的操作系统中；有内部碎片；存储器利用率极低



固定分区分配方式

将整个用户空间划分为若干个固定大小的分区，在每个分区中装入一道作业



分区大小相等：缺乏灵活性，但是很适用于用一台计算机控制多个相同对象的场合

分区大小不等：增加灵活性，可以满足不同大小的进程需求

操作系统需要建立一个数据结构 → 分区说明表

操作系统需要建立一个数据结构——**分区说明表**，来实现各个分区的分配与回收。每个表项对应一个分区，通常按分区大小排列。每个表项包括对应分区的大小、起始地址、状态（是否已分配）。

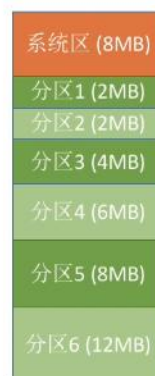
分区号	大小 (MB)	起始地址 (M)	状态
1	2	8	未分配
2	2	10	未分配
3	4	12	已分配
.....			

用数据结构的数组（或链表）即可表示这个表

当某用户程序要装入内存时，由操作系统内核程序根据用户程序大小检索该表，从中找到一个能满足大小的、未分配的分区，将之分配给该程序，然后修改状态为“已分配”。

优点：实现简单，**无外部碎片**。

缺点：a. 当用户程序太大时，可能所有的分区都不能满足需求，此时不得不采用覆盖技术来解决，但这又会降低性能；b. **会产生内部碎片**，内存利用率低。

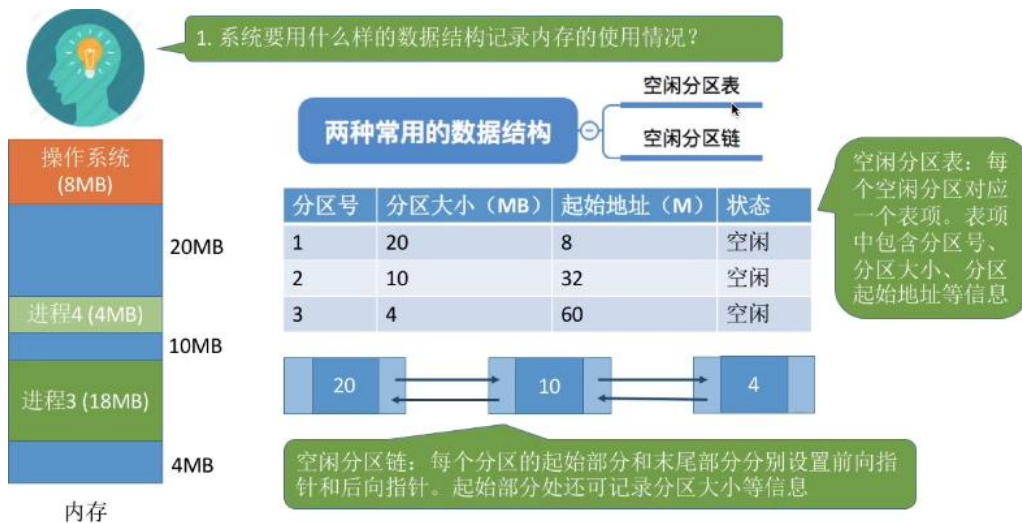


内存 (分区大小不等)

动态分区分配

又称为可变分区分配。这种分配方式**不会预先划分内存分区**，而是在进程装入内存时，

根据进程的大小动态的建立分区，并使分区的大小正好适合进程的需要



动态分区分配没有内部碎片，但是有外部碎片

内部碎片：分配给某进程的内存区域中，如有些部分没有用上

外部碎片：是指内存中的某些空闲分区由于太小而难以利用

如果内存中空闲空间的总和本来可以满足某进程的需求，但由于进程需要的是一整块连续的内存空间，因此这些碎片不能满足进程的要求。可以通过**紧凑**技术来解决外部碎片

动态分区分配算法：

(1) 首次适应算法：**每次从低地址开始查找**，找到第一个能满足大小的空闲分区；空闲分区以低地址递增的次序排列。每次分配内存时顺序查找空闲分区链（表），找到大小能满足要求的第一个空闲分区

(2) 最佳适应算法：**空闲区按照容量递增次序链接**。每次分配内存时顺序查找空闲分区表（链）。尽可能多的留下大片的空闲区，优先使用小的空闲区；**缺点**：每次都选最小的分区

进行分配，会留下越来越多的、很小的、难以利用的内存块，会产生很多的外部碎片

(3) 最坏适应算法：空闲分区按照容量递减次序链接。每次分配内存时顺序查找空闲分区表（链）。缺点：每次都选最大的分区进行分配，会导致较大的连续空闲区被迅速用完。之后到达大进程，就没有内存分区可用了

(4) 邻近适应算法：空闲区以地址递增的顺序排列（可排成一个循环链表）。每次分配内存时从上次查找结束的位置开始查找空闲分区表（链），找到大小能满足要求的第一个空闲分区；

首次适应算法的效果最好

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找适合的分区	空闲分区以地址递增次序排列	综合看性能最好。算法开销小，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片；算法开销大，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小碎片	大分区容易被用完，不利于大进程；算法开销大（原因同上）
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列（可排列成循环链表）	不用每次都从低地址的小分区开始检索。算法开销小（原因同首次适应算法）	会使高地址的大分区也被用完

非连续分配管理方式

非连续分配：为用户进程分配的可以是一些分散的内存空间

把分区大小设置的更小一些，内部碎片会更小，内存利用率更高。

分页存储管理的基本概念

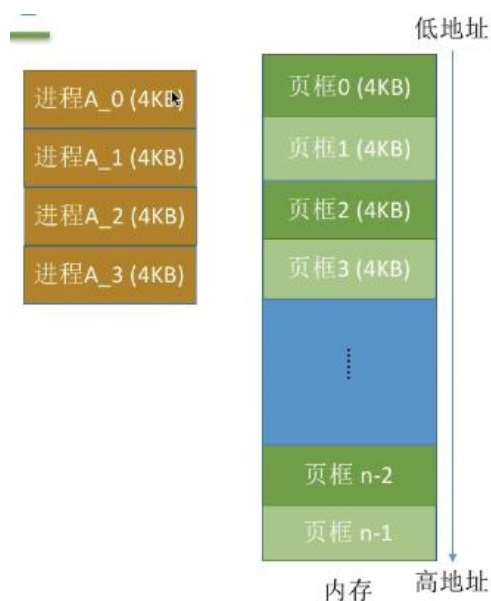
将内存空间分为一个个大小相等的分区，每个分区就是一个“页框”；页框号从0开始

将用户进程的地址空间也分为与页框大小相等的一个个区域，称为页\页面。每个页面有一个编号从 0 开始

页框不能太大，否则可能会产生过大的内部碎片

操作系统会以页框为单位为各个进程分配内存空间。进程的每个页面分别放入一个页框中。也就是说，进程的页面与内存的页框是一一对应的关系

各个页面不必连续存放，也不必按先后顺序来，可以到不相邻的各个也框中。



实现地址的转换



如何计算：

页号 = 逻辑地址 / 页面长度（取除法的整数部分）

页内偏移量 = 逻辑地址 % 页面长度（取除法的余数部分）

为了方便计算页号、页内偏移量，
页面大小一般要为 2 的整数幂

假设用 32 个二进制位表示逻辑地址，页面大小为 $2^{10}\text{B} = 1024\text{B} = 1\text{KB}$

0 号页的逻辑地址空间应该是 0~1023，用二进制表示应该是：

00000000000000000000000000000000 ~ 000000000000000000000000000000001111111111

1 号页的逻辑地址空间应该是 1024~2047，用二进制表示应该是：

00000000000000000000000000000000100000000000 ~ 000000000000000000000000000000001111111111

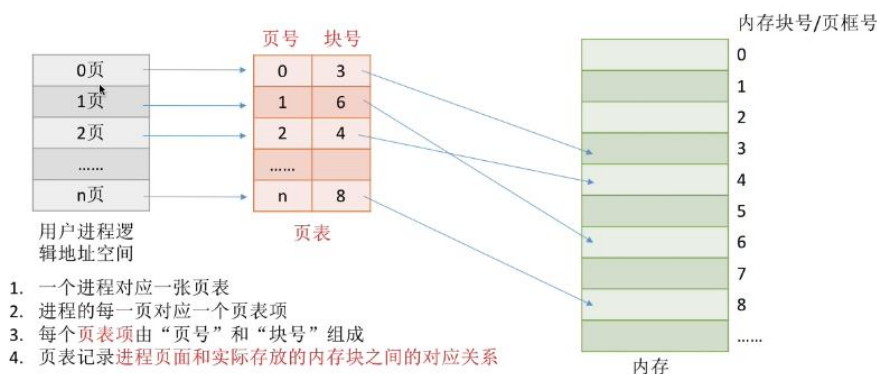
2 号页的逻辑地址空间应该是 2048~3071，用二进制表示应该是：

00000000000000000000000000000000100000000000 ~ 00000000000000000000000000000000101111111111

地址结构包含两个部分：前一部分为页号，后一部分为页内偏移量

为了知道进程的每个页面在内存中存放的位置，操作系统要为每个进程建立一张页表。

页面：



为什么 每个页表项的长度是相同的，页号是“隐含”的？

Eg: 假设某系统物理内存大小为 4GB，页面大小为 4KB，则每个页表项至少应该为多少字节？

$4\text{GB} = 2^{32}\text{B}$, $4\text{KB} = 2^{12}\text{B}$

因此 4GB 的内存总共会被分为 $2^{32} / 2^{12} = 2^{20}$ 个内存块，因此内存块号的范围应该是 $0 \sim 2^{20}-1$

因此至少要 20 个二进制位才能表示这么多的内存块号，因此至少要 3 个字节才够

（每个字节 8 个二进制位，3 个字节共 24 个二进制位）

页号	块号
0	3字节
1	3字节
.....	3字节
n	3字节

页表

各页表项会按顺序连续地存放在内存中

如果该页表在内存中存放的起始地址为 X，则

M 号页对应的页表项一定是存放在内存地址为 $X + 3 * M$

因此，页表中的“页号”可以是“隐含”的。

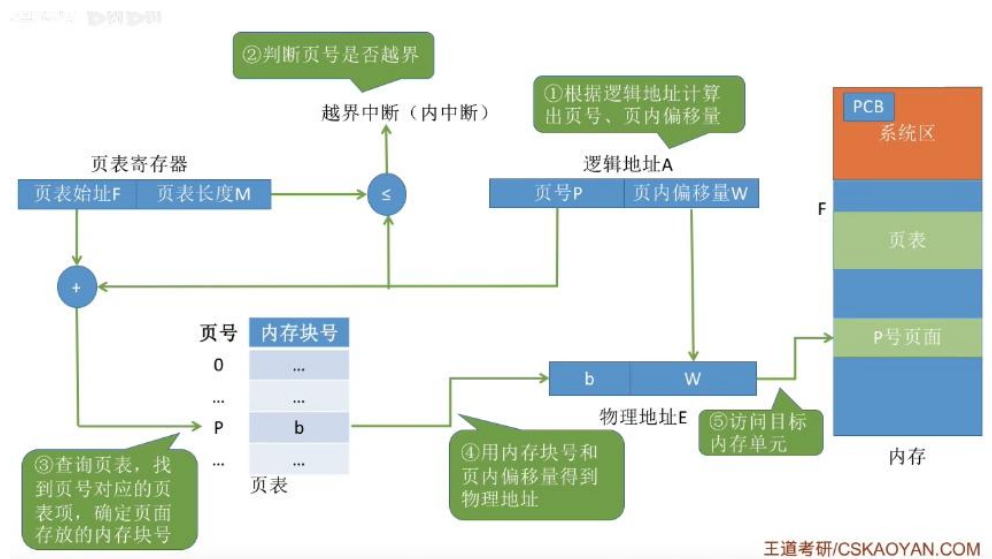
只需要知道页表存放的起始地址和页表项长度，即可找到各个页号对应的页表项存放的位置

基本分页存储管理

基本地址变换机构可以借助进程的页表将逻辑地址转换为物理地址

通常会在系统中设置一个页表寄存器，存放页表在内存中的起始地址 F 和页表长度 M。

进程未执行时，页表的始址和页表长度放在进程控制块（PCB）中，当进程被调度时，将他们放到页表寄存器中

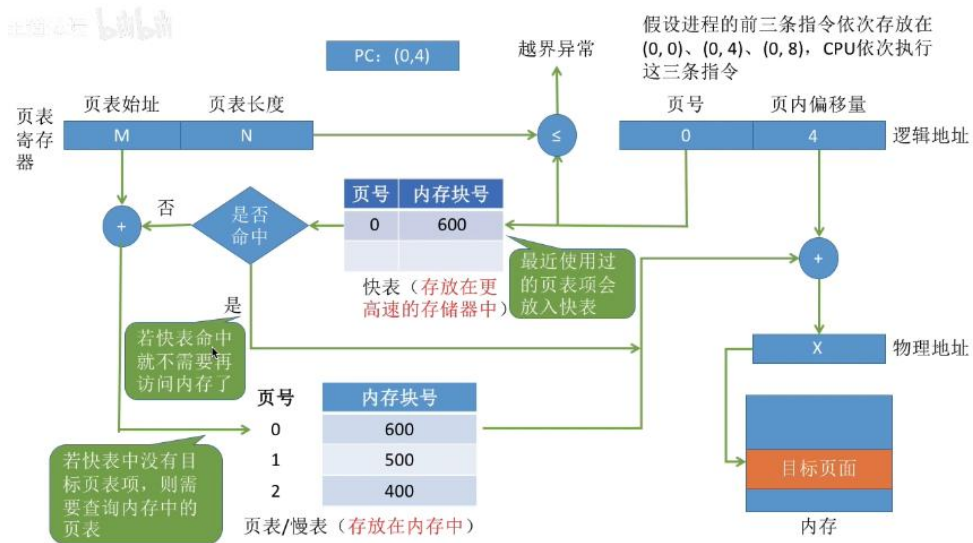


具有快表的地址变换机构

时间局部性：如果执行了程序中的某种指令，那么不久后这条指令有肯能再次执行：如果某个数据被访问过，不久之后该数据很可能再次被访问（循环）

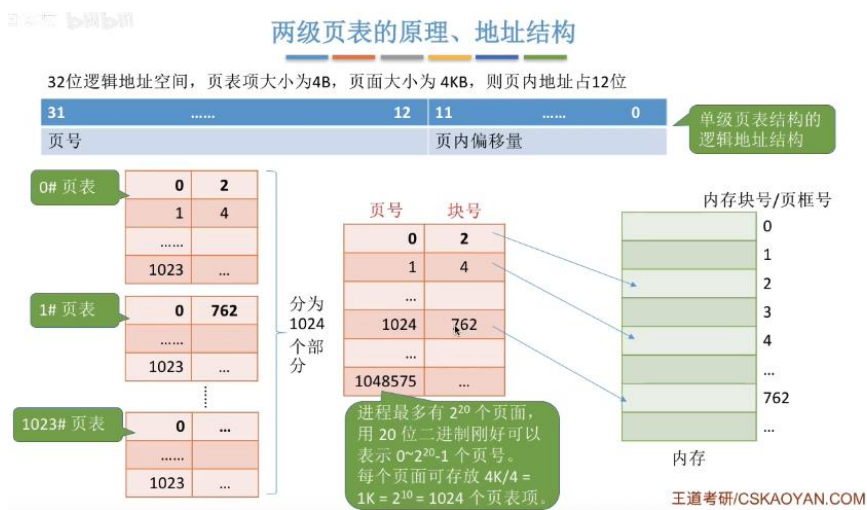
空间局部性：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也有可能被访问（数据在内存中都是连续存放的）

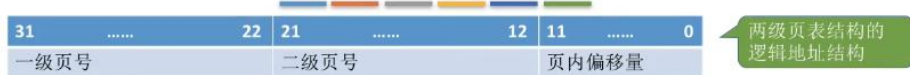
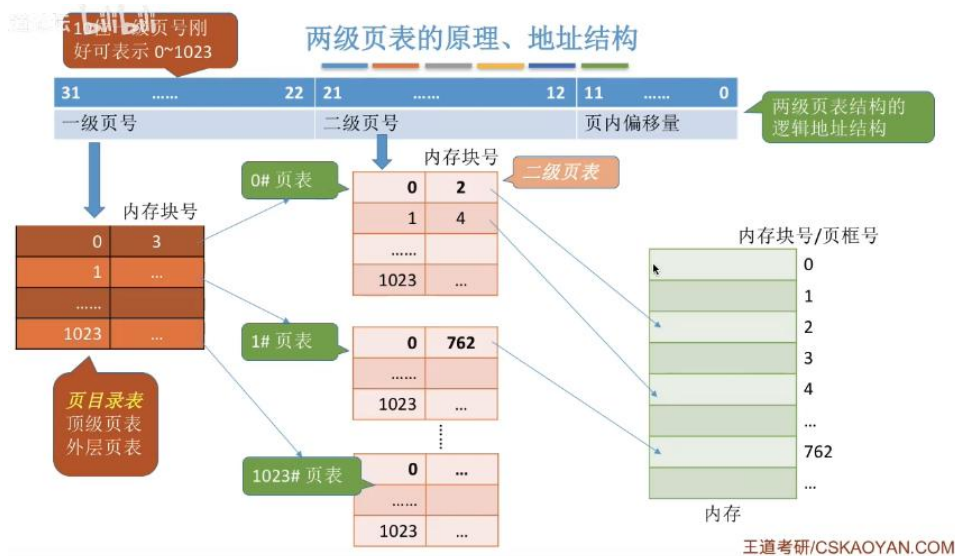
快表（一次访存）：联想寄存器，是一种访问速度比内存快很多的高速缓冲存储器，用来存放当前访问的若干页表项，以加速地址变换的过程。与此相应，内存中的页表称为慢表（两次访存）



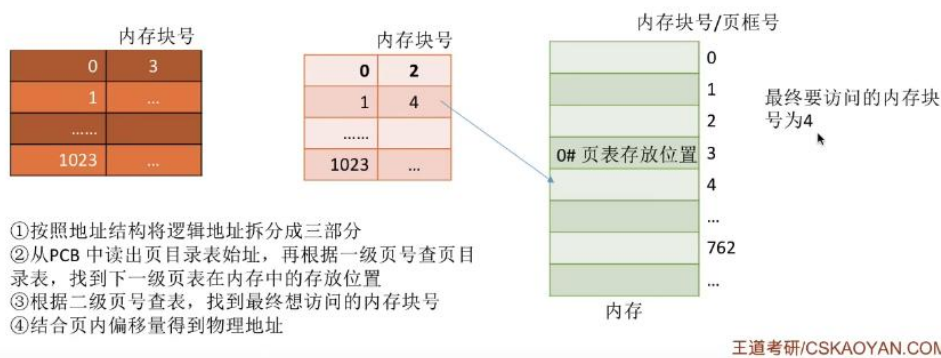
优先检查快表；若命中即可知到页面存放的内存块号；否则查询慢表

两级页表





例：将逻辑地址 (0000000000,0000000001,1111111111) 转换为物理地址（用十进制表示）。



当页表很大时采用两级页表结构，来解决需要占用很多个连续的页框

可以在需要访问页面时才把页面调入内存（虚拟存储技术）。可以在页表项中增加一个

标志位，用于表示该页面是否已经调入内存；若想访问的页面不在内存中，则产生缺页中断

（内中断），然后将目标页面从外存调入内存

1. 若采用多级页表机制，则各级页表的大小不能超过一个页面

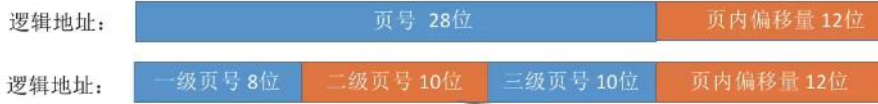
例：某系统按字节编址，采用 40 位逻辑地址，页面大小为 4KB，页表项大小为 4B，假设采用纯页式存储，则要采用（ ）级页表，页内偏移量为（ ）位？

页面大小 = 4KB = 2^{12} B，按字节编址，因此页内偏移量为 12 位

页号 = 40 - 12 = 28 位

页面大小 = 2^{12} B，页表项大小 = 4B，则每个页面可存放 $2^{12} / 4 = 2^{10}$ 个页表项

因此各级页表最多包含 2^{10} 个页表项，需要 10 位二进制位才能映射到 2^{10} 个页表项，因此每一级的页表对应页号应为 10 位。总共 28 位的页号至少要分为三级



2. 两级页表的访存次数分析（假设没有快表机构）

第一次访存：访问内存中的页目录表

第二次访存：访问内存中的二级页表

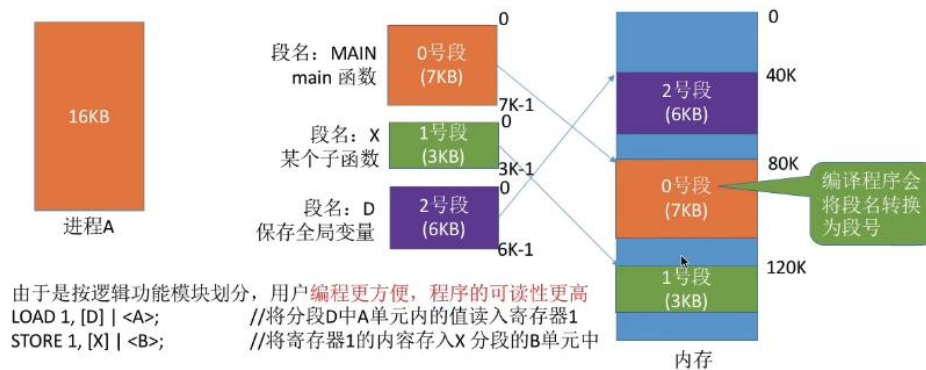
第三次访存：访问目标内存单元

如果只分为两级页表，则一级页号占 18 位，也就是说页目录表中最多可能有 2^{18} 个页表项，显然，一个页面是放不下这么多页表项的。

基本分段存储管理

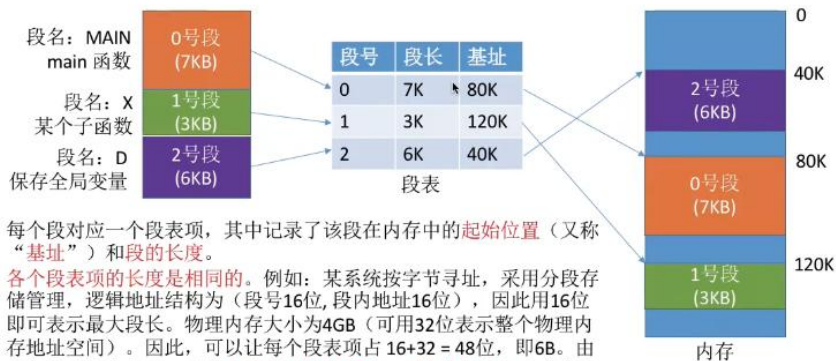
进程的地址空间：按照程序自身的逻辑关系划分为若干个段，每个段都有一个段名（在低级语言中，程序员使用段名来编程），每段从 0 开始编址

内存分配规则：以段为单位进行分配，每个段在内存中占据连续空间，但各段之间可以不相邻。



段表：每个进程建立一张段映射表

问题：程序分多个段，各段离散地装入内存，为了保证程序能正常运行，就必须能从物理内存中找到各个逻辑段的存放位置。为此，需为每个进程建立一张段映射表，简称“段表”。

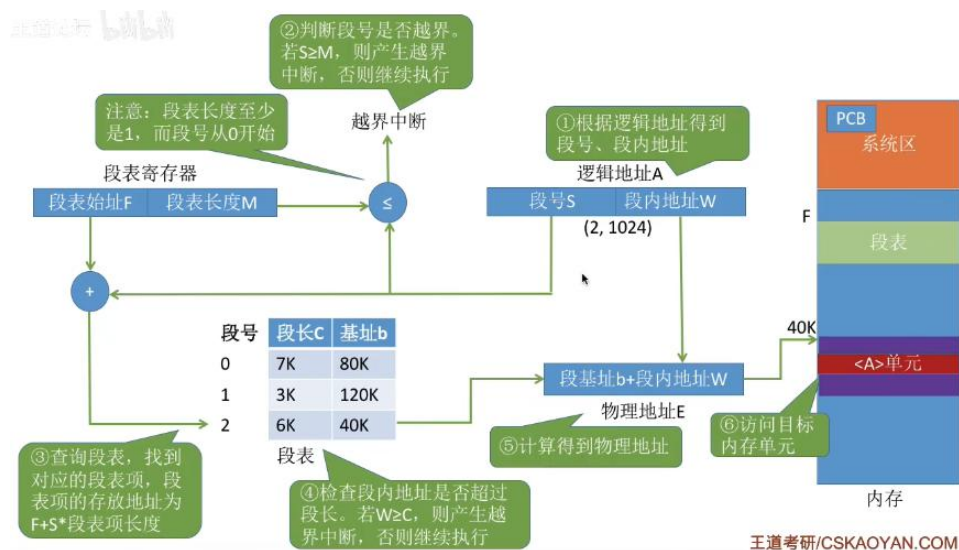


1. 每个段对应一个段表项，其中记录了该段在内存中的起始位置（又称“基址”）和段的长度。

2. 各个段表项的长度是相同的。例如：某系统按字节寻址，采用分段存储管理，逻辑地址结构为（段号 16 位，段内地址 16 位），因此用 16 位即可表示最大段长。物理内存大小为 4GB（可用 32 位表示整个物理内存地址空间）。因此，可以让每个段表项占 $16 + 32 = 48$ 位，即 6B。由于段表项长度相同，因此段号可以是隐含的，不占存储空间。若段表存放的起始地址为 M，则 K 号段对应的段表项存放的地址为 $M + K * 6$

王道考研/CSKAOYAN.COM

地址变换过程：



分页、分段管理的对比

页是信息的物理单位。分页的目的只要是为了实现离散分配，提高内存利用率。分页仅仅是系统管理上的需要，完全是系统行为，**对用户是不可见的**

段是信息的逻辑单位。分段主要目的是更好地满足用户需求。一个段通常包含着一组属于一个逻辑模块的信息。**分段是对用户可见的**，用户编程时需要显示的给出段名

页的大小固定且由系统决定；段的长度不固定，决定于用户编写的程序

分页地用户进程地址空间是**一维**的。

分段的用户进程地址空间是**二维**的

分段比分页更容易实现信息的共享和保护

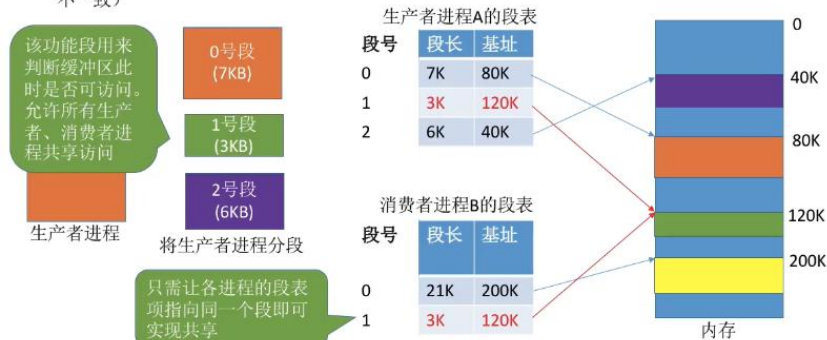
不能被修改的代码称为纯代码或者可重入代码（不属于临界资源）

分段、分页管理的对比

比如，有一个代码段只是简单的输出“Hello World!”

分段比分页更容易实现信息的共享和保护。

不能被修改的代码称为**纯代码**或**可重入代码**（不属于临界资源），这样的代码是可以共享的。可修改的代码是不能共享的（比如，有一个代码段中有很多变量，各进程并发地同时访问可能造成数据不一致）



段页式管理方式

分页+分段的结合

分页管理：优点：内部空间利用率高，不会产生外部碎片，只会有少量的页内碎片

缺点：不方便按照逻辑模块实现信息的共享和保护

分段管理：优点：很方便按照逻辑模块实现信息的共享和保护

缺点：如果段长过大，分配很大的连续空间会很不方便。会产生外部碎片。

分段系统的逻辑地址结构由段号和段内地址（段内偏移量）组成。如：

31		16	15		0
段号			段内地址		

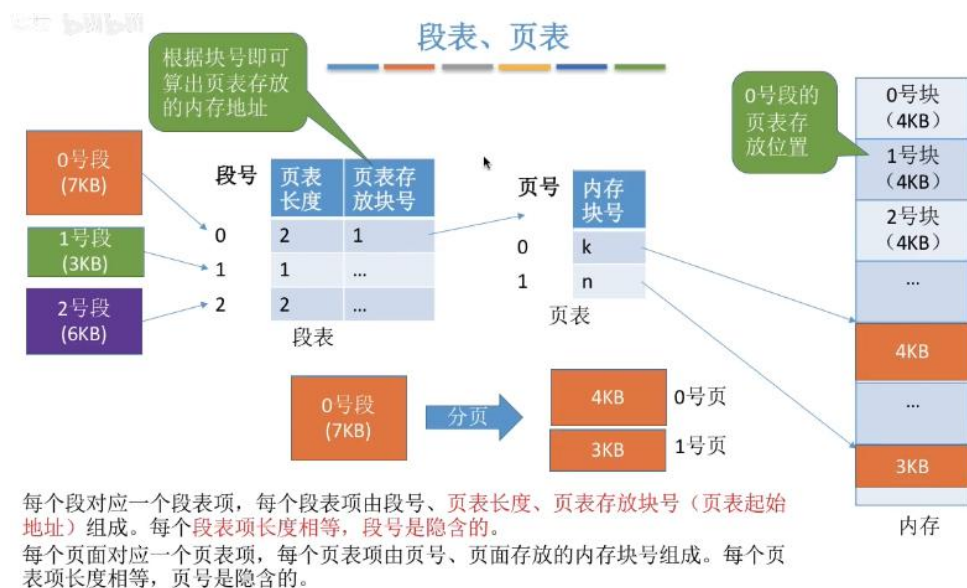
段页式系统的逻辑地址结构由段号、页号、页内地址（页内偏移量）组成。如：

31		16	15		12	11		0
段号			页号			页内偏移量		

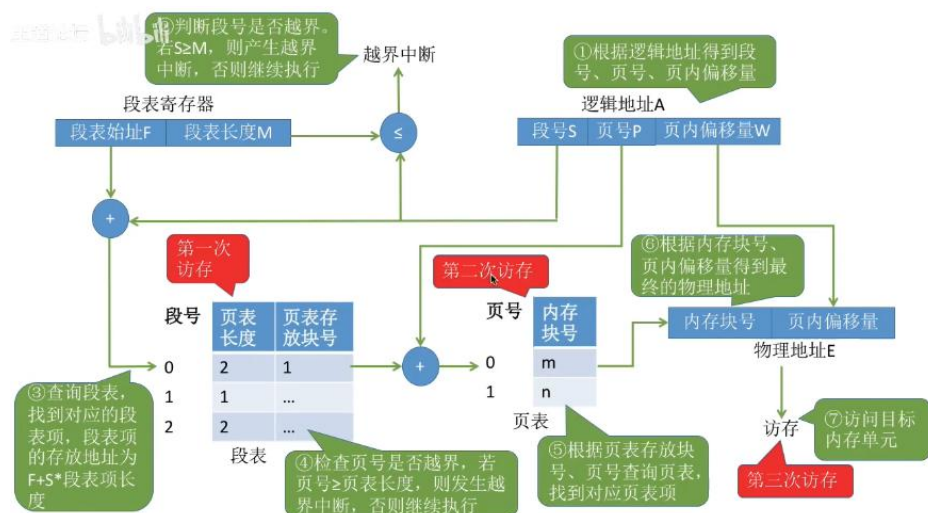
段号的位数决定了每个进程最多可以分几个段
 页号位数决定了每个段最大有多少页
 页内偏移量决定了页面大小、内存块大小是多少

在上述例子中，若系统是按字节寻址的，则
 段号占16位，因此在该系统中，每个进程最多有 $2^{16} = 64K$ 个段
 页号占4位，因此每个段最多有 $2^4 = 16$ 页
 页内偏移量占12位，因此每个页面\每个内存块大小为 $2^{12} = 4096 = 4KB$

“分段”对用户是可见的，程序员编程时需要显式地给出段号、段内地址。而将各段“分页”对用户是不可见的。系统会根据段内地址自动划分页号和页内偏移量。因此段页式管理的地址结构是二维的。



地址转换



虚拟内存

传统存储管理方式：

一次性：作业必须一次性全部装入内存后才能开始运行

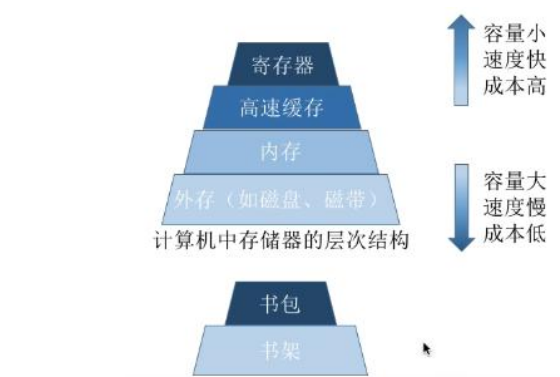
a) 作业很大时，不能全部装入内存，导致**大作业无法运行**

b) 大量作业要求运行时，**多道程序并发度下降**

驻留性：一旦作业被装入内存，就会一直驻留在内存

高速缓冲技术：将近期会频繁访问到的数据放到更高速的存储器中，暂时用不到的数据

放在更低速存储器中（快表机构）



基于局限性原理，在程序装入时，可以将程序很快用到的部分装入内存。暂时用不到的部分留在外存，就可以让程序开始执行。当访问的信息不在内存时，由操作系统负责将所需信息从外存调入内存，然后继续执行程序，若内存空间不够时，由操作系统将内存中暂时用不到的信息换出到外存

在操作系统的管理下，在用户看来似乎有一个比实际内存大得多的内存，这就是虚拟内存

虚拟内存有三个主要特征：

- a)：多次性：无需作业在运行时一次性全部装入内存，而是允许被分成多次调入内存
- b)：对换性：在作业运行时无需一直常驻内存，而是允许在作业运行过程中，将作业换入、换出

- c)：虚拟性：从逻辑上扩充了内存的容量

虚拟内存技术的实现需要建立在离散分配的内存管理方式基础上

- 1) 请求分页存储管理
- 2) 请求分段存储管理
- 3) 请求段页式存储管理

主要区别：请求调页功能：当访问的信息不在内存时，由操作系统负责将所需信息从外存调入内存

页面置换功能：若内存空间不足时，由操作系统将内存中暂时用不到的信息换出外存

请求分页管理方式

1) 页表机制

2) 缺页中断机构

3) 地址变换机构

请求分页存储管理与基本分页存储管理的主要区别：

在程序执行中，当访问的信息不在内存时，由操作系统负责将所需信息从外存调入内存；

若内存空间不足时，由操作系统负责将内存中暂时用不到的信息换出到外存

页表机制



与基本分页管理相比，请求分页管理中，为了实现“请求调页”，操作系统需要知道每个页面是否已经调入内存；如果还没调入，那么也需要知道该页面在外存中存放的位置。

当内存空间不够时，要实现“页面置换”，操作系统需要通过某些指标来决定到底换出哪个页面：有的页面没有被修改过，就不用再浪费时间写回外存。有的页面修改过，就需要将外存中的旧数据覆盖，因此，操作系统也需要记录各个页面是否被修改的信息。



缺页中断机构

在请求分页系统中，每当要访问的**页面不在内存**时，便产生一个**缺页中断**，然后由操作

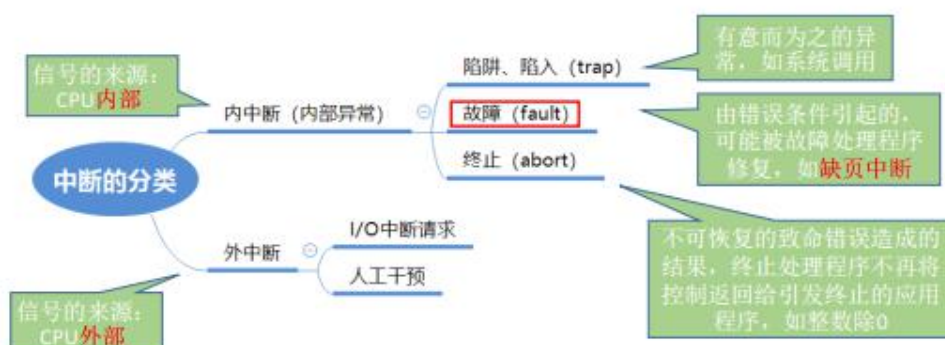
系统的**缺页中断处理程序**处理中断

此时缺页的进程**阻塞**，放入阻塞队列，调页完成后再次将其唤醒，放回就绪队列；

如果内存中有**空闲块**，则为其分配一个**内存块**，并修改页表中相应的页表项

如果内存中没有空闲块，则由页面置换算法选择一个页面淘汰，若该页面在内存期间被修改过，则要将其写回外存。

缺页中断是因为当前执行的指令想要访问的目标页面未调入内存而产生的，因此**属于内中断**。一条指令在执行期间，**可能产生多次缺页中断**。（如：copy A to B，即将逻辑地址A中的数据复制到逻辑地址B，而A、B属于不同的页面，则有可能产生两次中断）



地址变换机构

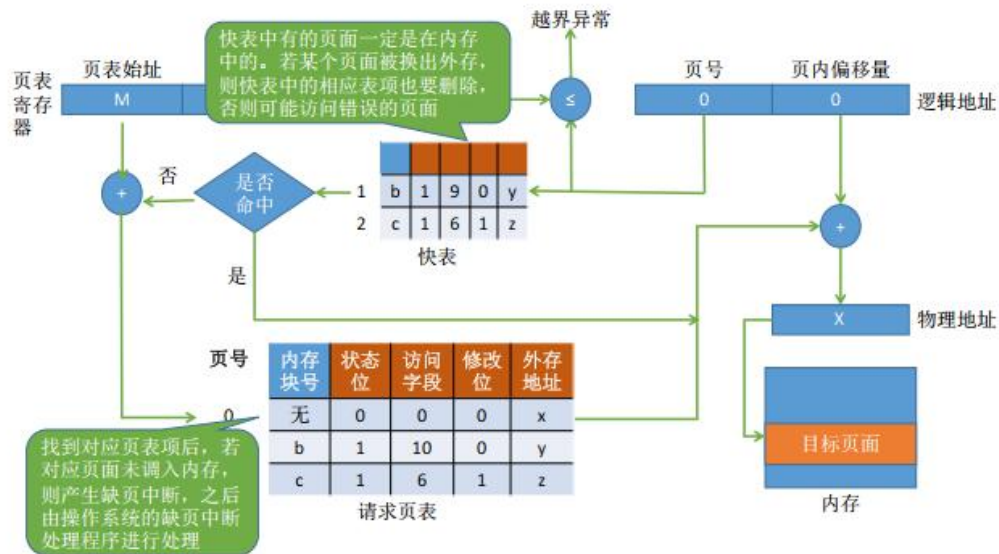
请求分页存储管理与基本分页存储管理的主要区别：

在程序执行过程中，当访问的信息不在内存时，由操作系统负责将所需信息从外存调入内存，然后继续执行程序；若内存空间不够，由操作系统将内存中暂时用不到的信息换出到外存

新增步骤：1) 请求调页

2) 页面置换

3) 需要修改请求页表中新增的表项



页面置换算法

最佳置换算法 (OPT)

最佳置换算法 (OPT, Optimal): 每次选择淘汰的页面将是以后永不使用，或者在最长时间内不再被访问的页面，这样可以保证最低的缺页率。

例: 假设系统为某进程分配了三个内存块，并考虑到有一下页面号引用串 (会依次访问这些页面): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
内存块1	7	7	7	2		2		2			2			2				7		
内存块2		0	0	0		0		4			0			0				0		
内存块3			1	1		3		3			3			1				1		
是否缺页	√	√	√	√		√		√			√			√				√		

选择从 0, 1, 7 中淘汰一页。按最佳置换的规则，往后寻找，最后一个出现的页号就是要淘汰的页面

整个过程缺页中断发生了9次，页面置换发生了6次。
注意：缺页时未必发生页面置换。若还有可用的空闲内存块，就不用进行页面置换。

缺页率 = $9/20 = 45\%$

最佳置换算法: 每次选择淘汰的页面将是以后永不使用，或者在最长时间内不被访问的页面，这样可以保证最低的缺页率；但实际上，最佳置换算法是无法实现的

先进先出置换算法 (FIFO)

先进先出置换算法 (FIFO)

先进先出置换算法 (FIFO)：每次选择淘汰的页面是**最早进入内存的页面**

实现方法：把调入内存的页面根据调入的先后顺序排成一个队列，需要换出页面时选择队头页面即可。
队列的最大长度取决于系统为进程分配了多少个内存块。

例：假设系统为某进程分配了三个内存块，并考虑到有以下页面号引用串：

3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4

访问页面	3	2	1	0	3	2	4	3	2	1	0	4
内存块1	3	3	3	0	0	0	4			4	4	
内存块2		2	2	2	3	3	3			1	1	
内存块3			1	1	1	2	2			2	0	
是否缺页	√	√	√	√	√	√	√			√	√	



分配三个内存块时，缺页次数：9次

只有 FIFO 算法会产生 Belady 异常。虽然 FIFO 算法实现简单，但是该算法与进程实际

运行时的规律不适应，因为先进入的页面也有可能最经常被访问。因此，算法性能差

最近最久未使用置换算法 (LRU)

最近最久未使用置换算法 (LRU, least recently used)：每次淘汰的页面是**最近最久未使用的页面**

实现方法：赋予每个页面对应的页表项中，用访问字段记录该页面自上次被访问以来所经历的时间t。当需要淘汰一个页面时，选择现有页面中t值最大的，即最近最久未使用的页面。

页号	内存块号	状态位	访问字段	修改位	外存地址
----	------	-----	------	-----	------

例：假设系统为某进程分配了四个内存块，并考虑到有以下页面号引用串
1, 8, 1, 7, 8, 2, 7, 2, 1, 8, 3, 8, 2, 1, 3, 1, 7, 1, 3, 7

访问页面	1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
内存块1	1	1	1	1	1	1				1	1					1	1			
内存块2		8		8	8	8				8	8					8	8			
内存块3				7	7	7				7	7					7	7			
内存块4					2	2				2	2					2	2			
缺页否	√	√		√	√	√				√	√					√	√			

该算法的实现需要专门的硬件支持，虽然算法性能好，但是实现困难，开销大

在手动做题时，若需要淘汰页面，可以逆向检查此时在内存中的几个页面号。在逆向扫描过程中最后一个出现的页号就是要淘汰的页面。

时钟置换算法 (CLOCK)

最佳置换算法性能最好，但无法实现；先进先出置换算法实现简单，但算法性能差；最近最久未使用置换算法性能好，是最接近OPT算法性能的，但是实现起来需要专门的硬件支持，算法开销大。

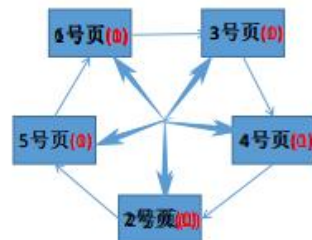
时钟置换算法是一种性能和开销较均衡的算法，又称CLOCK算法，或最近未用算法 (NRU, Not Recently Used)

简单的CLOCK算法实现方法：为每个页面设置一个访问位，再将内存中的页面都通过链接指针链接成一个循环队列。当某页被访问时，其访问位置为1。当需要淘汰一个页面时，只需检查页的访问位。如果是0，就选择该页换出；如果是1，则将它置为0，暂不换出，继续检查下一个页面，若第一轮扫描中所有页面都是1，则将这些页面的访问位依次置为0后，再进行第二轮扫描（第二轮扫描中一定会有访问位为0的页面，因此简单的CLOCK算法选择一个淘汰页面最多会经过两轮扫描）

页号	内存块号	状态位	访问位	修改位	外存地址
----	------	-----	-----	-----	------

访问位为1，表示最近访问过；
访问位为0，表示最近没访问过

例：假设系统为某进程分配了五个内存块，并考虑到有以下页面号引用串：
1, 3, 4, 2, 5, 6, 3, 4, 7



王道考研/CSKAOYAN.COM

改进型的时钟置换算法

简单的时钟置换算法仅考虑到一个页面最近是否被访问过。事实上，如果被淘汰的页面没有被修改过，就不需要执行I/O操作写回外存。只有被淘汰的页面被修改过时，才需要写回外存。

因此，除了考虑一个页面最近有没有被访问过之外，操作系统还应考虑页面有没有被修改过。在其他条件都相同时，应优先淘汰没有修改过的页面，避免I/O操作。这就是改进型的时钟置换算法的思想。

修改位=0，表示页面没有被修改过；修改位=1，表示页面被修改过。

为方便讨论，用（访问位，修改位）的形式表示各页面状态。如（1，1）表示一个页面近期被访问过，且被修改过。

算法规则：将所有可能被置换的页面排成一个循环队列

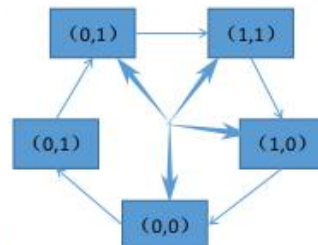
第一轮：从当前位置开始扫描到第一个（0，0）的帧用于替换。本轮扫描不修改任何标志位

第二轮：若第一轮扫描失败，则重新扫描，查找第一个（0，1）的帧用于替换。本轮将所有扫描过的帧访问位设为0

第三轮：若第二轮扫描失败，则重新扫描，查找第一个（1，0）的帧用于替换。本轮扫描不修改任何标志位

第四轮：若第三轮扫描失败，则重新扫描，查找第一个（1，1）的帧用于替换。

由于第二轮已将所有帧的访问位设为0，因此经过第三轮、第四轮扫描一定会有一个帧被选中，因此改进型CLOCK置换算法选择一个淘汰页面最多会进行四轮扫描



第一优先级：最近没访问且没修改的页面

第二优先级：最近没访问但修改过的页面

第三优先级：最近访问过但没修改的页面

第四优先级：最近访问过且修改过的页面

五种算法比较

	算法规则	优缺点
OPT	优先淘汰最长时间内不会被访问的页面	缺页率最小，性能最好；但无法实现
FIFO	优先淘汰最先进入内存的页面	实现简单；但性能很差，可能出现Belady异常
LRU	优先淘汰最近最久没访问的页面	性能很好；但需要硬件支持，算法开销大
CLOCK（NRU）	循环扫描各页面 第一轮淘汰访问位=0的，并将扫描过的页面访问位改为1。若第一轮没选中，则进行第二轮扫描。	实现简单，算法开销小；但未考虑页面是否被修改过。
改进型CLOCK（改进型NRU）	若用（访问位，修改位）的形式表述，则 第一轮：淘汰（0，0） 第二轮：淘汰（0，1），并将扫描过的页面访问位都置为0 第三轮：淘汰（0，0） 第四轮：淘汰（0，1）	算法开销较小，性能也不错

页面分配策略

驻留集：指请求分配存储管理中给进程分配的物理块的集合；在采用了虚拟存储技术系统中，驻留集大小一般小于进程的总大小。

固定分配：驻留集大小不变

可变分配：驻留集大小可以调整

局部置换：当发生缺页时，只能选择进程自己的物理块进行置换

全局置换：可以将操作系统保留的空闲物理块分配给缺页进程，也可以将别的进程持有的物理块置换到外存，再分配给缺页进程

固定分配局部置换

系统为每个进程分配一定数量的物理块，在整个运行期间都不改变。若进程在运行中发生缺页，则只能从该进程在内存中的页面中选出一页换出，然后再调入需要的页面。

缺点：很难在刚开始就确定应为每个进程分配多少个物理块才算合理

可变分配全局置换

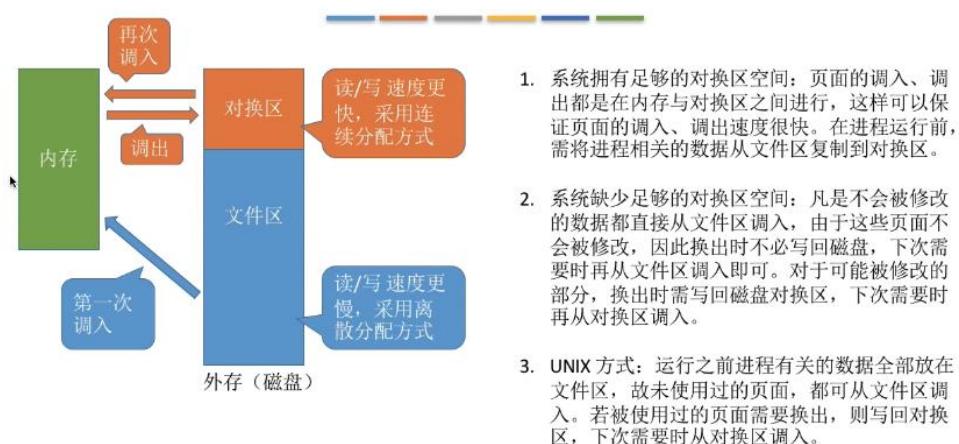
刚开始会为每个进程分配一定数量的物理块。操作系统会保持一个空闲物理块队列。当某进程发生缺页时，从空闲物理块中取出一块分配给该进程；若已无空闲物理块，则可选择将一个未锁定的页面换出外存，再将该物理块分配给缺页的进程。采用这种策略时，**只要某进程发生缺页都将获得新的物理块**，仅当空闲物理块用完时，系统才选择一个未锁定的页面调出。被选择调出的页可能是系统中任何一个进程中的页，因此这个**被选中的进程拥有的物理块会减少，缺页率会增加**。

可变分配局部置换

刚开始会为每个进程分配一定数量的物理块。当某进程发生缺页时，只允许从该进程自己的物理块中选出一个进行换出外存。**系统根据发生缺页的频率来动态地增加或减少进程的物理块**。

预调页策略：一次调入若干个相邻的页面。主要用于进程地首次调入（运行前调入）

请求调页策略：进程在运行期间发现缺页时才将所缺页面调入内存（运行时调入）



抖动(颠簸)现象 : 刚刚换出的页面马上又换入内存 , 刚刚换入的页面马上又换出内存 ;

产生的**主要原因**是进程频繁访问的页面数目高于可用的物理块数(**分配给进程的物理块不够**)

工作集 : 指在某段时间间隔内 , 进程实际访问页面的集合

操作系统会根据“窗口尺寸”来算出工作集。例：
某进程的页面访问序列如下，**窗口尺寸为4**，各时刻的工作集为？



工作集大小可能小于窗口尺寸，实际应用中，操作系统可以统计进程的工作集大小，根据工作集大小给进程分配若干内存块。如：窗口尺寸为5，经过一段时间的监测发现某进程的工作集最大为3，那么说明该进程有很好的局部性，可以给这个进程分配3个以上的内存块即可满足进程的运行需要。
一般来说，**驻留集大小不能小于工作集大小**，否则进程运行过程中将频繁缺页。

拓展：基于局部性原理可知，进程在一段时间内访问的页面与不久之后会访问的页面是有相关性的。因此，可以根据进程近期访问的页面集合（工作集）来设计一种页面置换算法——选择一个不在工作集中的页面进行淘汰。

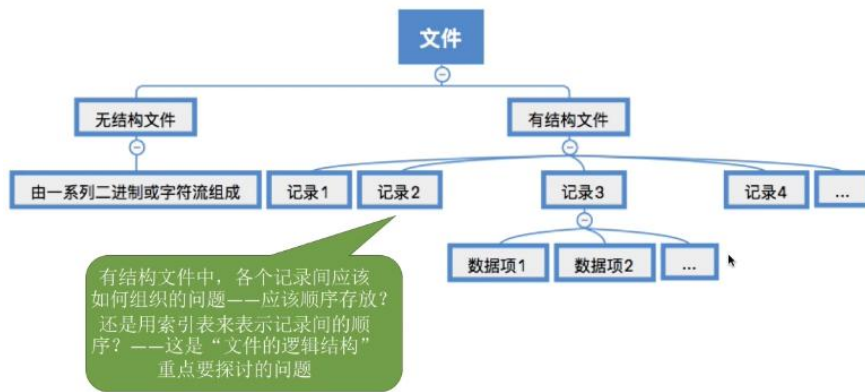
文件

文件管理

文件名、标识符、类型、位置、大小、创建时间、上次修改时间、文件所有者信息、保护信息

无结构文件（文本文件）：由一些二进制或字符流组成，又称流式文件

有结构文件（数据库表）：由一组相似的记录组成，又称记录式文件



目录其实也是一种特殊的有结构文件，所谓的“目录”其实就是我们熟悉的“文件夹”

操作系统向上提供的功能

可以“创建”文件：当创建文件时 → create 系统调用

可以“读”文件：将文件数据读入内存，才能让 CPU 处理 → read 系统调用

可以“写”文件：将文件数据从内存写回外存 → write 系统调用

可以“删除”文件：删除文件 → delete 系统调用

外存中存放的文件

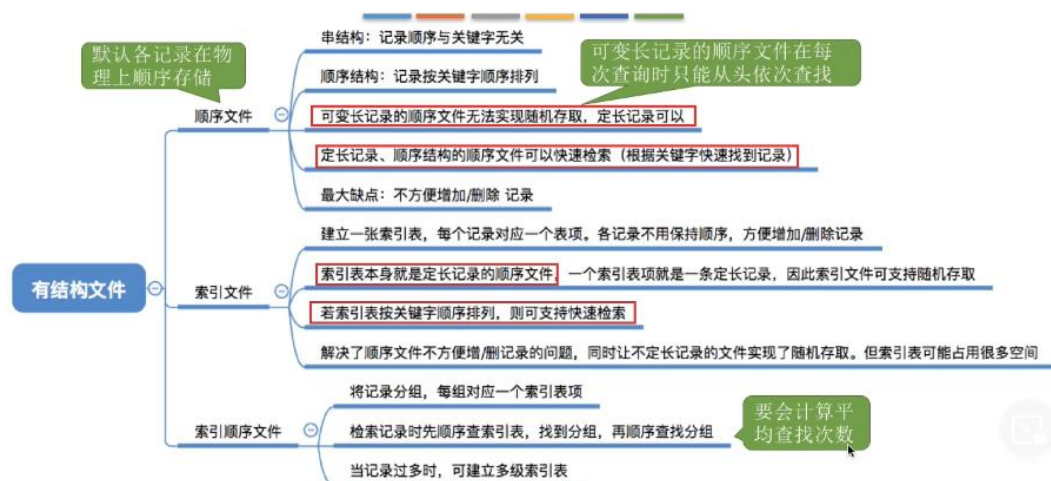
操作系统以“块”为单位为文件分配存储空间，因此即使一个文件大小只有 10B，但它依然需要占用 1KB 的磁盘块。外存中的数据读入内存时同样以块为单位；外存会分为一个个“块/磁盘块/物理块”大小是相等的；与内存一样，外存也是由一个个存储单元组成的，每个存储单元可以存储一定量的数据

文件的逻辑结构

按文件是否有结构分类：有结构文件、无结构文件

有结构文件：由一组相似的记录组成，又称记录式文件。每条记录有一个数据项可作为

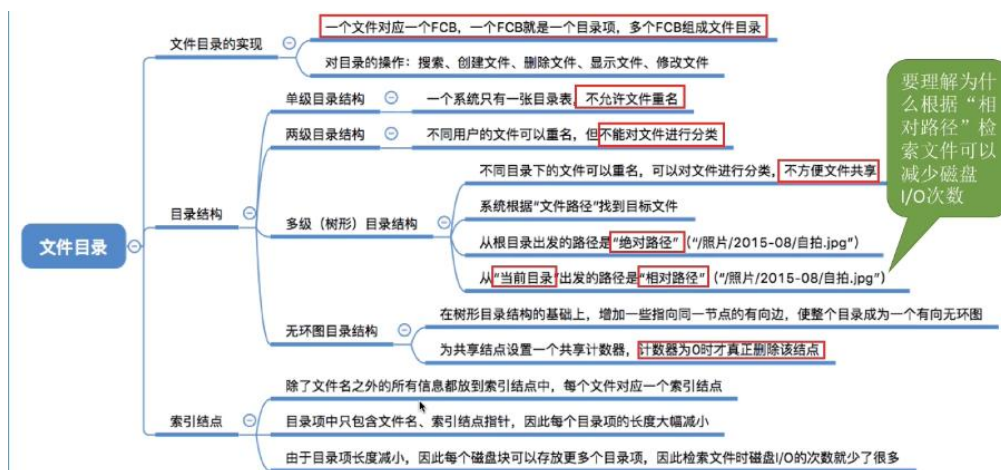
关键字。根据各条记录的长度(占用的存储空间)是否相等,又可以分为定长记录和可变长。



文件目录

文件控制块(实现文件目录的关键数据结构)、目录结构、索引结点(文件控制块优化)

文件控制块: 目录文件中的一条记录就是一个“文件控制块”(FCB); FCB地有序集合称为“文件目录”, 一个FCB就是一个文件目录项。FCB中包含了文件的基本信息、存取控制信息、实用信息



文件的物理结构

对非空闲磁盘块的管理

文件的物理结构：连续分配、链接分配（隐式链接、显式链接）、索引分配

类似于内存分页，磁盘中的存储单元也会被分为一个个“块/磁盘块”。很多操作系统中，磁盘块的大小与内存块、页面的大小相同；内存与磁盘之间的数据交换都是以“块”为单位进行的，即每次连续读入一块，或写入一块。

在外存管理中，为了方便对文件数据的管理。文件的逻辑地址空间也被分为了一个个地文件“块”。于是文件的逻辑地址也可以表示为（逻辑块号，块内地址）的形式。

连续分配：要求每个文件在磁盘上占有一组连续的块。

（逻辑块号，块内地址）→（物理块号，块内地址）的转变：文件目录中记录存放的起始块号和长度（总共占用几个块）用户给出想要访问的逻辑块号，操作系统找到该文件的FCB：**物理块号 = 起始块号 + 逻辑块号**。因此连续分配支持顺序访问和直接访问。

结论：连续分配的文件在**顺序读/写时速度最快（磁盘指针）**；物理上采用连续分配的文件**拓展不方便**；连续分配对**存储空间利用率低，会产生难以利用的磁盘碎片**（类似于外部碎片）可以用**紧凑**来解决碎片，但需要很大的时间代价

链接分配：采用离散分配的方式，可以为文件分配离散的磁盘块。分为**隐式链接**和**显式链接**两种

a)：**隐式链接**：目录中记录了文件存放的起始块号和结束块号，除了文件的最后一个磁盘块之外，每个磁盘块都会有保存指向下一个盘块的指针，这些指针是对用户透明的

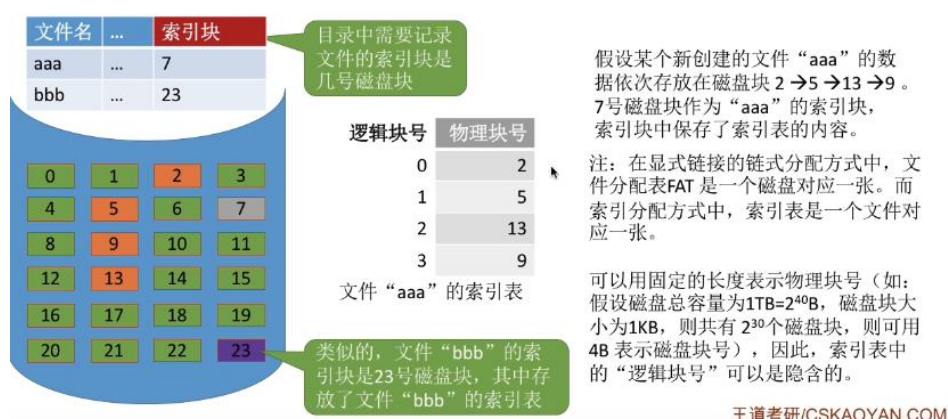
结论：采用链式分配（隐式链接）方式的文件，只支持顺序访问，不支持随机访问；很

方便文件拓展。所有的空闲磁盘块都可以被利用，不会有碎片问题，外存利用率高。

b): **显式链接**: 把用于链接文件各物理块的指针显式地存放在一张表中。即 文件分配表 (FAT), 目录中只需要记录文件的起始块号。逻辑块号转换成物理块号地过程不需要读磁盘操作。支持顺序访问也支持随机访问。相比于隐式链接来说访问速度快很多。



索引分配: 允许文件离散地分配在各个磁盘块中, 系统会为每个文件建立一张索引表, 索引表中记录了文件的各个逻辑块对应的物理块 (类似于页面)。索引表存放的磁盘块称为索引块。文件数据存放的磁盘块称为数据块。可以支持随机访问; 文件拓展也很容易实现 (只需要给文件分配一个空闲块, 并增加一个索引表即可)



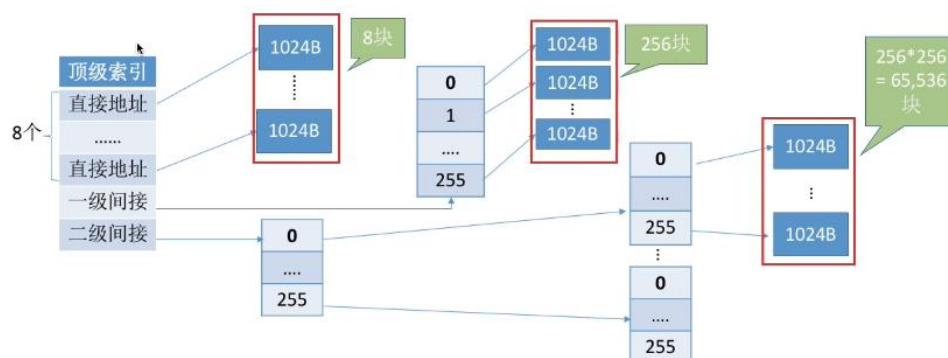
如果一个磁盘块装不下文件的整张索引表, 解决方法: **链接方案、多层索引、混合索引**。

链接方案: 一个索引块装不下, 可以将多个索引块连接起来存放。

如果想访问文件的最后一个逻辑块, 就必须找到最后一个索引块, 而各个索引块之间使用指针链接起来的, 因此必须先顺序地读入之前索引块 → 低效

多级索引：(类似于多级页表) 使第一层索引块指向第二层的索引块依次.....；即使是小文件，访问一个数据块依然需要 $k+1$ 次读磁盘

混合索引：多种索引分配方式的结合。例如一个文件的顶级索引表中，既包含**直接地址索引**(直接指向数据块)，又包含**一级间接索引**(指向单层索引表)，还包含**两级间接索引**(指向两层索引表) 对于小文件，只需要较少的都磁盘次数就可以访问目标数据块，都磁盘次数更小



总结：

	How?	目录项内容	优点	缺点
顺序分配	为文件分配的必须是连续的磁盘块	起始块号、文件长度	顺序存取速度快，支持随机访问	会产生碎片，不利于文件拓展
链接分配	隐式链接 除文件的最后一个盘块之外，每个盘块中都存有指向下一个盘块的指针	起始块号、结束块号	可解决碎片问题，外存利用率高，文件拓展实现方便	只能顺序访问，不能随机访问。
	显式链接 建立一张文件分配表(FAT)，显式记录盘块的先后关系（开机后FAT常驻内存）	起始块号	除了拥有隐式链接的优点之外，还可通过查询内存中的FAT实现随机访问	FAT需要占用一定的存储空间
索引分配	为文件数据块建立索引表。若文件太大，可采用链接方案、多层索引、混合索引	链接方案记录的是第一个索引块的块号，多层/混合索引记录的是顶级索引块的块号	支持随机访问，易于实现文件的拓展	索引表需占用一定的存储空间。访问数据块前需要先读入索引块。若采用链接方案，查找索引块时可能需要很多次读磁盘操作。

对空闲磁盘块的管理

磁盘分区（C：盘、D：盘）存储空间的划分：将物理磁盘划分为一个个文件卷

目录区：文件信息（FCB）、用于磁盘存储空间管理的信息

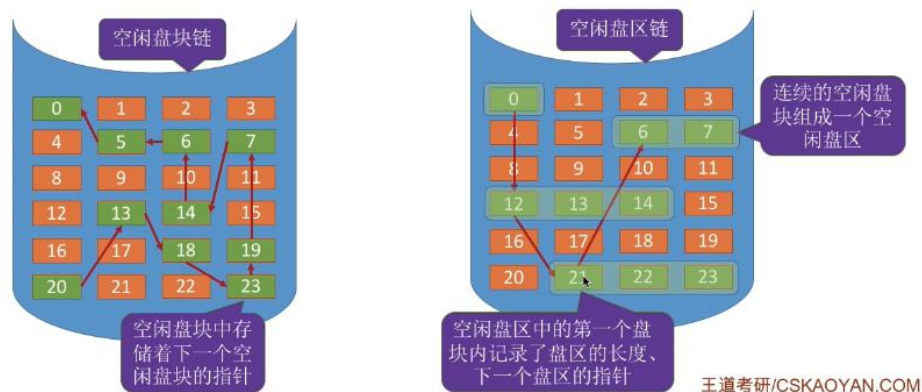
文件区：普通文件数据

空闲表法：适用于连续分配方式（首次适应、最佳适应、最坏适应等算法）

第一个空闲盘块号	空闲盘块数
0	2
5	1
10	5
18	3
23	1

回收磁盘块：回收时需要注意表项的合并现象。

空闲链表法：空闲盘块链、空闲盘区链



空闲盘块链：操作系统保存着链头、链尾指针；适用于离散分配的物理结构

分配：从链头依次摘下 K 个盘块分配，并修改空闲链的链头指针

回收：回收的盘块一次挂到链尾，并修改空闲链的链尾指针

空闲盘区链：适用于离散分配也适用于连续分配，效率更高

分配：若文件申请 K 个盘块，则可以采用首次适应、最佳适应算法，从链头开始检索；

若没有合适的连续空闲块，也可以将不同盘区的盘块同时分配给一个文件；

回收：若回收区和某个空闲盘区相邻，则需要将回收区合并到空闲盘区中，也可以将回

收区作为单独的一个空闲区挂载到链尾

位示图法：

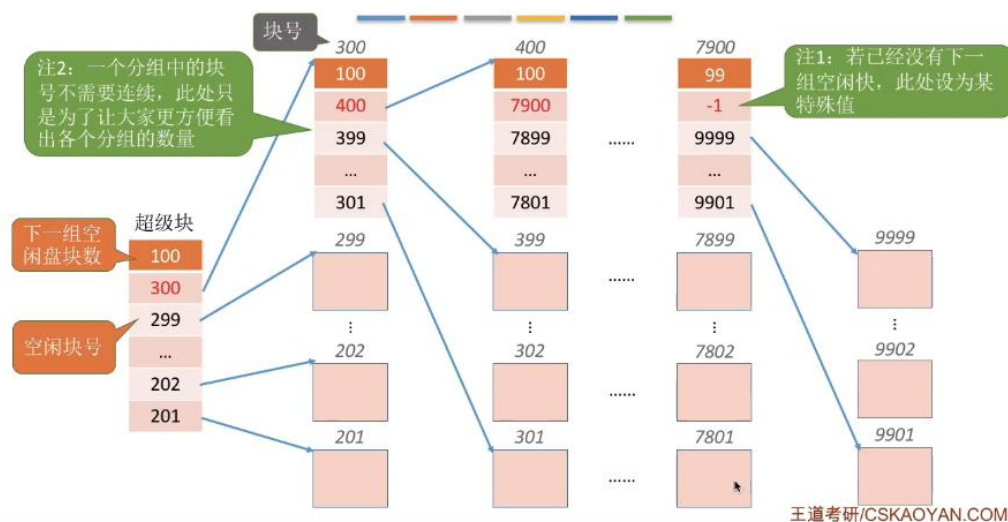


分配：若文件需要 K 个块；1) 顺序扫描位示图，找到 K 个相邻或不相邻的 “0”；2) 根据字号、位号算出对应的盘块号，将相应的盘块分配给文件；3) 将相应位设置为 “1”

回收：1) 根据回收的盘块号计算出相对应的字号、位号；2) 将相应的二进制位设为 “0”

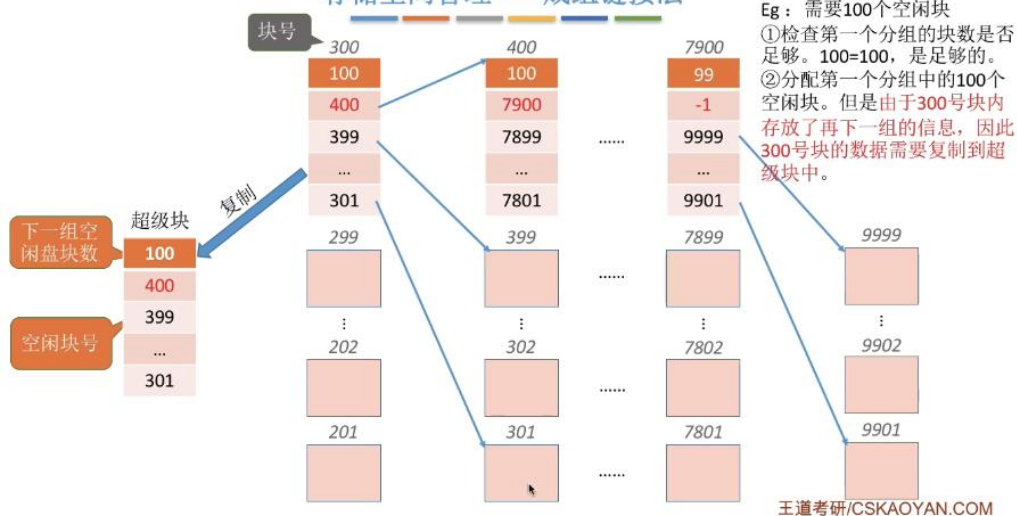
空闲表法、空闲链接法不适用于大型文件系统。UNIX 系统中采用了**成组链接法**对磁盘空闲块进行管理

文件卷的目录区中专门用一个磁盘块作为 “**超级块（链头作用）**”，当系统启动时需**要将超级块读入内存**。并且需要保证**内存、外存**中的超级块数据一致（理解即可）

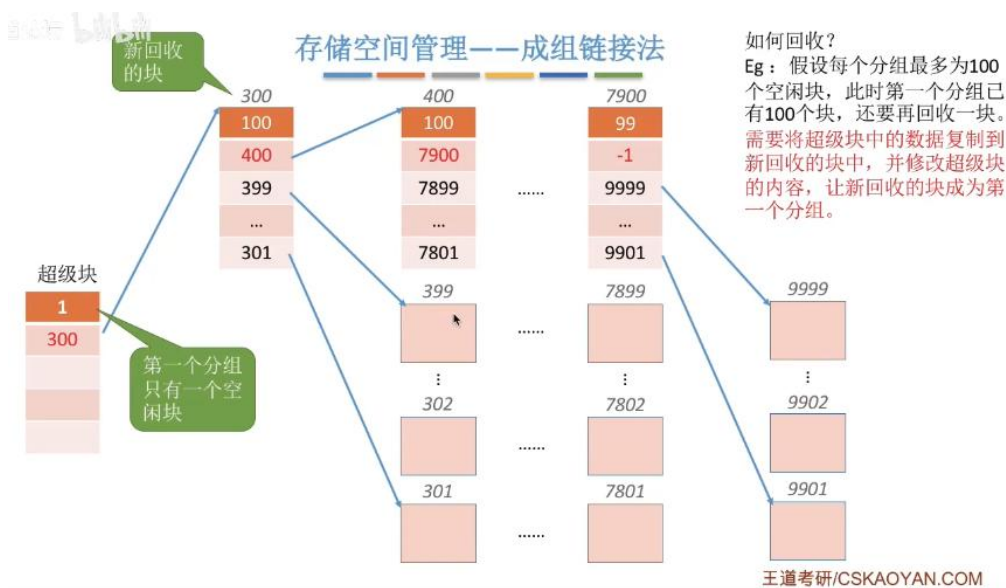


分配：

存储空间管理——成组链接法



回收：



文件的基本操作

创建文件、删除文件、读文件、写文件、打开文件、关闭文件

创建文件：进行 *create* 系统调用

1. 所需的外存空间大小

2. 文件存放路径

3. 文件名

操作系统在处理 `create` 系统调用时：

1) 在外存中找到文件所需的空间（结合空闲链表法、位示图、成组链接法）

2) 根据文件存放路径的信息找到该目录对应的目录文件，在目录中创建该文件对

应的目录项

删除文件：进行 *delete* 系统调用

1. 文件存放路径

2. 文件名

操作系统在处理 `delete` 系统调用时：

1) 根据文件存放路径的信息找到该目录对应的目录文件，在目录中创建该文件对

应的目录项

2) 根据该目录项记录的文件在外存的存放位置、文件大小等信息，回收文件占用

的磁盘块（空闲表法、空闲链表法、位图等）

3) 从目录表中删除文件对应的目录项

打开文件：进行 *open* 系统调用

1. 文件存放路径

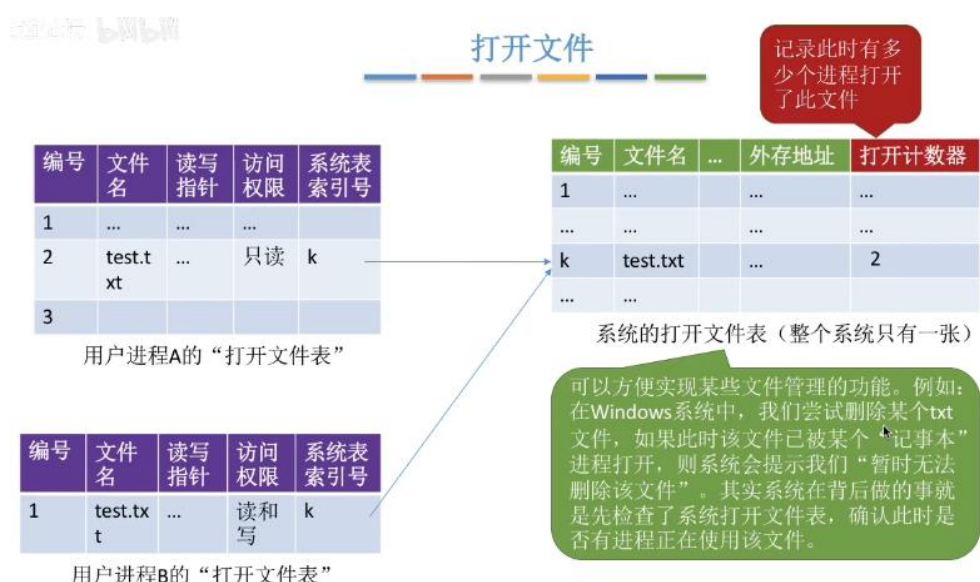
2. 文件名

3. 要对文件的操作类型

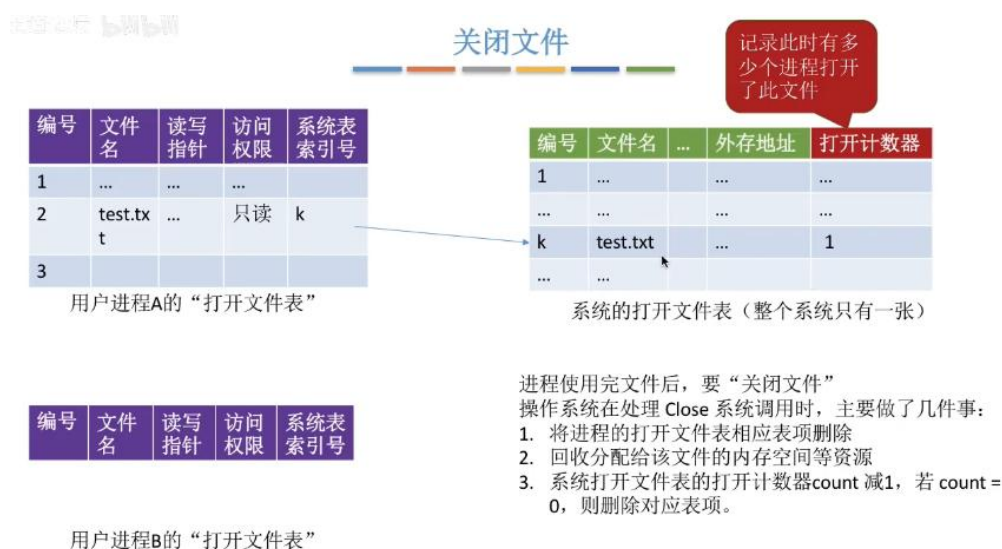
操作系统在处理 `open` 系统调用时：

1) 根据文件存放路径的信息找到该目录对应的目录文件，从目录中找到文件名对应的目录项，并检查该用户是否有指定的操作权限

2) 将目录项复制到内存中的“打开文件表”中。并将对应表目的编号返回给用户。之后用户使用打开文件表的编号来指明要操作的文件



关闭文件：进行 *close* 系统调用



读文件：进行 *read* 系统调用

操作系统在处理 *delete* 系统调用时：

会从读指针指向的外存中，将用户指定大小的数据读入用户指定的内存区域中

写文件：进行 *write* 系统调用

操作系统在处理 *write* 系统调用时：

会从用户指定的内存区域中，将指定大小的数据写回写指针指向的外存

文件共享

硬链接、软链接

文件保护

口令保护：为文件设置一个口令，用户请求访问该文件时必须提供口令

加密保护：在访问文件时需要提供正确的密码

Eg：一个最简单的加密算法——异或加密
假设用于加密/解密的“密码”为“01001”

文件的原始数据：

0	0	1	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	0	1	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

加密密码：

0	1	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1	0	0	1	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

加密结果：

0	1	1	0	0	0	0	1	0	1	0	0	1	1	1	1	1	0	0	0	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

不一致的解密密码：

0	1	1	1	1	...
---	---	---	---	---	-----

解密结果：

0	0	0	1	1	0	1	0	1	0	0	1	0	0	1	0	1	1	1	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

优点：保密性强，不需要在系统中存储“密码”

缺点：编码/译码，或者说加密/解密要花费一定时间。

访问控制：在每个文件的 FCB 中增加访问控制列表

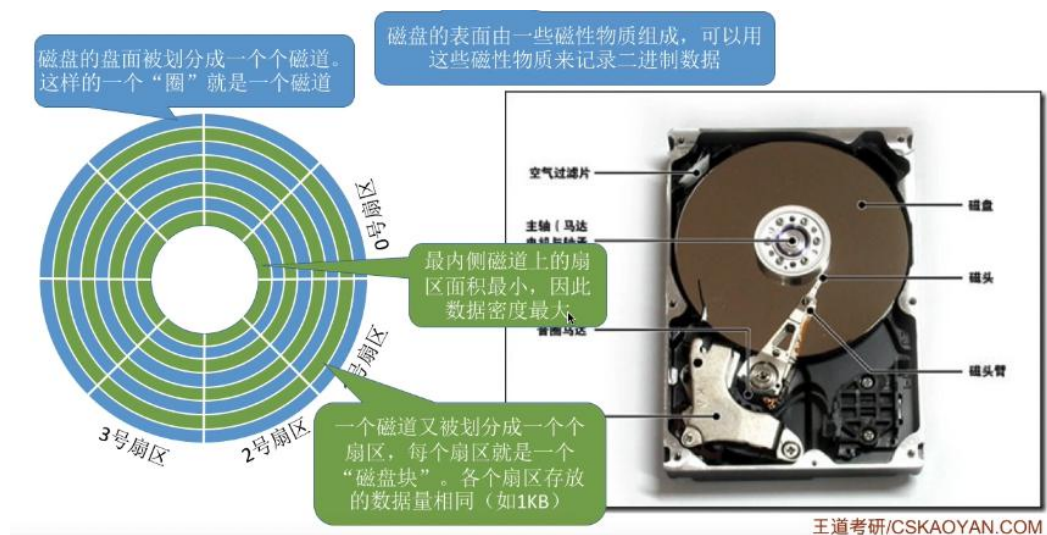
文件层次结构



磁盘

磁盘的结构

磁盘、磁道、扇区



可以用（柱面号，盘面号，扇区号）来定位任意一个“磁盘块” 所有的磁头都是连在同一个臂上的，因此所有磁头只能“共进退”

- 1) 根据 “柱面号” 移动磁臂，让磁头指向指定柱面
- 2) 激活指定柱面对应的磁头
- 3) 磁盘旋转的过程中，指定的扇区会从磁头下面划过，这样就完成了对指定扇区的读/写

磁盘调度算法（减少寻道时间）

一次磁盘读/写操作需要的时间：

- 1): 寻找时间：将磁头移动到指定磁道所花的时间。

a.启动磁头臂

b.移动磁头

- 2): 延迟时间：通过旋转磁盘，使磁头定位到目标扇区所需要的时间

- 3): 传输时间：从磁盘读出或者向磁盘写入数据所经历的时间

先来先服务算法(FCFS)

根据进程请求访问磁盘的先后顺序进行调度

优点：公平；如果请求访问的磁道比较集中的话，算法性能还算过得去

缺点：如果有大量进程竞争使用磁盘，请求访问的磁道很分散，则 FCFS 在性能上很差

最短寻找时间优先（SSTF）

SSTF 算法会优先处理的磁道是与当前磁头最近的磁道。可以保证每次的寻道时间最短，

但是并不能保证总的寻道时间最短（贪心算法）

优点：性能较好，平均寻道时间短

缺点：可能会产生饥饿现象（磁头在一个小区域内循环移动）

扫描算法 (SCAN)

只有磁头移动到最外侧磁道的时候才能往内移动, 移动到最内侧磁道的时候才能往外移动 (电梯算法)

优点: 性能较好, 不会产生饥饿现象

缺点: 只有到达最边上的磁道时才能改变磁头移动方向; 对于各个位置的磁道的响应频率不平均

LOOK 调度算法

如果在磁盘头移动方向上已经没有别的需求, 就可以立即改变磁头移动方向

优点: 不需要每次都移动到最外侧或最内侧才改变磁头方向, 缩短寻道时间

循环扫描算法

只有磁头朝某个特定方向移动时才能处理磁道访问请求, 而返回时直接快速移动至起始端而不处理任何请求

假设某磁盘的磁道为 0~200 号, 磁头的初始位置是 100 号磁道, 且此时磁头正在往磁道号增大的方向移动, 有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(200-100) + (200-0) + (90-0) = 390$ 个磁道

响应一个请求平均需要移动 $390/9 = 43.3$ 个磁道 (平均寻找长度)

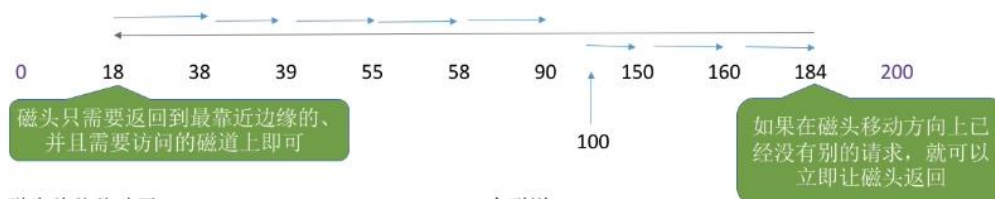
优点: 比起 SCAN 来, 对于各个位置磁道的响应频率很平均。

缺点: 只有到达最边上的磁道时才能改变磁头移动方向, 事实上, 处理了 184 号磁道的访问请求之后就不需要再往右移动磁头了; 并且, 磁头返回时其实只需要返回到 18 号磁道即可, 不需要返回到最边缘的磁道。另外, 比起 SCAN 算法来, 平均寻道时间更长。

C-LOOK 算法

如果磁头移动的方向上已经没有了磁道访问请求,就可以让磁头立即返回,并且磁头只需要返回到有磁道访问请求的位置即可

假设某磁盘的磁道为 0~200号,磁头的初始位置是100号磁道,且此时磁头正在往磁道号增大的方向移动,有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道



磁头总共移动了 $(184-100) + (184-18) + (90-18) = 322$ 个磁道

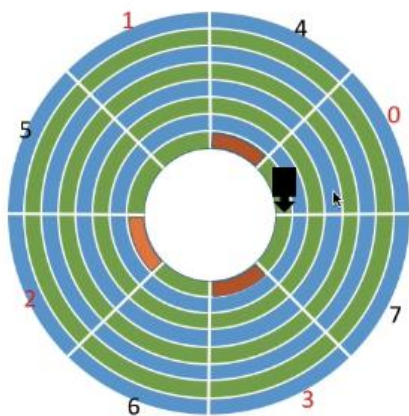
响应一个请求平均需要移动 $322/9 = 35.8$ 个磁道 (平均寻找长度)

优点: 比起 C-SCAN 算法来, 不需要每次都移动到最外侧或最内侧才改变磁头方向, 使寻道时间进一步缩短

减少磁盘延迟时间

磁头读入一个扇区数据后需要一小段时间处理,如果逻辑上相邻的扇区在物理上也相邻,则读入几个连续的逻辑扇区,可能需要很长的“延时时间”

交替编号: 让相邻的扇区在物理上有一定的间隔

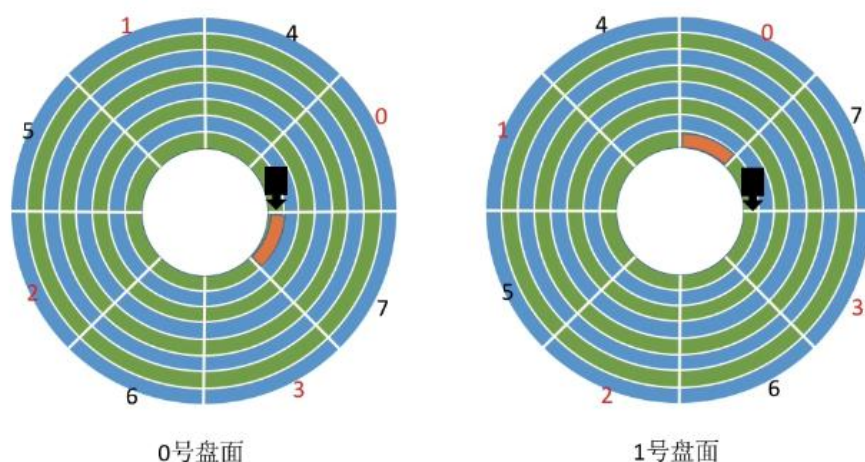


磁盘地址结构的设计

磁盘的物理结构是 (柱面号, 盘面号, 扇区号) 采用这种地址结构时可以减少磁头移动消耗的时间

错位命名：磁头读入一个扇区数据后需要一小段时间处理，所有盘面都是一起连轴转的

(根本原因)。



磁盘管理

磁盘初始化：

a)：进行低级格式化（物理格式化），将磁盘的各个磁道划分为扇区。一个扇区通常可分为头、数据区域、尾三个部分。管理扇区所需要的各种数据结构一般存放在头、尾两个部分，包括扇区校验码

b)：将磁盘分区，每个分区由若干个柱面组成（C\D\E）

c)：进行逻辑格式化，创建文件系统，包括创建文件系统的根目录、初始化存储空间管理所用的数据结构（位示图、空闲分区表）

磁盘引导块：计算机开机时需要进行一系列初始化的工作，这些初始化工作是通过执行初始化程序（自举程序）完成的；完整的自举程序放在磁盘的启动块上，启动块位于磁盘的固定位置，拥有启动分区的磁盘称为启动磁盘或者系统磁盘

ROM：存放很小的“自举装入程序”，开机时计算机先运行“自举装入程序”，通过执行该程序就可以找到引导块，并将完整的“自举程序”读入内存，完成初始化；

坏块的管理（无法使用的扇区）

在逻辑格式化时，对整个磁盘进行坏块检查，表明那些扇区是坏扇区，如在 FAT 表明

对于复杂的磁盘，磁盘控制器会维护一个坏块链表

会保留一些“备用扇区”，用于替换坏块。这种方案称为扇区备用。且这种处理方式，

坏块对操作系统透明

IO

“I/O” 就是输入输出设备（人机交互类外部设备、存储设备、网络通信设备）

UNIX 系统将外部设备抽象为一种特殊文件，用户可以使用与文件操作相同的方式对外部设备进行操作

IO 控制器

IO 控制器 → 实现 CPU 对设备的控制 又称设备控制器

1): 接受和识别 CPU 发出的命令（控制寄存器 CR）

2): 向 CPU 报告设备的状态（状态寄存器）

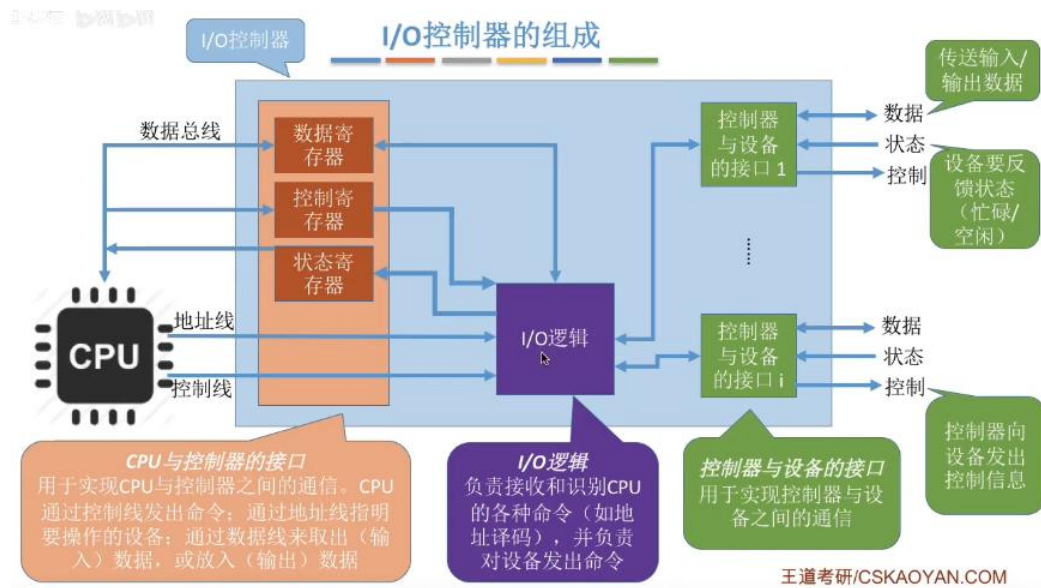
3): 数据交换（数据寄存器 DR）

4): 地址识别（IO 控制器通过 CPU 提供的地址，来判断 CPU 要读写的是那个寄存器）

注意： 一个 IO 控制器可能会对应多个设备

数据寄存器、控制寄存器、状态寄存器可能有多，这些寄存器都要有相应的地址，才能方便 CPU 操作。有的计算机让这些寄存器占用内存地址的一部分，称为内存

映像 IO；另一些计算机则采用 IO 专用地址，即寄存器独立编址。



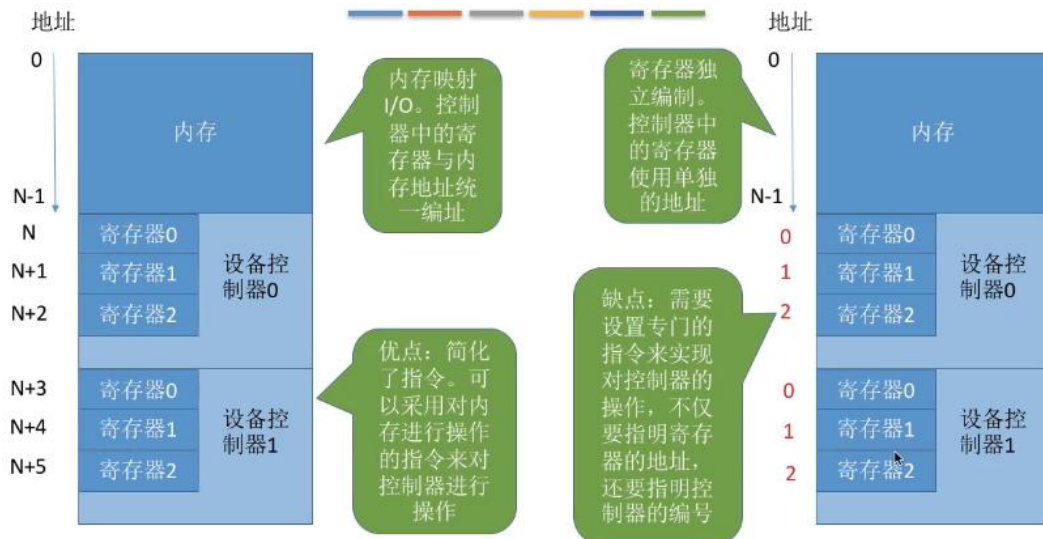
一个 IO 控制器可能会对应多个控制器于设备的接口

数据寄存器、控制寄存器、状态寄存器也会有多个

- 1) CPU 通过控制线向 IO 逻辑器发出具体的 IO 指令，在地址线说明操纵的是那个设备
- 2) CPU 通过数据总线，将所需要的数据记录到 IO 控制器的数据寄存器；IO 逻辑器从数据寄存器取得 CPU 想要输出的数据
- 3) CPU 的 IO 指令的参数放到控制寄存器；IO 逻辑器从控制器中得出参数
- 4) CPU 从状态寄存器读出各个设备的状态，IO 逻辑会写入状态寄存器告诉 CPU 各个设备的状态
- 5) 设备要反馈给 IO 逻辑器设备的忙碌、空闲状态

寄存器编址

内存映像 vs 寄存器独立编址



IO 控制方式

程序直接控制方式、中断驱动方式、DMA 方式、通道控制方式

程序直接控制方式 (keyword: 轮询)

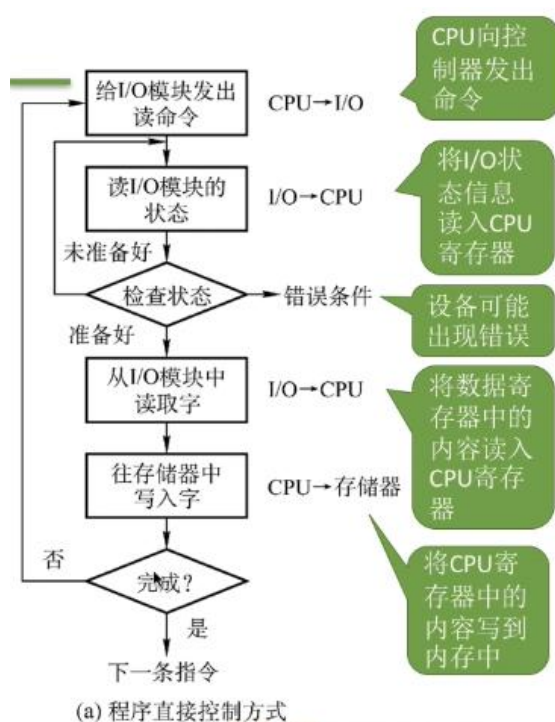
1. 完成一次读/写操作

- CPU 通过控制线发出读指令，IO 控制器根据 CPU 要求启动设备，并将状态寄存器设为 1
- 设备比 CPU 速度慢很多，在设备没有完成 IO 之前，CPU 不断轮询状态寄存器的状态
- 设备准备好数据之后，将数据传给 IO 控制器，并报告自身状态
- IO 逻辑器将输入的数据放到数据寄存器，并且将状态寄存器改为 0
- 当 CPU 发现状态寄存器为 0 时，将数据寄存器的数据读入 CPU 的寄存器中，再把 CPU 寄存器中的内容放入内存

CPU 需要不断轮询检查，CPU 干预很频繁

数据的传输单位为一个字（字符设备）

数据的流向：读操作：IO → CPU → 内存



CPU 寄存器 → 内存：e.g.程序中将变量进行赋值

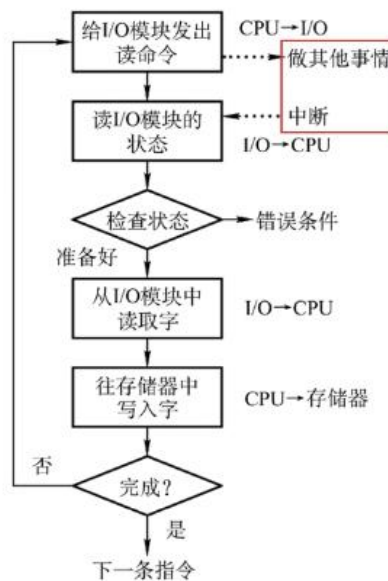
优点：实现简单。

缺点：CPU 和 IO 设备只能串行工作，CPU 需要一直轮询检查，长期处于忙等状态，利用率低

中断驱动方式

引入**中断机制**，在 CPU 发出读写指令之后，可将等待 IO 的进程阻塞，先切换到别的进程执行。当 IO 完成后会向 CPU 发出**中断信号**，CPU 收到中断信号后，保存当前执行进程的**运行环境**，转去执行**中断处理**，在处理中断的过程中，CPU 从 IO 控制器（数据寄存器）中**一个字（字符设备）**的数据传送到 CPU 寄存器，再写入主存。之后 CPU 恢复等待 IO 的

进程（或其他进程）的运行环境。继续执行

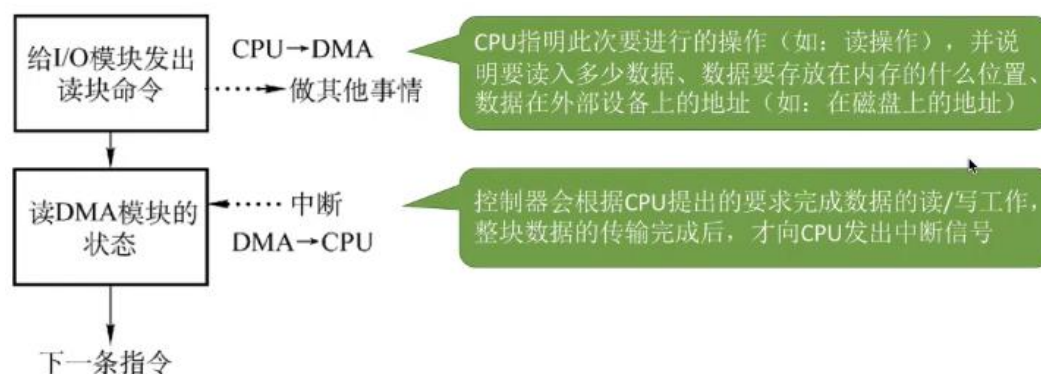


优点：可将 CPU 与 IO 设备并行工作，CPU 利用率得到提升

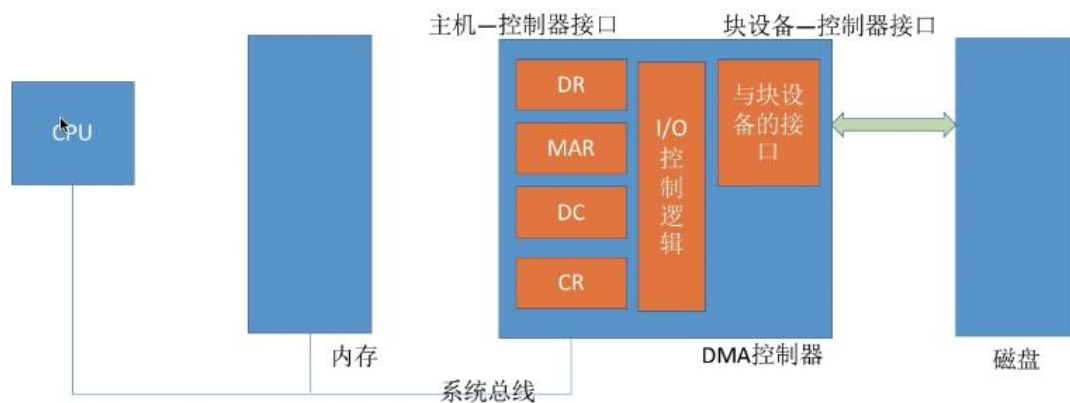
缺点：每次传输只能传输一个字，会发生大量的中断，频繁的中断处理会消耗较多的 CPU 时间。

DMA 方式（块设备）

- ①、数据的传输单位是“块”，不再是一个字一个字地传送
- ②、数据的流向是从设备直接放入内存，不再需要 CPU
- ③、仅在传送一个或多个数据块的开始和结束时，才需要 CPU 干预



DMA 也是一种 IO 控制器（类似）



DR (数据寄存器): 暂存从设备到内存，或从内存到设备

MAR (内存地址寄存器): 在输入时，MAR 表示数据应放到内存中的什么地址

DC (数据计数器): 表示剩余要读/写的字节数

CR (命令/状态寄存器): 用于存放 CPU 发来的 IO 命令，或设备的状态信息

DMA 在**读入数据**时也是一个字一个字读入，读入的数据放在 DR 中再从 DR 写入到内存中

CPU 仅在传送**一个或多个数据块**的开始和结束时，才需要 CPU 干预

每次读写一个或多个块(**每次读写的只能是连续的多个块，且这些块读入内存后在内存中也必须是连续的**)

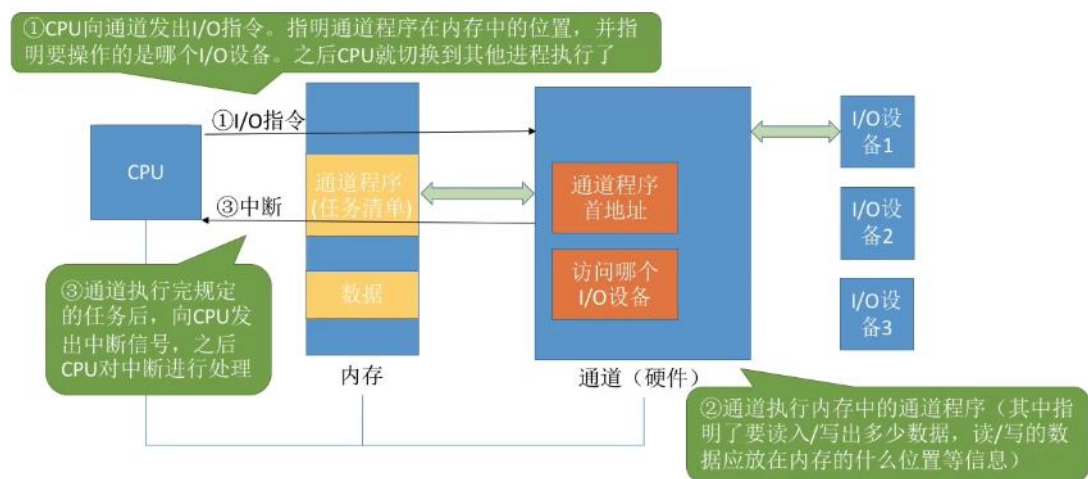
数据流向：IO → 内存

优点：提高了数据传输的效率；传输的过程不需要 CPU；性能得到提升

缺点：CPU 每发一次 IO 指令，只能读写一个或多个连续的数据块

通道（弱版 CPU）

通道是一种硬件



CPU 干预率：极低通道会根据 CPU 的指示执行相应的通道程序，只有完成一组数据块的读

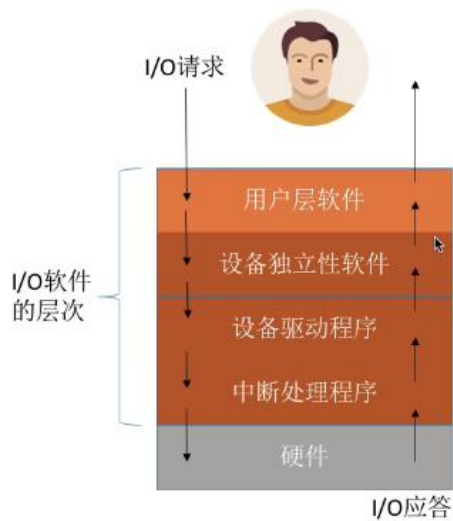
写后才需要发出中断信号，请求 CPU 干预

数据的传输：**每次读写一组数据快**

数据的流向：IO 设备 → 内存

	完成一次读/写的过程	CPU干预频率	每次I/O的数据传输单位	数据流向	优缺点
程序直接控制方式	CPU发出I/O命令后需要不断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的优点都是解决了上一阶段的最大缺点。总体来说，整个发展过程就是要尽量减少CPU对I/O过程的干预，把CPU从繁杂的I/O控制事务中解脱出来，以便更多地去完成数据处理任务。
中断驱动方式	CPU发出I/O命令后可以其他事，本次I/O完成后设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以其他事，本次I/O完成后DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方式	CPU发出I/O命令后可以其他事。通道会执行通道程序以完成I/O，完成后通道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

IO 软件的层次



用户层软件：实现了与用户交换的接口（库函数）

设备独立性软件：①、用户层软件将用户请求翻译成格式化的 IO 请求，并通过“系统调用”

请求操作系统内核的服务（window API）

②、设备的保护：不同用户对各个文件的访问权限是不一样的，同时对设备的访问权限也不同

③、差错处理

④、设备的分配与回收：临界资源

⑤、数据缓冲区管理

⑥、建立逻辑设备名（**用户看到的设备名**）到物理设备名的映射关系：根据设备类型调用相应的驱动程序；通过逻辑设备表（LUT）

逻辑设备名	物理设备名	驱动程序入口地址
/dev/打印机1	3	1024
/dev/打印机2	5	2046
...	...	...

I：整个系统只设置一张 LUT；仅限于单用户 II：为每个用户设置一张 LUT

设备驱动程序：对硬件设备的具体控制；根据不同 IO 设备不同的硬件特性，需要提供各自

的设备驱动程序

中断处理程序：当 IO 完成任务时，IO 控制器会发送出一个中断信号，系统根据中断信号类型找到相应的中断处理程序并执行

IO 核心子系统



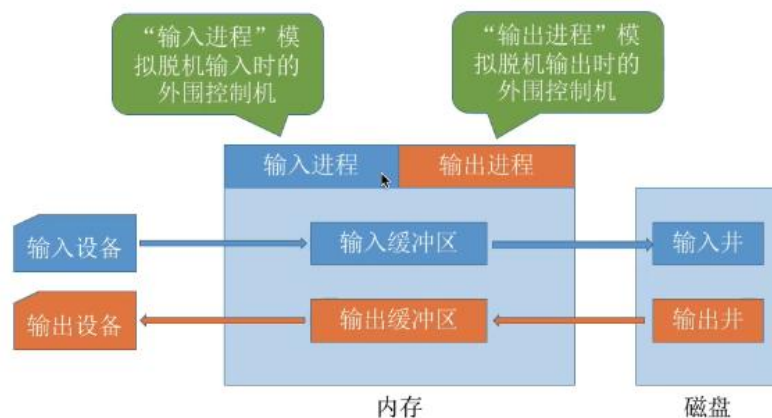
IO 调度：用某种算法来确定一个好的顺序来处理各个 IO 请求（如磁盘调度）

设备保护：根据 FCB → 设备是特殊的文件

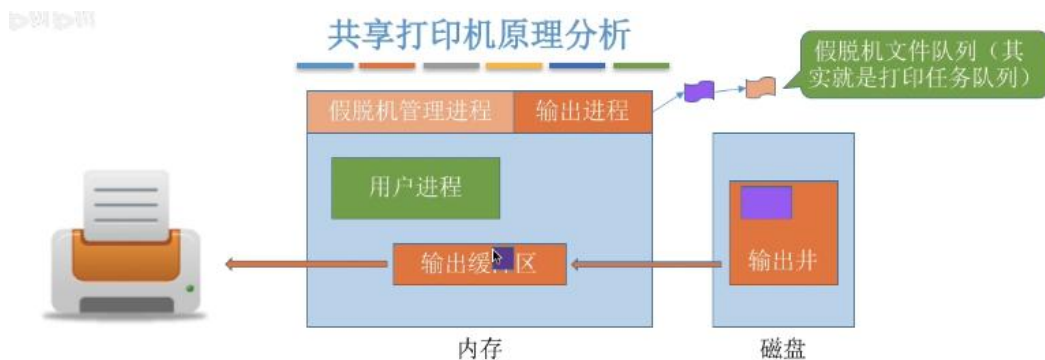
假脱机技术 SPOOLING

批处理阶段引入了脱机输入输出技术（用磁带完成）

假脱机技术：用软件的方式模拟脱机技术



共享打印机原理



- ①、在磁盘输出井中为进程申请一个空闲缓冲区，并将要打印的数据送入其中
- ②、为用户进程申请一张空白的打印请求表，并将用户的打印请求填入表中（用来说明用户的打印数据存放位置等信息），再将该表挂到假脱机文件队列中
- ③、当打印机空闲时，输出进程会从文件队列的对头取出一张打印请求表，并根据表中的要求将要打印的数据从输出井传送到输出缓冲区，在输出到打印机进行打印

设备的分配与回收

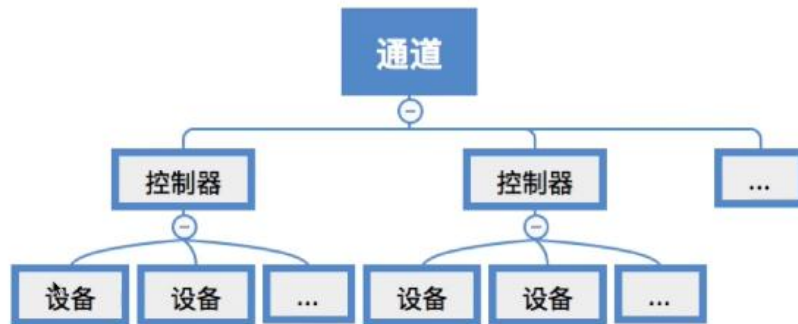
设备的固有属性：**独占设备**、**共享设备**、**虚拟设备（SPOOLING 技术）**

设备分配算法：①、**安全分配方式**（为进程分配一个设备后就将进程阻塞，本次 IO 完成后被唤醒） ②、**不安全分配方式**（进程不会被阻塞，可以继续执行，之后还可以发出新的 IO 请求；只有某个 IO 请求得不到满足时才将其阻塞）

静态分配：进程运行前为其分配全部所需资源，运行结束后归还资源

动态分配：进程在运行过程中动态申请设备资源

“设备、控制器、通道”之间的关系：



一个通道可控制多个设备控制器，每个设备控制器可控制多个设备。

系统为每个设备配置一张 DCT (设备控制表)

设备控制表 (DCT)	
设备类型	如：打印机/扫描仪/键盘
设备标识符	即物理设备名，系统中的每个设备的物理设备名唯一
设备状态	忙碌/空闲/故障...
指向控制器表的指针	每个设备由一个控制器控制，该指针可找到相应控制器的信息
重复执行次数或时间	当重复执行多次I/O操作后仍不成功，才认为此次I/O失败
设备队列的队首指针	指向正在等待该设备的进程队列（由进程PCB组成队列）

控制器控制表 (COCT)：每个设备控制器都会对应一张 COCT。

控制器控制表 (COCT)	
控制器标识符	各个控制器的唯一ID
控制器状态	忙碌/空闲/故障...
指向通道表的指针	每个控制器由一个通道控制，该指针可找到相应通道的信息
控制器队列的队首指针	指向正在等待该控制器的进程队列（由进程PCB组成队列）
控制器队列的队尾指针	

通道控制表 (CHCT)：

通道控制表 (CHCT)	
通道标识符	各个通道的唯一ID
通道状态	忙碌/空闲/故障...
与通道连接的控制器表首址	可通过该指针找到该通道管理的所有控制器相关信息 (COCT)
通道队列的队首指针	指向正在等待该通道的进程队列（由进程PCB组成队列）
通道队列的队尾指针	

系统设备表 (SDT)：记录了**系统中全部设备**的情况，每个设备对应一个表目



设备分配的步骤：SDT → DCT(设备控制表) → COCT(控制器控制表) → CHCT(通道控制表)

- ①、根据进程请求的物理设备名查找 SDT
- ②、根据 SDT 找到 DCT，若设备忙碌则将进程 PCB 挂到设备等待队列，不忙则分配
- ③、根据 DCT 找到 COCT 判断控制器状态
- ④、根据 COCT 找到 CHCT 判断通道状态

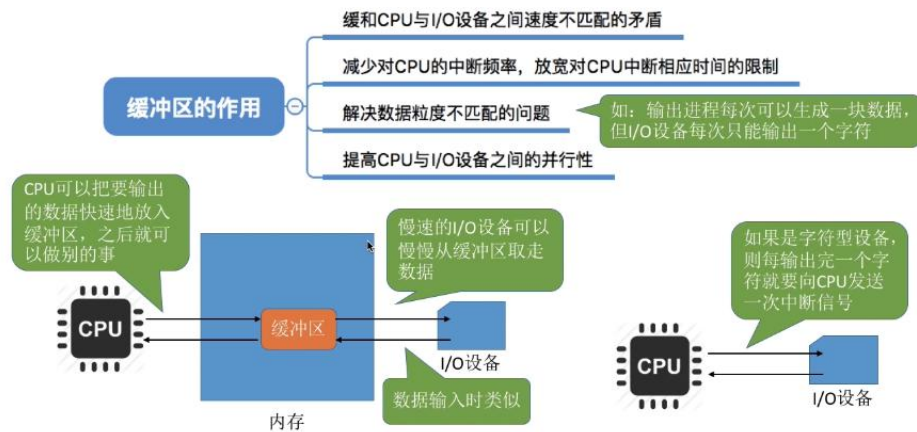
缺点：用户需要提供系统设备名；若换了一个物理设备，则程序无法运行、若进程请求的物理设备正在忙碌，即使系统中还有同类型的设备、进程也必须阻塞等待。

改进方法：建立逻辑设备名与物理设备名映射（设备类型）

- ①根据进程请求的**逻辑设备名**查找SDT（注：用户编程时提供的逻辑设备名其实就是“设备类型”）
- ②查找SDT，找到用户进程指定类型的、并且空闲的设备，将其分配给该进程。操作系统在**逻辑设备表（LUT）**中新增一个表项。
- ③根据DCT找到COCT，若控制器忙碌则将进程PCB挂到控制器等待队列中，不忙碌则将控制器分配给进程。
- ④根据COCT找到CHCT，若通道忙碌则将进程PCB挂到通道等待队列中，不忙碌则将通道分配给进程。



缓冲区管理

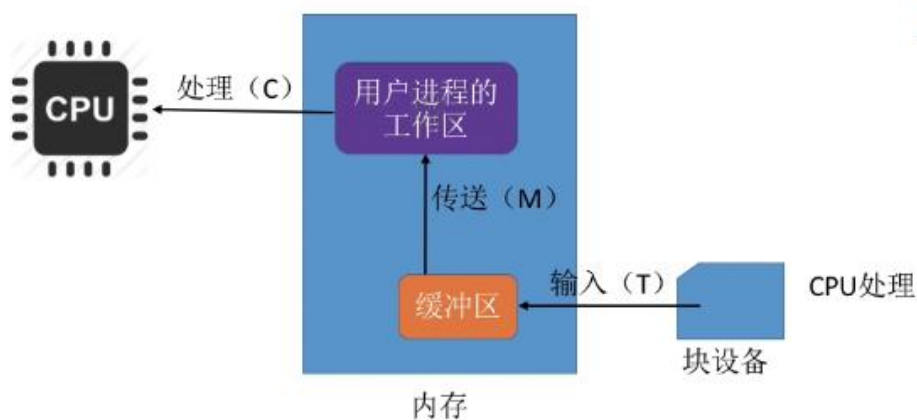


单缓冲（通信管道）

操作系统会在主存中为其分配一个缓冲区（通常为一个块的大小）

注意：当缓冲区数据非空时，不能忘缓冲区冲入数据，只能从缓冲区把数据传出；当缓冲区

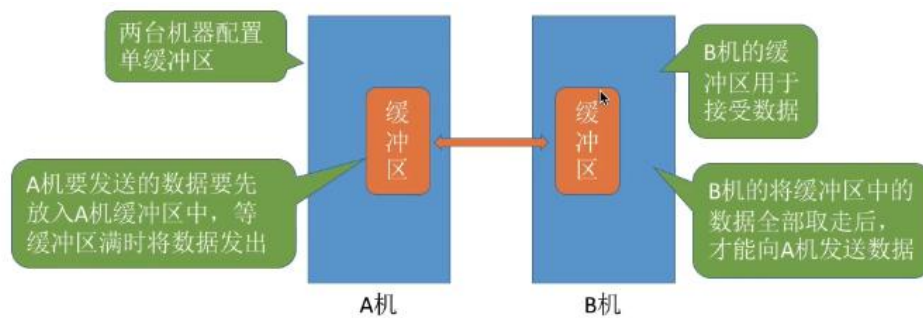
为空时，可以往缓冲区冲入数据，但必须叫缓冲区充满以后，才能从缓冲区把数据传出。



双缓冲

操作系统会在主存中为其分配两个缓冲区

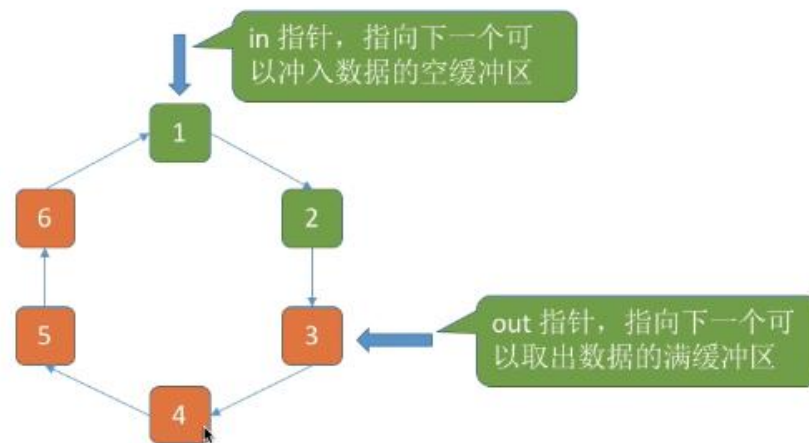
两台机器之间通信时，可以配置缓冲区用于数据的发送和接受



如果两个互相通信的机器只设置单缓冲区，在任意时刻只能单向传输

管道通信中的“管道”其实就是缓冲区。要实现数据的双向传输，必须设置两个管道

循环缓冲区



缓冲池

缓冲池由系统中公用的缓冲区组成；这些缓冲区按使用状况可以分为：空缓冲队列、装满输入数据的缓冲队列、装满输出数据的缓冲队列

另外，根据一个缓冲区在实际运算中扮演的功能不同又可以分为：收容输入数据的工作缓冲区（hin）、提取输入数据的工作缓冲区（sin）、收容输出数据的工作缓冲区（hout）、提取输出数据的工作缓冲区（sout）

